The Synapse Cookbook

Contents

Introduction

The Synapse Cookbook — articles about network programming with the Ararat Synapse TCP/IP library for Object Pascal.

This cookbook began as a community wiki that later went offline; version 0.1 was reconstructed from the Internet Archive. It was always meant as a practical, example-first companion to the library — not a formal reference.

Brand new to this? Start with **Getting Started** — it assumes no networking knowledge and has you talking to the internet in a few minutes. From there, the **Appendix — Synapse in Two Hours** goes deeper (HTTP, a threaded server, TLS). Readers who want to understand *why* the library is shaped the way it is should read **The Architecture of Synapse** — it is the heart of this cookbook, and the reason the recipes are so short.

> **Status (2026 revival):** this Markdown edition is being migrated, section-by-section, from the recovered 0.1 text (`releases/synapsecookbook-0.1.txt`). Code examples are to be re-verified against current FPC + Ararat Synapse.

About Synapse

Ararat Synapse is a lightweight, blocking (and optionally non-blocking) TCP/IP library for Object Pascal — Delphi, Kylix, and Free Pascal / Lazarus — by **Lukas Gebauer**. It provides socket classes and a range of client protocol implementations without visual components or external dependencies. This cookbook teaches how to use it; all credit for the library belongs to its author.

Getting Started

New to Synapse — or to network programming in general? Start here. In a few minutes you will have a program that talks to the internet. No prior networking knowledge assumed.

A 60-second mental model

Networking is simpler than it sounds:

- A **host** is a computer on a network, named by an address like `example.com` or `93.184.216.34`. Turning a name into an address is **DNS**.
- A **port** is a numbered door on that host. Web servers listen on port 80 (443 for HTTPS), mail on 25, and so on.
- A **socket** is your end of a connection to `host:port`. You open it, send bytes, receive bytes, close it. That's it.

A **client** opens connections (your browser); a **server** waits for them (the web server). Synapse does both. Everything else in this book is detail on top of those four words: host, port, socket, bytes.

What you need

1. **A Pascal compiler.** Free Pascal (FPC) with or without the Lazarus IDE, or Delphi. All are supported.
2. **Ararat Synapse** itself — it is a set of source units, nothing to compile or install as a library. Get it from github.com/geby/synapse (or the homepage) and unzip it somewhere, e.g. `~/synapse`.

That's the whole setup. Synapse has **no external dependencies** for the basics (see Rolling its own crypto); you only add an SSL unit later if you want TLS.

Hello, network — fetch a web page

The friendliest possible start. This downloads a page and prints it:

```
program hello_net;
uses httpsend;              // Synapse's HTTP client
var
  page: string;
begin
  if HttpGetText('http://example.com/', page) then
    WriteLn(page)
  else
```

```
    WriteLn('request failed');
end.
```

`HttpGetText` is a one-line convenience: give it a URL and a string, it fills the string with the response body and returns `True`/`False`. You just made an HTTP request. (For HTTPS, add one unit — see below.)

Hello, socket — the raw version

To *see* the socket underneath, connect to a host and read what it says. Here we open a plain TCP connection and read the greeting a server sends:

```
program hello_socket;
uses blcksock;              // Synapse's socket classes
var
  sock: TTCPBlockSocket;
begin
  sock := TTCPBlockSocket.Create;
  try
    sock.Connect('example.com', '80');        // host, port (as strings)
    if sock.LastError <> 0 then
    begin
      WriteLn('connect failed: ', sock.LastErrorDesc);
      Exit;
    end;
    sock.SendString('GET / HTTP/1.0' + CRLF + CRLF);   // ask for the page
    WriteLn(sock.RecvString(3000));           // read one line, wait up to 3 s
  finally
    sock.Free;
  end;
end.
```

Two habits to notice, because they run through all of Synapse:

- **Ports are strings** (`'80'`), not integers — Synapse also accepts service names.
- **Check `LastError` after operations.** Synapse *reports* problems rather than raising exceptions, which keeps your code a straight line. `LastError = 0` means success; `LastErrorDesc` is the human-readable reason. This is the whole error model — no try/except gymnastics required.

Compiling it

With FPC on the command line — point it at the Synapse source with `-Fu`:

```
fpc -Fu~/synapse hello_net.pas
./hello_net
```

With Lazarus — Project → Project Options → Compiler Options → Paths, and add `~/synapse` to *Other unit files (-Fu)*. Then Run.

That's it — the same `-Fu path` idea, once per project.

Going encrypted (one line)

Want HTTPS instead of HTTP? Add a single SSL unit to `uses` and use an `https://` URL — nothing else changes:

```
uses httpsend, ssl_openssl;        // that extra unit is the entire TLS wiring
...
HttpGetText('https://example.com/', page);
```

That one line is Synapse's SSL plugin seam at work — TLS is a plugin you switch on by including it.

Where to go next

- **Synapse in Two Hours** — the fuller tutorial: HTTP GET/POST, a threaded echo *server*, TLS.
- **The Architecture of Synapse** — *why* it is shaped this way; short and worth it.
- **Recipes** — copy-paste solutions to common tasks (send mail, download a file, resolve a name, ping a host, talk to a serial port).

You now know the whole shape: open a socket to `host:port`, send and receive bytes, check `LastError`, close. Everything else is a protocol class doing that for you.

The Architecture of Synapse

Before the recipes, the design — because Synapse's design is the reason the recipes are so short. Ararat Synapse is a masterclass in doing a great deal with very little: no visual components, no framework, no external runtime, and (for the essentials) no external libraries at all. Just Object Pascal, a clean set of classes, and a few genuinely clever seams.

It was written in an era when **`async` did not exist** as a language feature and preemptive **timeslicing was still being researched and proven**. Synapse's answer was not to wait for the future — it was to build a *blocking* socket library with precise timeouts, designed from the start to be driven by threads. That decision aged remarkably well: the code is linear and readable (no callback spaghetti, no state machines), and concurrency is just "run another thread."

Four ideas carry the whole library:

1. **One blocking socket, every transport.** `TBlockSocket` and its descendants put TCP, UDP, raw IP — and even **serial ports** — behind a single blocking, timeout-driven API. Learn one class, use them all. → *Blocking model & threads*
2. **A thin OS seam.** `synsock` isolates every platform difference (Winsock, Unix libc, Windows CE) behind one internal interface, so the socket classes above it are written **once** and compile everywhere. → *Cross-platform: synsock*
3. **TLS as a swappable plugin, bolted on mid-stream.** SSL/TLS is *not* baked into the socket. A single global metaclass (`SSLImplementation`) is overridden simply by which `ssl_*` unit you place in `uses`. Any block socket can be upgraded to TLS on a live connection (STARTTLS-style). This is the library's most elegant hack. → *The SSL plugin seam*
4. **Batteries included, dependencies excluded.** MD5, MD4, SHA-1, HMAC, Base64, quoted-printable, DES/3DES — all **hand-rolled** in `synacode`/`synacrypt`, so authentication and encoding work with zero external libraries. → *Rolling its own crypto*

On top of those four seams sit the protocol clients — SMTP, POP3, IMAP, HTTP, FTP, NNTP, LDAP, SNMP, SNTP, DNS, ping, telnet — each a thin, readable class that speaks its protocol over a `TBlockSocket`. That is the whole shape of the library: **thin, orthogonal layers, each doing one thing well.** Once you see the seams, the rest of this cookbook is just usage.

> **Why celebrate the architecture at all?** Because it teaches transferable design: push platform differences down into a seam, express optional features (TLS) as pluggable metaclasses instead of flags, keep the I/O model simple (blocking + threads) and let the OS scheduler do the hard part. These lessons outlive Object Pascal.

The Blocking Model & Threads

Synapse is a **blocking** socket library. That is a deliberate design choice, not a limitation, and understanding it is the key to using Synapse well.

The era it came from

When Synapse was designed, `async/await` did not exist as a language construct, and reliable preemptive **timeslicing** of threads was still being researched and proven. Two schools competed for network I/O:

- **Non-blocking / select loops** — one thread juggling many sockets via a state machine. Powerful, but the code becomes a tangle of callbacks and saved state.
- **Blocking + threads** — each connection handled by straight-line code in its own thread; the OS scheduler interleaves them.

Synapse bet on the second, *provided* you had dependable timeouts and threads. That bet aged well: the code reads top-to-bottom like a description of the protocol, and scaling to many connections is "spawn more threads."

What "blocking with timeouts" means

Every read/write takes a timeout in milliseconds. A call returns when the operation completes **or** the timeout elapses — it never hangs forever:

```
sock.Connect(host, port);            // blocks until connected or ConnectTimeout
line := sock.RecvString(5000);       // blocks up to 5 s for a CRLF-terminated line
sock.SendString('HELLO' + CRLF);     // blocks until sent
if sock.LastError <> 0 then ...      // every call reports status, no exceptions-as-flow
```

The timeout is what makes blocking *safe*: no runaway waits, and you can build keep-alive and cancellation on top of it. `CanRead(timeout)` / `CanWrite(timeout)` let you poll readiness without committing to a read — the ingredients of a select loop, if you want one, without leaving the blocking API.

Why this makes concurrency simple

Because each socket's code is linear and self-contained, concurrency is just threading:

```
type
  TConnThread = class(TThread)
    FSock: TTCPBlockSocket;
```

```
    procedure Execute; override;   // straight-line protocol code here
  end;
```

One connection, one thread, one readable `Execute`. No shared state machine, no re-entrancy puzzles per socket. This is exactly the model that `pastella` (and Visual Synapse's servers) built on: a listener thread accepts, then hands each connection to its own handler thread, each speaking the protocol in plain sequential code.

> **The transferable lesson:** a simple I/O model plus the OS scheduler often beats a clever single-threaded event loop — in *readability* always, and in throughput more often than people expect. Synapse chose boring-and-correct, and it is still in service two decades later.

Practical notes

- Give every socket a **thread of its own** for servers; never share one `TBlockSocket` across threads.
- Choose timeouts per protocol step; they double as your liveness policy.
- Check `LastError`/`LastErrorDesc` after operations — Synapse reports rather than throws, keeping the linear flow intact.

Cross-Platform: the synsock Seam

Synapse runs on Windows, Linux, other Unixes, and Windows CE, across Delphi, Kylix, and Free Pascal — and the socket classes are written **once**. The trick is a single thin unit that absorbs every platform difference: `synsock`.

The idea

Operating systems disagree about sockets: Windows has Winsock (`ws2_32.dll`, an explicit `WSAStartup`/`WSACleanup` lifecycle, `SOCKET` handles), Unix has BSD sockets in libc (file descriptors, `errno`), Windows CE has its own quirks. If those differences leak upward, every socket class needs `{$IFDEF}` clutter.

`synsock` is the **seam** that stops the leak. It exposes one internal socket API — the calls `blcksock` needs — and implements it per platform behind conditional compilation:

```
unit synsock;
{$IFDEF WINCE}
  ...windows CE backend...
{$ELSE}
  {$I sslinux.inc}   // unix/linux backend, pulled in by include
{$ENDIF}
```

Above `synsock`, `blcksock.pas` is platform-agnostic. It calls the seam's functions and never names a specific OS. Port to a new platform → implement the seam once, and every socket class and protocol client comes along for free.

Why a seam beats scattered `IFDEF`s

- **One place to be right.** Platform bugs live in `synsock`, not smeared across 50 units.
- **The layer above stays readable.** `TTCPBlockSocket` reads like sockets, not like a compatibility matrix.
- **New platforms are additive.** You extend the seam; you don't touch the library on top of it.

This is the same principle as a Hardware Abstraction Layer, or a driver interface: *define the narrow contract the upper layers need, then satisfy it per target.* Synapse applied it to the OS socket API long before "platform abstraction layer" was a buzzword — and it is why one Pascal codebase quietly serves every target at once.

Companion seams

`synsock` handles sockets; two sibling units finish the portability story:

- **`synafpc`** — smooths *compiler* differences (Delphi vs Kylix vs Free Pascal): types, RTL calls, and dialect quirks, so the same source compiles on each.
- **`synachar` / `synaicnv`** — charset conversion, so text protocols behave the same regardless of the host's locale.

Together: `synsock` abstracts the OS, `synafpc` abstracts the compiler, `synachar` abstracts the locale. Three narrow seams, and the rest of Synapse is written once for all of them.

The SSL Plugin Seam

This is Synapse's most elegant idea, and worth studying even if you never write Pascal: **TLS is not a feature of the socket — it is a plugin selected at link time and attached to a live connection.**

The problem it solves

A naive socket library bakes SSL in: `#ifdef` around OpenSSL, a `UseSSL: Boolean` flag, and the library now *depends* on one TLS implementation forever. Change your mind (OpenSSL → SChannel → a hardware library) and you rewrite the socket.

Synapse refuses that. The block socket knows *nothing* about any specific TLS library. It only knows an abstract contract.

How it works

In `blcksock.pas`:

```
type
  TCustomSSL = class;                 // abstract TLS contract
  TSSLClass = class of TCustomSSL;    // a metaclass (class reference)

var
  SSLImplementation: TSSLClass = TSSLNone;   // global default: "no TLS"
```

Three moving parts:

1. **`TCustomSSL`** — an abstract base class defining what a TLS provider must do (`Connect`, `Accept`, `Shutdown`, read/write, certificate handling…).
2. **`TSSLNone`** — a null-object implementation. With no TLS unit included, this is what you get: a socket that simply passes bytes through, unencrypted.
3. **`SSLImplementation`** — a single global **metaclass variable**. Every socket creates its TLS object from it:

   ```
   FSSL := SSLImplementation.Create(self);   // in the socket constructor
   ```

The trick: each concrete provider unit **sets that global in its own initialization**. `ssl_openssl.pas`, `ssl_schannel.pas`, `ssl_cryptlib.pas`, `ssl_streamsec.pas`, `ssl_libssh2.pas`, and more each do, in effect:

```
initialization
  SSLImplementation := TSSLOpenSSL;   // (or TSSLSChannel, TSSLCryptLib, …)
```

So **you choose your TLS backend simply by which unit appears in `uses`.** No flag, no factory call, no configuration:

```
uses blcksock, ssl_openssl;   // <-- this line is the entire choice
```

Include `ssl_openssl` and every socket transparently gains OpenSSL. Swap it for `ssl_schannel` and the *same socket code* now uses Windows' native TLS. Include nothing and you get `TSSLNone` — plaintext, zero TLS dependency. The socket classes never change.

TLS "midway" — upgrading a live socket

Because TLS is a layer attached to an already-open `TBlockSocket`, you can start a connection in plaintext and **upgrade it to TLS mid-stream** — exactly what protocols like SMTP `STARTTLS`, IMAP, FTP-over-TLS, and LDAP need:

```
sock.Connect(host, '25');        // plaintext
// ... negotiate STARTTLS in the protocol ...
sock.SSLDoConnect;               // upgrade this live socket to TLS
```

`SSLDoConnect` hands the existing connection to the plugin, which performs the handshake over it. The protocol clients (`smtpsend`, `imapsend`, …) use exactly this to implement opportunistic encryption.

Why it is beautiful

- **Zero coupling.** The socket depends on an *interface* (`TCustomSSL`), never a library. OpenSSL is not a dependency of Synapse — it is a dependency of *your choice to include `ssl_openssl`*.
- **Link-time polymorphism, no cost.** The selection is a metaclass assignment at unit initialization — no runtime factory, no config parsing.
- **Extensible by outsiders.** Anyone can add a new TLS provider by writing one unit that descends `TCustomSSL` and sets `SSLImplementation`. The core never learns their name. (The ~20 `ssl_*` units in the tree are exactly this, accreted over 20 years.)
- **Graceful default.** No TLS unit → `TSSLNone` → it still compiles and runs, just unencrypted. Nothing to stub.

This is the null-object pattern and link-time strategy selection, done in a few lines of Object Pascal, years before those names were common vocabulary.

Rolling Its Own Crypto & Encoding

Open a mail client library today and it pulls in a crypto dependency for the message digests that authentication needs. Synapse pulls in **nothing** — the hashes, HMACs, ciphers, and transfer encodings are all hand-written in Object Pascal, in `synacode` and `synacrypt`. For the essentials, Synapse has *no external dependency at all*.

What's in the box

From `synacode.pas` — digests, MACs, and encodings, all from scratch:

```
function MD5(const Value: AnsiString): AnsiString;
function MD4(const Value: AnsiString): AnsiString;
function SHA1(const Value: AnsiString): AnsiString;
function HMAC_MD5(Text, Key: AnsiString): AnsiString;
function HMAC_SHA1(Text, Key: AnsiString): AnsiString;
function MD5LongHash(const Value: AnsiString; Len: integer): AnsiString;
function SHA1LongHash(const Value: AnsiString; Len: integer): AnsiString;
```

plus Base64, quoted-printable, UU/XX, and the other MIME transfer encodings the mail and HTTP clients need.

From `synacrypt.pas` — symmetric ciphers (DES, 3DES, and friends) implemented in pure Pascal, for the protocols and SASL mechanisms that require them.

Why roll your own here

In the era Synapse was built, you could not assume a crypto library was present and portable across Windows, Linux, Kylix, and CE. The choices were: depend on something that might not be there, or **implement the handful of algorithms you actually need**, portably, once. Synapse chose the second — and it is exactly why `pop3send`, `smtpsend`, `imapsend`, and the SASL/APOP/CRAM-MD5 authentication paths *just work* with nothing else installed.

Note the deliberate scope: these are the digests and ciphers protocols require for **authentication and encoding**, not a general-purpose crypto suite. For transport encryption (TLS), Synapse does the opposite and delegates to a best-in-class library through the SSL plugin seam — because reimplementing TLS would be reckless, while reimplementing MD5 is a weekend and removes a dependency forever.

The lesson (and the caveat)

- **The lesson:** know which wheels are worth reinventing. A 200-line MD5 that erases a fragile, non-portable dependency is a *good* trade. Synapse reinvents precisely the small, stable, spec-frozen algorithms — and delegates the large, evolving, security-critical one (TLS).
- **The caveat for today:** MD5, MD4, SHA-1, and single-DES are here because the *protocols* still specify them (APOP, CRAM-MD5, legacy auth), not because they are secure choices for new designs. Use them to speak the protocols that demand them; do not reach for them as your application's security primitives. Modern strength lives behind the TLS plugin.

Read `synacode.pas` end to end at least once — it is a compact, readable tour of how these algorithms actually work, unobscured by macros or assembly. As teaching material, it is worth as much as the cookbook chapters that use it.

The Socket Class Family

Synapse's socket layer is one file — `blcksock.pas` — and one idea: **a single blocking base class, `TBlockSocket`, specialised by protocol.** Everything you send or receive, in any protocol, over TCP, UDP, ICMP, or a raw socket, goes through a descendant of that base. Learn the base once and every socket in the tree behaves the same way.

The hierarchy

The real tree, straight from `blcksock.pas`:

```
TBlockSocket                    (the blocking core: buffers, timeouts, LastError)
├── TSocksBlockSocket           (adds SOCKS4/4a/5 proxy plumbing — do not use directly)
│   ├── TTCPBlockSocket         (TCP: connect/listen/accept, SSL/TLS plugin, HTTP-
tunnel)
│   └── TDgramBlockSocket       (datagram base: Send/Recv redirected to the *To/*From forms)
│       ├── TUDPBlockSocket     (UDP: unicast, broadcast, multicast)
│       └── TICMPBlockSocket    (raw ICMP — needs privilege)
├── TRAWBlockSocket             (raw IP socket — needs privilege)
├── TPGMMessageBlockSocket      (PGM reliable multicast, message mode)
└── TPGMStreamBlockSocket       (PGM reliable multicast, stream mode)
```

Two things are worth reading off that diagram:

- **SOCKS lives in the *middle* of the tree, not at the top.** `TBlockSocket` itself knows nothing about proxies. `TSocksBlockSocket` inserts the proxy machinery, and both the TCP and the datagram families inherit from it — so proxy support comes "for free" to the sockets that can use it, while `TRAWBlockSocket` (which cannot) simply skips that layer by descending straight from `TBlockSocket`.
- **`TDgramBlockSocket` is a helper, not a socket you instantiate.** It exists only to redirect the connection-style `SendBuffer`/`RecvBuffer` onto the connectionless `SendBufferTo`/`RecvBufferFrom`, so datagram code can use the same high-level helpers (`SendString`, `RecvPacket`) as stream code. You create `TUDPBlockSocket` or `TICMPBlockSocket`; you never create `TDgramBlockSocket` or `TSocksBlockSocket` directly — their own source comments say as much ("Do not use this class directly").

What unifies them

Every socket in the family shares one contract, defined on `TBlockSocket`:

A blocking API with timeouts. Reads and writes block until they complete or a millisecond timeout elapses; nothing hangs forever. See The Blocking Model & Threads for why that choice ages so well.

The `LastError` model — report, don't throw. After any operation you check `LastError` (an integer socket error code) and `LastErrorDesc` (a human string). Exceptions are opt-in: set `RaiseExcept := True` and Synapse raises `ESynapseError` on failure; leave it `False` (the default) and your protocol code stays linear.

```
sock.Connect('example.org', '80');
if sock.LastError <> 0 then
  Writeln('connect failed: ', sock.LastErrorDesc);
```

Two tiers of I/O on every socket. Low-level `SendBuffer`/`RecvBuffer` move raw bytes; high-level helpers built on an internal `LineBuffer` (`SendString`, `RecvString`, `RecvPacket`, `RecvTerminated`, `SendInteger`, `RecvBlock`, the stream methods…) do the framing for you. You can mix the two tiers freely on the same socket because the high-level side always drains `LineBuffer` first.

Shared knobs. Timeouts (`SetTimeout`, `ConnectionTimeout`), readiness polls (`CanRead`, `CanWrite`), byte counters (`RecvCounter`, `SendCounter`), bandwidth throttles (`MaxSendBandwidth`, `MaxRecvBandwidth`), a soft-abort `StopFlag`, and IPv4/IPv6 selection via the `Family` property — all live on the base and work identically for TCP, UDP, and the rest.

Choosing one

You want to…	Use
Speak a stream protocol (HTTP, SMTP, a custom line protocol)	`TTCPBlockSocket`
Wrap that stream in TLS/SSL or SSH	`TTCPBlockSocket` + an `ssl_*` plugin
Send/receive datagrams, broadcast, or multicast	`TUDPBlockSocket`
Send ICMP (ping-style) packets	`TICMPBlockSocket`
Craft raw IP packets yourself	`TRAWBlockSocket`
Reach any of the above through a SOCKS proxy	set `SocksIP` on the TCP/UDP socket you already have

The rest of this chapter walks the four you will actually reach for: the base class, TCP, UDP, and the SOCKS proxy layer that quietly sits between them.

TBlockSocket — the base class

`TBlockSocket` is the root of the whole family. You almost never instantiate it directly — you create a `TTCPBlockSocket` or `TUDPBlockSocket` — but *every* method and property you use lives here or is overridden from here. This chapter is the reference you come back to.

The lifecycle

A Synapse socket has a simple, explicit life:

`Create` ⟶ `Connect / Bind+Listen` ⟶ `Send/Recv …` ⟶ `CloseSocket` ⟶ `Free`

- **`constructor Create`** — makes the object and loads the socket library, but does *not* create an OS socket yet. There is also `CreateAlternate(Stub: string)` for loading a non-default socket provider.
- **`Connect(IP, Port: string)`** / **`Bind(const IP, Port: string)`** — the OS socket is created lazily on first use. Both take strings, numeric or symbolic: `'192.168.0.1'` or `'cosi.nekde.cz'`, `'80'` or `'http'`.
- **`CloseSocket`** — closes the OS handle. It is `virtual` (TCP overrides it to shut down TLS first). The destructor calls it for you, so `Free` is enough, but closing explicitly when you are done is good manners.
- **`AbortSocket`** — the hard version: abandon any work and destroy the handle.

```
var sock: TTCPBlockSocket;
begin
  sock := TTCPBlockSocket.Create;
  try
    sock.Connect('example.org', '80');
    if sock.LastError = 0 then
      { ...talk... };
  finally
    sock.CloseSocket;
    sock.Free;
  end;
end;
```

The `LastError` model

Synapse **reports** rather than throws. After any operation:

```
property LastError: Integer read FLastError;      // 0 = success
property LastErrorDesc: string read FLastErrorDesc;
```

`LastError` is a standard socket error code (e.g. `WSAETIMEDOUT` on a timeout). `LastErrorDesc` is a human string for it, and the class function `GetErrorDesc(ErrorCode)` / method `GetErrorDescEx` turn any code into text. `ResetLastError` clears both back to the non-error state.

If you *prefer* exceptions, set `RaiseExcept := True` and Synapse raises `ESynapseError` whenever a call fails. The default is `False` — the linear, check-after-each-step style that keeps protocol code readable.

Sending — low level and high level

Low level, raw bytes in your buffer:

```
function SendBuffer(const Buffer: TMemory; Length: Integer): Integer; virtual;
procedure SendByte(Data: Byte); virtual;
```

`TMemory` is Synapse's pointer-to-bytes type; you pass the address of your data and a length. `SendBuffer` returns the number of bytes actually sent.

High level, framing done for you:

```
procedure SendString(Data: AnsiString); virtual;   // sends bytes as-is — NO terminator added
procedure SendInteger(Data: Integer); virtual;      // four bytes
procedure SendBlock(const Data: AnsiString); virtual;   // 4-byte length prefix, then data
procedure SendStreamRaw(const Stream: TStream); virtual;
procedure SendStream(const Stream: TStream); virtual;   // length-framed, pairs with RecvStream
```

Note the honest caveat baked into `SendString`: **it adds no line terminator.** If your protocol is line-based, you append `CRLF` yourself:

```
sock.SendString('USER alice' + CRLF);
```

Receiving — low level and high level

Low level (you must know how much is coming, or check `WaitingData` first, or you risk a deadlock):

```
function RecvBuffer(Buffer: TMemory; Length: Integer): Integer; virtual;
function RecvByte(Timeout: Integer): Byte; virtual;
```

High level, all take a millisecond `Timeout` and set `LastError` to `WSAETIMEDOUT` if it lapses:

```
function RecvBufferEx(Buffer: TMemory; Len, Timeout: Integer): Integer; virtual;
function RecvBufferStr(Len, Timeout: Integer): AnsiString; virtual;
function RecvString(Timeout: Integer): AnsiString; virtual;   // reads up to a CRLF, strips it
function RecvTerminated(Timeout: Integer; const Terminator: AnsiString): AnsiString; virtual;
function RecvPacket(Timeout: Integer): AnsiString; virtual;   // whatever is waiting, dynamic s
function RecvInteger(Timeout: Integer): Integer; virtual;
function RecvBlock(Timeout: Integer): AnsiString; virtual;    // reads a SendBlock frame
procedure RecvStream(const Stream: TStream; Timeout: Integer); virtual;
```

The three you will reach for most:

- **`RecvString`** — a CRLF-terminated line, returned *without* the CRLF. The bread and butter of text protocols (SMTP, POP3, HTTP status lines). Under the hood it calls `RecvTerminated(Timeout, CRLF)`.
- **`RecvPacket`** — "give me whatever is there right now." Ideal when you do not know the length ahead of time — reading a chunk of a TCP stream, or a whole UDP datagram.

- **`RecvBufferEx`** — "give me exactly `Len` bytes, waiting up to `Timeout`." For binary framing where the length is known.

The high-level functions share an internal `LineBuffer` (exposed as a property), which is why you can mix them: a `RecvString` that over-reads leaves the surplus in `LineBuffer` for the next call.

`ConvertLineEnd := True` relaxes `RecvString` to accept lone CR or lone LF as a line end too, not only CRLF — handy against sloppy peers.

Readiness and waiting

You do not usually need these — the `Recv*` timeouts already call them internally — but they are the ingredients of a hand-rolled select loop:

```
function CanRead(Timeout: Integer): Boolean; virtual;   // data waiting, or (TCP) an incoming c
function CanReadEx(Timeout: Integer): Boolean; virtual; // also true if LineBuffer has data
function CanWrite(Timeout: Integer): Boolean; virtual;  // send buffer has room
function WaitingData: Integer; virtual;                 // bytes ready to read (or datagram len
```

`Timeout = 0` means "test and return immediately"; `-1` means "wait, possibly forever." `GroupCanRead` polls a whole `TSocketList` at once — a real `select()` over many sockets, if you want one without leaving the blocking API.

Timeouts and other knobs

```
property ConnectionTimeout: Integer;      // connect timeout in ms (0 = system default)
procedure SetTimeout(Timeout: Integer);    // send + recv timeout (if the provider supports it)
procedure SetSendTimeout(Timeout: Integer);
procedure SetRecvTimeout(Timeout: Integer);
property MaxLineLength: Integer;           // cap on RecvString/RecvTerminated — anti-DoS
property StopFlag: Boolean;                // set True to soft-abort a long operation
property RecvCounter: int64;               // bytes received this connection
property SendCounter: int64;               // bytes sent this connection
property Family: TSocketFamily;            // SF_Any (default), SF_IP4, SF_IP6
```

`MaxLineLength` deserves a mention: it bounds how much `RecvString` / `RecvTerminated` will buffer before erroring, so a hostile peer that never sends a terminator cannot make you allocate memory until you fall over. Default `0` means unlimited — set it on any server that reads lines from untrusted input.

A worked example — a tiny POP3-style dialogue

Line protocols map almost one-to-one onto `RecvString`/`SendString`:

```
uses blcksock, synautil;

procedure FetchGreeting;
var
  sock: TTCPBlockSocket;
  line: AnsiString;
begin
  sock := TTCPBlockSocket.Create;
  try
```

```
    sock.ConnectionTimeout := 5000;
    sock.Connect('mail.example.org', '110');
    if sock.LastError <> 0 then
    begin
      Writeln('connect: ', sock.LastErrorDesc);
      Exit;
    end;

    line := sock.RecvString(15000);        // server greeting, e.g. '+OK POP3 ready'
    Writeln('S: ', line);

    sock.SendString('USER alice' + CRLF);  // remember: we add CRLF ourselves
    Writeln('S: ', sock.RecvString(15000));

    sock.SendString('QUIT' + CRLF);
    Writeln('S: ', sock.RecvString(15000));

    Writeln(sock.RecvCounter, ' bytes received');
  finally
    sock.CloseSocket;
    sock.Free;
  end;
end;
```

Every line of that is base-class API — `TTCPBlockSocket` only adds the transport underneath. That is the point of `TBlockSocket`: the vocabulary is fixed once, and the protocol descendants and your own code both speak it unchanged.

TTCPBlockSocket — TCP

`TTCPBlockSocket` is the workhorse of the family: a blocking TCP socket that speaks stream protocols top-to-bottom, upgrades to TLS through the SSL plugin seam, and can route itself through a SOCKS or HTTP proxy without your protocol code noticing. It descends `TSocksBlockSocket`, so it inherits everything in the base class plus the proxy plumbing.

Declared features, from the source:

> IPv4, IPv6, SSL/TLS or SSH (depending on used plugin), SOCKS5 proxy (outgoing and limited incoming), SOCKS4/4a proxy (outgoing and limited incoming), TCP through HTTP proxy tunnel.

The client pattern

Connect, check `LastError`, talk, close. That is the whole shape:

```
uses blcksock, synautil;

procedure HttpHead(const Host: string);
var
  sock: TTCPBlockSocket;
  line: AnsiString;
begin
  sock := TTCPBlockSocket.Create;
  try
    sock.ConnectionTimeout := 5000;
    sock.Connect(Host, '80');
    if sock.LastError <> 0 then
    begin
      Writeln('connect: ', sock.LastErrorDesc);
      Exit;
    end;

    sock.SendString('HEAD / HTTP/1.0' + CRLF +
                    'Host: ' + Host + CRLF + CRLF);

    repeat
      line := sock.RecvString(10000);
      if sock.LastError = 0 then
        Writeln('S: ', line);
    until (sock.LastError <> 0) or (line = '');
  finally
    sock.CloseSocket;
```

```
    sock.Free;
  end;
end;
```

`Connect(IP, Port)` creates the OS socket if needed and blocks until the connection is up or `ConnectionTimeout` expires. Ports may be numeric (`'80'`) or symbolic (`'http'`). Set `Family` to `SF_IP4` or `SF_IP6` if you want to pin the address family; the default `SF_Any` picks based on how the host resolves.

The listen / accept pattern

A TCP server binds, listens, then accepts. `Accept` returns a raw OS socket handle (`TSocket`) for the new connection — you wrap it in a fresh `TTCPBlockSocket` by assigning its `Socket` property:

```
procedure RunServer;
var
  listener, client: TTCPBlockSocket;
begin
  listener := TTCPBlockSocket.Create;
  try
    listener.CreateSocket;
    listener.SetLinger(True, 10000);
    listener.Bind('0.0.0.0', '8080');     // cAnyHost, any interface
    listener.Listen;
    if listener.LastError <> 0 then
    begin
      Writeln('bind/listen: ', listener.LastErrorDesc);
      Exit;
    end;

    while True do
    begin
      if listener.CanRead(1000) then       // 1 s poll so we can check for shutdown
      begin
        client := TTCPBlockSocket.Create;
        client.Socket := listener.Accept;  // adopt the accepted handle
        HandleClient(client);              // in real code: hand to a thread (below)
      end;
    end;
  finally
    listener.CloseSocket;
    listener.Free;
  end;
end;
```

`CanRead` on the listener is how you tell an incoming connection is waiting without blocking `Accept` forever — the same readiness poll from the base class.

The threaded server

The idiomatic Synapse server gives every connection its own thread of straight-line protocol code. This is exactly the model described in The Blocking Model & Threads: the listener accepts, then hands each connection to a handler thread.

```
uses blcksock, synautil, Classes;

type
  TClientThread = class(TThread)
  private
    FSock: TTCPBlockSocket;
  public
    constructor Create(AHandle: TSocket);
    procedure Execute; override;
  end;

constructor TClientThread.Create(AHandle: TSocket);
begin
  FSock := TTCPBlockSocket.Create;
  FSock.Socket := AHandle;         // adopt the handle Accept returned
  FreeOnTerminate := True;
  inherited Create(False);
end;

procedure TClientThread.Execute;
var
  line: AnsiString;
begin
  try
    line := FSock.RecvString(30000);
    while (FSock.LastError = 0) and (line <> '') do
    begin
      FSock.SendString('echo: ' + line + CRLF);
      line := FSock.RecvString(30000);
    end;
  finally
    FSock.CloseSocket;
    FSock.Free;
  end;
end;
```

The listener loop becomes: `TClientThread.Create(listener.Accept)` for each connection. One socket, one thread, one readable `Execute` — no shared state machine. **Never share a single `TTCPBlockSocket` across threads**; give each connection its own.

SSL/TLS upgrade

`TTCPBlockSocket` never encrypts by itself. TLS is a plugin attached at link time — include an `ssl_*` unit and the socket gains it transparently. The full story is in The SSL Plugin Seam; the socket-side methods are:

```
constructor CreateWithSSL(SSLPlugin: TSSLClass);  // pick a plugin per-socket
procedure SSLDoConnect;                            // upgrade a live client socket to TLS
function  SSLAcceptConnection: Boolean;            // server side: start TLS on an accepted soc
procedure SSLDoShutdown;                           // downgrade back to plain TCP
property  SSL: TCustomSSL read FSSL;               // the plugin instance (config: certs, keys,
```

Implicit TLS (HTTPS-style — encrypted from the first byte): include `ssl_openssl`, then call `SSLDoConnect` right after `Connect`:

```
uses blcksock, ssl_openssl;   // <-- this line chooses OpenSSL

sock.Connect('example.org', '443');
sock.SSLDoConnect;
if sock.LastError <> 0 then
  Writeln('TLS handshake failed: ', sock.LastErrorDesc);
```

Opportunistic TLS (STARTTLS-style — start plaintext, negotiate, then upgrade the *same* live connection):

```
sock.Connect('mail.example.org', '25');   // plaintext SMTP
sock.SendString('STARTTLS' + CRLF);
sock.RecvString(10000);                    // '220 ready to start TLS'
sock.SSLDoConnect;                         // upgrade in place
```

Server side, configure certificates on the `SSL` object before you accept, then turn the accepted socket into a TLS session:

```
client.Socket := listener.Accept;
client.SSL.CertificateFile := 'server.crt';
client.SSL.PrivateKeyFile  := 'server.key';
if not client.SSLAcceptConnection then
  Writeln('TLS accept failed: ', client.LastErrorDesc);
```

Other TCP-specific methods

- **`GetRemoteSinIP` / `GetRemoteSinPort` / `GetLocalSinIP` / `GetLocalSinPort`** — who you are talking to and from, once connected.
- **`SetLinger(Enable, Linger)`** (from the base) — control how long `close` waits to flush unsent data; stream-socket only.
- **`OnAfterConnect`** — an event fired right after a successful TCP connect, useful for logging or per-connection setup.
- **`WaitingData` / `RecvPacket`** are overridden to account for buffered TLS data, so they stay correct once you are inside a TLS session.

Honest caveats

- `SendString` adds **no** terminator — you append `CRLF` for line protocols (see the base class chapter). This is not TCP-specific but it bites here most.
- Synapse is optimised for **synchronous** use. `NonBlockMode` exists but "not all functions work properly in it" — the source's own warning. Prefer blocking + threads.
- In SOCKS mode a server can accept only **one** connection, and `Accept` reuses the listening socket rather than making a new one — a limitation of SOCKS BIND, not of Synapse. For real servers, proxy the clients, not the listener.

TUDPBlockSocket — UDP

`TUDPBlockSocket` is the connectionless member of the family. There is no connect handshake, no stream, no "connection closed" — just datagrams in and datagrams out. It descends `TDgramBlockSocket` (which descends `TSocksBlockSocket`), and that middle class does one clever thing for you: it **redirects the ordinary `SendBuffer/RecvBuffer` onto the connectionless `SendBufferTo/RecvBufferFrom`**, so all the high-level helpers from the base class — `SendString`, `RecvPacket`, and friends — just work on datagrams.

Declared features, from the source:

> IPv4, IPv6, unicasts, broadcasts, multicasts, SOCKS5 proxy (only unicasts, outgoing and incoming).

Do I need a thread? No.

Unlike TCP, UDP is fine on your main thread. Sending a datagram costs about as much as handing the packet to the network adapter, and for receiving you can poll with a `0` ms timeout and simply check whether anything arrived. Blocking UDP in the main thread is perfectly workable.

The two addressing styles

Synapse gives you two ways to name the far end:

1. **`Connect(IP, Port)`** — for UDP this does *not* handshake. It just fills in the remote address (`RemoteSin`), so subsequent `SendString` / `SendBuffer` go to that peer. Convenient when you talk to one fixed target.
2. **`SendBufferTo` + `RecvBufferFrom`** — explicit per-datagram addressing. `RecvBufferFrom` fills `RemoteSin` with the *sender's* address, so you can reply to whoever just wrote to you. This is the natural server style.

```
function SendBufferTo(const Buffer: TMemory; Length: Integer): Integer; override;
function RecvBufferFrom(Buffer: TMemory; Length: Integer): Integer; override;
```

Worked example — a client that sends and waits for a reply

The UDP echo service (port 7) is the classic demo: send a datagram, read the one that comes back.

```
uses blcksock;

procedure UdpEcho(const Host, Msg: string);
```

```
var
  sock: TUDPBlockSocket;
  reply: AnsiString;
begin
  sock := TUDPBlockSocket.Create;
  try
    sock.Connect(Host, '7');        // fixes RemoteSin; no handshake happens
    sock.SendString(Msg);           // redirected to SendBufferTo(RemoteSin)
    if sock.LastError <> 0 then
    begin
      Writeln('send: ', sock.LastErrorDesc);
      Exit;
    end;

    reply := sock.RecvPacket(2000); // whole datagram, up to 2 s
    if sock.LastError = 0 then
      Writeln('got back: ', reply)
    else
      Writeln('no reply: ', sock.LastErrorDesc);
  finally
    sock.CloseSocket;
    sock.Free;
  end;
end;
```

`RecvPacket` is the right receive call for UDP: one call returns exactly one whole datagram (its size is dynamic, so you never guess a buffer length).

Worked example — a UDP server

A server binds a port and loops, replying to each sender. Because `RecvBufferFrom` (and therefore `RecvPacket`) records the sender in `RemoteSin`, `SendString` back out goes straight to them.

```
uses blcksock;

procedure RunUdpServer(const Port: string);
var
  sock: TUDPBlockSocket;
  data: AnsiString;
begin
  sock := TUDPBlockSocket.Create;
  try
    sock.Bind('0.0.0.0', Port);     // cAnyHost = all interfaces
    if sock.LastError <> 0 then
    begin
      Writeln('bind: ', sock.LastErrorDesc);
      Exit;
    end;

    while True do
    begin
      data := sock.RecvPacket(1000);         // 1 s poll
      if sock.LastError = 0 then
```

```
      begin
        Writeln('from ', sock.GetRemoteSinIP, ':', sock.GetRemoteSinPort,
                ' -> ', data);
        sock.SendString('ack: ' + data);     // replies to RemoteSin (the sender)
      end;
      // on WSAETIMEDOUT, loop again — lets you check a shutdown flag
    end;
  finally
    sock.CloseSocket;
    sock.Free;
  end;
end;
```

If you ever need to send to a peer you have not received from, set the target explicitly with `SetRemoteSin(IP, Port)` (inherited) before `SendString`.

Broadcast

UDP can hit every host on the local network. Enable it, then send to the broadcast address (`cBroadcast = '255.255.255.255'`):

```
sock := TUDPBlockSocket.Create;
try
  sock.EnableBroadcast(True);          // must be on before broadcasting
  sock.Connect('255.255.255.255', '9999');
  sock.SendString('DISCOVER');
  // ...then RecvPacket in a loop to collect replies from responders...
finally
  sock.CloseSocket;
  sock.Free;
end;
```

`EnableBroadcast` binds the socket if it is not already bound. Note from the source: broadcast is **not** available in SOCKS5 mode, and **IPv6 has no broadcast at all** — use multicast there instead.

Multicast

For group communication (and the IPv6 replacement for broadcast):

```
procedure AddMulticast(MCastIP: string);        // join a group
procedure DropMulticast(MCastIP: string);       // leave it
procedure EnableMulticastLoop(Value: Boolean);  // hear your own sends, or not
property  MulticastTTL: Integer;                 // router hops (1 = local network only)
```

```
sock := TUDPBlockSocket.Create;
try
  sock.Bind('0.0.0.0', '5000');
  sock.AddMulticast('239.1.2.3');      // join the group
  sock.MulticastTTL := 1;              // stay on the local segment
  sock.SendString('hello group');      // reaches all joined members
finally
  sock.DropMulticast('239.1.2.3');
  sock.CloseSocket;
```

```
  sock.Free;
end;
```

`MulticastTTL` is a genuine footgun in disguise: leave it at `1` unless you mean to cross routers, because raising it sprays your traffic outward — and many routers drop multicast anyway, so a high TTL is often disappointment plus risk.

Honest caveats

- **UDP is lossy and unordered by design.** Synapse does not add reliability. A `RecvPacket` timeout on a UDP socket usually means the datagram (or its reply) was simply dropped — retry logic is your job.
- A datagram larger than your buffer in the low-level `RecvBufferFrom` is **truncated**; `RecvPacket` avoids this by sizing dynamically.
- SOCKS support here is **unicast only** — no broadcast or multicast through a proxy.

TSocksBlockSocket — the proxy layer

`TSocksBlockSocket` is the quiet middle of the hierarchy. You never instantiate it — its own source comment says "Do not use this class directly" — but it sits between the base class and both `TTCPBlockSocket` and the datagram sockets, and it carries the machinery that lets any of them reach the network **through a SOCKS proxy**. The elegant part: it does this without adding a single new call to your protocol code.

The idea: proxying is configuration, not a code path

A naive design would give you a "connect through proxy" method distinct from a "connect direct" one. Synapse refuses that. Instead, the proxy is a set of **properties** on the socket. Set them and the ordinary `Connect` / `Bind` / `Listen` you already call quietly route through the proxy; leave them empty and the same code connects directly.

```
property SocksIP: string;        // proxy address — EMPTY = SOCKS disabled; setting it ENABLES
property SocksPort: string;      // proxy port, default '1080'
property SocksUsername: string;  // set if the proxy needs auth
property SocksPassword: string;
property SocksTimeout: integer;  // proxy-conversation timeout, default one minute
property SocksResolver: Boolean; // let the PROXY resolve hostnames (default True)
property SocksType: TSocksType;  // ST_Socks5 (default) or ST_Socks4
```

The trigger is `SocksIP`: assigning any non-empty value turns SOCKS mode on; clearing it turns it off. There is no separate "enable" switch — the source is explicit that "assigning any value to this property enables SOCKS mode."

Using it — the whole change is three lines

Take the TCP client from the TCP chapter and route it through a SOCKS5 proxy. The protocol code is untouched; only setup changes:

```
uses blcksock;

var
  sock: TTCPBlockSocket;
begin
  sock := TTCPBlockSocket.Create;
  try
    sock.SocksIP   := '10.0.0.1';    // <-- these three lines are the entire
    sock.SocksPort := '1080';        //     difference from a direct connection
```

```
    sock.SocksType := ST_Socks5;

    sock.Connect('example.org', '80');   // now tunnelled through the proxy
    if sock.LastError <> 0 then
    begin
      Writeln('connect via proxy: ', sock.LastErrorDesc);
      Exit;
    end;
    sock.SendString('HEAD / HTTP/1.0' + CRLF + CRLF);
    Writeln(sock.RecvString(10000));
  finally
    sock.CloseSocket;
    sock.Free;
  end;
end;
```

`Connect` internally opens the connection to `SocksIP:SocksPort`, performs the SOCKS handshake (authenticating with `SocksUsername`/`SocksPassword` if set), asks the proxy to reach `example.org:80`, and only then returns. Your `SendString`/`RecvString` see a normal live socket.

With auth:

```
sock.SocksIP       := '10.0.0.1';
sock.SocksUsername := 'alice';
sock.SocksPassword := 's3cret';
```

SOCKS4, SOCKS4a, SOCKS5 — and who resolves the name

`SocksType` picks the protocol, and it interacts with `SocksResolver`:

- **`ST_Socks5`** (default) — full SOCKS5, with username/password auth and remote name resolution.
- **`ST_Socks4`** with `SocksResolver = True` — uses **SOCKS4a**, the extension that lets the proxy resolve the hostname.
- **`ST_Socks4`** with `SocksResolver = False` — pure SOCKS4, which cannot carry a hostname, so the name is resolved **locally** first.

`SocksResolver` (default `True`) is the privacy-relevant knob: leave it on and DNS lookups happen at the proxy, so the target hostname never leaks from your machine. Turn it off and your host resolves the name before connecting.

It works for UDP too

Because `TDgramBlockSocket` also descends `TSocksBlockSocket`, `TUDPBlockSocket` inherits the same properties. Set `SocksIP` on a UDP socket and unicast datagrams route through a SOCKS5 UDP association — with the honest limitation, noted in the UDP chapter, that **only unicast** works through a proxy: no broadcast, no multicast.

Status and internals

A few members let you inspect what happened:

```
property UsingSocks: Boolean read FUsingSocks;      // True once a proxy is actually in use
property SocksLastError: integer read FSocksLastError; // error code returned BY the proxy
```

The class also exposes `SocksOpen`, `SocksRequest`, and `SocksResponse`, but its comments flag all three as "needed only in special cases (it is called internally)" — you drive them by setting properties and calling `Connect`, not by calling these directly.

Not to be confused with the HTTP tunnel

`TTCPBlockSocket` has a *second*, independent way to traverse a proxy — an HTTP `CONNECT` tunnel, configured with `HTTPTunnelIP` / `HTTPTunnelPort` / `HTTPTunnelUser` / `HTTPTunnelPass`. That lives on `TTCPBlockSocket`, not here, and the source is emphatic: **you cannot combine the two.** SOCKS mode (`SocksIP` set) and HTTP-tunnel mode (`HTTPTunnelIP` set) are mutually exclusive — pick one proxy mechanism per socket.

Why it is nicely designed

- **Proxying is orthogonal.** It rides on the same `Connect`/`Bind`/`Listen` every socket already has, so *any* code built on `TTCPBlockSocket` or `TUDPBlockSocket` — including Synapse's own HTTP, SMTP, and FTP clients — gains proxy support for free, just by having the properties set.
- **Placed exactly once, inherited exactly where it belongs.** Putting SOCKS on a mid-tree class means TCP and UDP get it while `TRAWBlockSocket` (which cannot proxy) simply descends past it. The hierarchy encodes the capability.
- **Configuration over API surface.** No new verbs to learn; the proxy is data you set, not a control path you branch on. The same lesson the SSL plugin seam teaches, applied to proxies instead of TLS.

Synapse Protocol Classes

Above the socket layer sits the part of Synapse most people actually reach for: the protocol clients. SMTP, POP3, IMAP, HTTP, FTP, NNTP, LDAP, SNMP, SNTP, DNS, ping, telnet — each is a small, readable class that speaks one protocol over a `TTCPBlockSocket`. There is no framework here, no plugin registry, no visual component to drop on a form. There is a class, you set a few properties, you call a few methods in order, and you read the result. That is the whole story, and it is the same story for every protocol in the list.

One shape, repeated

Almost every client descends from a single tiny base class, `TSynaClient` (defined in `blcksock`). It contributes nothing protocol-specific — just the connection knobs every client needs:

```
TSynaClient = class(TObject)
published
  property TargetHost: string;   // server IP or name  (default 'localhost')
  property TargetPort: string;   // server port        (client sets its default)
  property IPInterface: string;  // outgoing local address
  property Timeout: integer;     // socket timeout, ms
  property UserName: string;      // auth, if the protocol needs it
  property Password: string;
  property OAuth2Token: string;
end;
```

A concrete client — `TPOP3Send`, `TSMTPSend`, `TLDAPSend`, and the rest — then owns a `TTCPBlockSocket` (exposed as the `Sock` property) and adds the verbs of its protocol. The constructor picks the standard port for you, so you usually only need to set `TargetHost`, maybe `UserName`/`Password`, and go. The socket is created and freed with the client; you never manage it directly, though `Sock` is right there when you want to attach an `OnStatus` hook or tune a timeout.

The request/response rhythm

The clients are *blocking* by design (see The blocking model). That makes their internals a linear script rather than a state machine, and it makes the public API read the same way. The pattern, across protocols, is:

1. **`Login`** — connect the socket, do the protocol greeting/handshake, and (if you supplied credentials) authenticate. Returns `Boolean`.
2. **Issue verbs** — `Stat`, `Retr`, `MailFrom`, `Search`, `HTTPMethod`… each sends a command, blocks for the reply, and returns `True`/`False`.

3. **Read the result** — most clients expose `ResultCode` / `ResultString` for the last reply, plus a protocol-appropriate payload holder (`FullResult`, `Document`, `SearchResult`, …).
4. **`Logout`** — send the protocol's goodbye and close the socket.

Because each verb blocks until its reply lands, you never poll and you never register a callback. Concurrency, when you want it, is "run the client on another thread" — nothing in the class needs to change.

TLS drops in from the side

None of these clients contain any TLS code. Encryption is supplied by the SSL plugin seam: you add one `ssl_*` unit to your `uses` clause and the global `SSLImplementation` metaclass is swapped for a real backend.

```
uses
  smtpsend, ssl_openssl;   // that one extra unit is the entire TLS wiring
```

With a plugin present, two idioms cover almost everything:

- **Implicit TLS (tunnel from the first byte).** Set `FullSSL := True` before `Login`. The socket does its TLS handshake immediately on connect — this is the "SSL on a dedicated port" style (POP3S :995, SMTPS :465, IMAPS :993, HTTPS :443, LDAPS :636).
- **Explicit TLS (STARTTLS-style upgrade).** Set `AutoTLS := True` and `Login` will, if the server advertises the capability, upgrade the *live* plaintext connection to TLS mid-session (POP3 `STLS`, SMTP/LDAP `STARTTLS`). Each client that supports it also exposes a public `StartTLS` method if you want to drive the upgrade yourself.

`HTTPS` is even quieter: `THTTPSend` upgrades automatically when the URL scheme is `https://`, so a plugin in `uses` is all it takes.

The clients

Each lives in its own unit in the Synapse source (one unit per protocol). Ports are the defaults each client's constructor sets.

Protocol	Unit	Class	Default port	Notes
SMTP / ESMTP	`smtpsend`	`TSMTPSend`	25	STARTTLS, CRAM-MD5/PLAIN/LOGIN auth · chapter
POP3	`pop3send`	`TPOP3Send`	110	APOP, STLS, XOAUTH2 · chapter
IMAP4	`imapsend`	`TIMAPSend`	143	full mailbox client · chapter
HTTP / HTTPS	`httpsend`	`THTTPSend`	80	convenience `HttpGetText` etc. · chapter
FTP	`ftpsend`	`TFTPSend`	21	active/passive data connections · chapter

Protocol	Unit	Class	Default port	Notes
NNTP	`nntpsend`	`TNNTPSend`	119	Usenet news · chapter
LDAP v2/v3	`ldapsend`	`TLDAPSend`	389	bind/search, SASL DIGEST-MD5 · chapter
SNMP	`snmpsend`	`TSNMPSend`	161	plus traps on 162 · chapter
SNTP	`sntpsend`	`TSNTPSend`	123	network time · chapter
DNS	`dnssend`	`TDNSSend`	53	resolver client · chapter
Ping	`pingsend`	`TPINGSend`	—	ICMP echo (raw socket) · chapter
Telnet	`tlntsend`	`TTelnetSend`	23	terminal session · chapter
Syslog	`slogsend`	`TSyslogSend`	514	RFC-3164 logging (UDP) · chapter
ClamAV	`clamsend`	`TClamSend`	3310	virus-scan daemon client

The chapters that follow work through the most-used clients in detail — POP3, LDAP, SMTP, HTTP, IMAP, FTP, and DNS — with the rest surveyed in other protocols. Once you have read one, the others hold no surprises: learn the rhythm once, and it repeats all the way down the table.

TPOP3Send

POP3 is the simplest of the mail protocols — a numbered mailbox you connect to, count, download, and delete from — and `TPOP3Send` (in `pop3send`) is a faithful, compact mirror of it. If you have read the protocol-class overview, everything here will feel familiar: a `TSynaClient` descendant wrapping a `TTCPBlockSocket`, a `Login`, a handful of verbs, a `Logout`.

The class at a glance

The credentials and connection properties come from the base `TSynaClient` (`TargetHost`, `TargetPort`, `UserName`, `Password`, `Timeout`). `TPOP3Send` adds the POP3 verbs and a few result holders:

```
TPOP3Send = class(TSynaClient)
  function Login: Boolean;
  function Logout: Boolean;
  function Stat: Boolean;              // -> StatCount, StatSize
  function List(Value: Integer): Boolean;   // 0 = all messages
  function Retr(Value: Integer): Boolean;   // download message N
  function RetrStream(Value: Integer; Stream: TStream): Boolean;
  function Dele(Value: Integer): Boolean;   // mark message N deleted
  function Top(Value, Maxlines: Integer): Boolean;
  function Uidl(Value: Integer): Boolean;   // unique-id listing
  function Reset: Boolean;            // RSET
  function NoOp: Boolean;
  function Capability: Boolean;       // CAPA
  function StartTLS: Boolean;         // STLS upgrade
  function CustomCommand(const Command: string; MultiLine: Boolean): Boolean;

  property ResultCode: Integer;       // 1 = OK, 0 = error
  property ResultString: string;      // the server's +OK / -ERR line
  property FullResult: TStringList;   // multiline payload (message, listing…)
  property StatCount: Integer;        // messages in mailbox, after Stat
  property StatSize: Integer;         // total bytes, after Stat
  property ListSize: Integer;         // size of one message, after List(N)
  property TimeStamp: string;         // APOP challenge, if the server sent one
  property AuthType: TPOP3AuthType;   // POP3AuthAll | POP3AuthLogin | POP3AuthAPOP
  property AutoTLS: Boolean;
  property FullSSL: Boolean;
  property Sock: TTCPBlockSocket;
end;
```

A couple of things worth noting from the source. `ResultCode` is *not* an HTTP- style code — POP3 only ever answers `+OK` or `-ERR`, so `ResultCode` is simply `1` for success and `0` for failure,

and the raw reply line is in `ResultString`. Every multiline reply (a retrieved message, a `LIST` of the whole mailbox) lands in `FullResult`, a `TStringList`, with POP3's byte-stuffed leading dots already un-stuffed for you.

Authentication: where the hand-rolled crypto earns its keep

`Login` does the whole handshake for you, and its auth logic is a nice illustration of why Synapse rolls its own crypto. The default `AuthType` is `POP3AuthAll`, which means *autodetect*:

- If the server's greeting carries an **APOP timestamp** (a `<...>` token, stored in the `TimeStamp` property), `Login` tries **APOP**: it hashes `MD5(TimeStamp + Password)` and sends the digest, so the password never crosses the wire. That `MD5` is the one in `synacode` — no external library.
- Otherwise it falls back to plain **USER/PASS**.
- If you set `OAuth2Token` on the client, `Login` uses **XOAUTH2** instead.

You can force one path by setting `AuthType` to `POP3AuthLogin` (USER/PASS only) or `POP3AuthAPOP` (APOP only).

> **Honest caveat.** USER/PASS sends your password in the clear, and APOP — though it hides the password — relies on MD5 and is long deprecated. Neither is safe on an untrusted network *by itself*. For real security, put the session inside TLS (below); then even USER/PASS is protected by the transport.

TLS: implicit tunnel or STLS upgrade

As with every Synapse client, TLS is a `uses` clause away — add an `ssl_*` plugin and pick one of two styles:

```
uses pop3send, ssl_openssl;
```

- **POP3S (implicit).** Set `FullSSL := True` and point `TargetPort` at `995`. The socket does its TLS handshake the moment it connects.
- **STLS (explicit upgrade).** Set `AutoTLS := True`. During `Login`, if the server's `CAPA` advertises `STLS`, the live plaintext connection is upgraded to TLS before authentication — Synapse re-runs `Capability` afterwards so the post- TLS capability set is the one you see.

Worked example: fetch and print the first message

```
program PopFetch;

uses
  SysUtils, Classes,
  pop3send, ssl_openssl;   // ssl_openssl supplies TLS for STLS / POP3S

var
  Pop: TPOP3Send;
  i: Integer;
begin
  Pop := TPOP3Send.Create;
  try
    Pop.TargetHost := 'mail.example.com';
    Pop.UserName   := 'alice';
```

```
    Pop.Password   := 's3cret';
    Pop.AutoTLS    := True;          // upgrade with STLS if offered

    if not Pop.Login then
    begin
      Writeln('Login failed: ', Pop.ResultString);
      Halt(1);
    end;

    // How many messages, and how many bytes total?
    if Pop.Stat then
      Writeln(Format('Mailbox: %d messages, %d bytes',
        [Pop.StatCount, Pop.StatSize]));

    if Pop.StatCount > 0 then
    begin
      // Retrieve message #1 in full; it lands in FullResult, line by line.
      if Pop.Retr(1) then
      begin
        Writeln('--- message 1 ---');
        for i := 0 to Pop.FullResult.Count - 1 do
          Writeln(Pop.FullResult[i]);
      end
      else
        Writeln('RETR failed: ', Pop.ResultString);

      // To delete it instead (marked now, removed at QUIT):
      // Pop.Dele(1);
    end;

    Pop.Logout;                     // sends QUIT and closes the socket
  finally
    Pop.Free;
  end;
end.
```

The retrieved message in `FullResult` is the raw RFC-822 text — headers, a blank line, then the body, exactly as the server stored it. To turn that into decoded subject/from/body fields (and to walk MIME attachments), feed it to a `TMimeMess` and call `DecodeMessage`; the mail-message chapter covers that.

Peeking without downloading

Two verbs let you inspect a mailbox cheaply:

- **`Top(N, Lines)`** downloads message `N`'s headers plus the first `Lines` lines of its body into `FullResult` — ideal for a message-list preview without pulling megabytes.
- **`Uidl(N)`** returns a stable *unique id* for a message (or, with `0`, for the whole mailbox in `FullResult`). Because POP3 message numbers are only stable within a single session, UIDLs are how a "leave mail on server, download only what's new" client remembers what it has already seen.

Deletion is deferred

`Dele(N)` does not delete immediately — it *marks* the message, and the server only actually removes marked messages when the session ends cleanly with `Logout` (QUIT). Call `Reset` to clear all delete marks in the current session, and remember that dropping the connection without `Logout` abandons the deletions.

Streaming large messages

For a big message you do not want the whole thing buffered in a `TStringList`. `RetrStream(N, AStream)` writes message `N` straight into any `TStream` (a file, a memory stream) as it arrives, bypassing `FullResult` entirely — the right tool for large attachments.

TLDAPSend — LDAP directory queries

LDAP is a different animal from the mail protocols. Where POP3 and SMTP trade lines of ASCII text, LDAP speaks **ASN.1 BER**, a binary encoding, over the wire. Synapse hides all of that: `TLDAPSend` (in `ldapsend`) builds and decodes the BER packets internally — leaning on the `asn1util` unit — and hands you plain Pascal objects. You bind, you search, you read result objects. The binary protocol never surfaces.

The class at a glance

Connection and credentials come from the `TSynaClient` base (`TargetHost`, `TargetPort`, `UserName`, `Password`). `TLDAPSend` adds the directory operations and, crucially, a structured result model:

```
TLDAPSend = class(TSynaClient)
  function Login: Boolean;            // connect (+ optional StartTLS) -- does NOT bind
  function Bind: Boolean;             // simple bind (plaintext password!)
  function BindSasl: Boolean;         // SASL DIGEST-MD5 bind (password never sent)
  function Logout: Boolean;
  function Search(obj: AnsiString; TypesOnly: Boolean; Filter: AnsiString;
    const Attributes: TStrings): Boolean;
  function Compare(obj, AttributeValue: AnsiString): Boolean;
  function Add(obj: AnsiString; const Value: TLDAPAttributeList): Boolean;
  function Modify(obj: AnsiString; Op: TLDAPModifyOp;
    const Value: TLDAPAttribute): Boolean;
  function ModifyDN(obj, newRDN, newSuperior: AnsiString;
    DeleteoldRDN: Boolean): Boolean;
  function Delete(obj: AnsiString): Boolean;
  function Extended(const Name, Value: AnsiString): Boolean;
  function StartTLS: Boolean;

  property Version: integer;          // default 3
  property ResultCode: Integer;       // 0 = Success (LDAP result code)
  property ResultString: AnsiString;  // human-readable description
  property SearchScope: TLDAPSearchScope;   // SS_BaseObject|SS_SingleLevel|SS_WholeSubtree
  property SearchSizeLimit: integer;
  property SearchTimeLimit: integer;
  property SearchResult: TLDAPResultList;   // <-- results land here
  property Referals: TStringList;
  property AutoTLS: Boolean;
  property FullSSL: Boolean;
```

```
  property Sock: TTCPBlockSocket;
end;
```

Note the LDAP convention on `ResultCode`: **0 means Success** here (it is the LDAP protocol result code — 0 Success, 49 Invalid credentials, 32 No such object, and so on), the opposite of POP3's `1 = OK`. `ResultString` gives you the human-readable form for logging.

Login, then bind — they are two steps

This is the one place `TLDAPSend` differs from the other clients in a way that trips people up. Its `Login` **only opens the connection** (and runs `StartTLS` if `AutoTLS` is set) — it does *not* authenticate:

```
function TLDAPSend.Login: Boolean;
begin
  Result := False;
  if not Connect then Exit;
  Result := True;
  if FAutoTLS then
    Result := StartTLS;
end;
```

Authentication is a separate `Bind` (or `BindSasl`). So the sequence is always `Login` → `Bind`/`BindSasl` → your operations → `Logout`. An empty `UserName` and `Password` at `Bind` time is a valid **anonymous bind** — common for public, read-only directory lookups.

- **`Bind`** sends a *simple bind*: the password travels in a BER octet-string, in the clear. Fine over TLS, unsafe otherwise.
- **`BindSasl`** does a **SASL DIGEST-MD5** exchange (built with `synacode`'s HMAC/MD5, no external library) so the password is never transmitted. Prefer it when you cannot use TLS — though DIGEST-MD5 is itself dated, so the real answer for a hostile network is TLS underneath.

TLS

The usual seam applies — add an `ssl_*` plugin and choose implicit or explicit:

```
uses ldapsend, ssl_openssl;
```

- **LDAPS (implicit)** — set `FullSSL := True`, `TargetPort := '636'`.
- **STARTTLS (explicit)** — set `AutoTLS := True` so `Login` upgrades the plain :389 connection before you bind, or call `StartTLS` yourself.

The result model

A `Search` fills `SearchResult`, a `TLDAPResultList`. The shape mirrors LDAP itself:

- `SearchResult` — a list of `TLDAPResult` objects, one per directory entry found. Indexed with `[i]`, counted with `.Count`.
- Each `TLDAPResult` has an **`ObjectName`** (the entry's DN) and an **`Attributes`** list (`TLDAPAttributeList`).
- Each `TLDAPAttribute` is a `TStringList` descendant with an **`AttributeName`** and one line per value (attributes can be multi-valued). `Attributes.Get(name)` is a shortcut to the first value of a named attribute.

Worked example: bind and search

Find every person whose surname is `Smith` under a base DN, and print their name and email:

```
program LdapSearch;

uses
  SysUtils, Classes,
  ldapsend, ssl_openssl;   // ssl_openssl enables StartTLS / LDAPS

var
  Ldap: TLDAPSend;
  Attrs: TStringList;
  i, j: Integer;
  Entry: TLDAPResult;
  Attr: TLDAPAttribute;
begin
  Ldap := TLDAPSend.Create;
  Attrs := TStringList.Create;
  try
    Ldap.TargetHost := 'ldap.example.com';
    Ldap.UserName   := 'cn=reader,dc=example,dc=com';
    Ldap.Password   := 'readonly';
    Ldap.AutoTLS    := True;             // upgrade to TLS before binding

    if not Ldap.Login then               // connect (+ StartTLS)
    begin
      Writeln('Connect failed');
      Halt(1);
    end;

    if not Ldap.Bind then                // authenticate; 0 = Success
    begin
      Writeln('Bind failed: ', Ldap.ResultString);
      Halt(1);
    end;

    // Only fetch the attributes we care about (empty list = all attributes).
    Attrs.Add('cn');
    Attrs.Add('mail');

    Ldap.SearchScope := SS_WholeSubtree;
    if Ldap.Search('dc=example,dc=com',  // base DN
                   False,                // TypesOnly = False -> return values
                   '(sn=Smith)',         // filter
                   Attrs) then
    begin
      Writeln(Ldap.SearchResult.Count, ' entries:');
      for i := 0 to Ldap.SearchResult.Count - 1 do
      begin
        Entry := Ldap.SearchResult[i];
        Writeln('DN: ', Entry.ObjectName);
        for j := 0 to Entry.Attributes.Count - 1 do
        begin
```

```
          Attr := Entry.Attributes[j];
          // Attr is a TStringList of values under Attr.AttributeName
          Writeln('  ', Attr.AttributeName, ' = ', Attr.Text);
        end;
      end;
    end
    else
      Writeln('Search failed: ', Ldap.ResultString);

    Ldap.Logout;
  finally
    Attrs.Free;
    Ldap.Free;
  end;
end.
```

The `Search` arguments map straight onto the LDAP operation: the **base DN** to search under, `TypesOnly` (pass `False` to get attribute values, `True` for just the attribute names), an **RFC-4515 filter string** (`(sn=Smith)`, `(&(objectClass=person)(mail=*))`, …) which `TLDAPSend` translates to BER for you, and the list of attributes to return (leave it empty to ask for all of them). Scope is set separately via `SearchScope`, and you can cap a runaway query with `SearchSizeLimit` / `SearchTimeLimit`.

Writing to the directory

The same object drives the write side, each method a one-to-one LDAP operation: `Add` (create an entry from a `TLDAPAttributeList`), `Modify` (add/delete/replace an attribute's values via `TLDAPModifyOp = MO_Add/MO_Delete/MO_Replace`), `ModifyDN` (rename or move an entry), and `Delete`. All obey the same `ResultCode = 0` success convention, and all require a bind with sufficient privilege first.

> **Honest caveat.** A simple `Bind` sends the password in cleartext BER; even `BindSasl`'s DIGEST-MD5 is a legacy mechanism. Treat TLS (LDAPS or STARTTLS) as mandatory for anything but an anonymous read against a public directory.

TSMTPSend — sending mail

`TSMTPSend` (in `smtpsend`) is the send half of the mail story. It speaks SMTP and ESMTP, handles the `EHLO`/`HELO` negotiation, authenticates with the best mechanism the server offers, and — with an SSL plugin in `uses` — upgrades the session to TLS. As with every Synapse client it is a thin `TSynaClient` descendant over a `TTCPBlockSocket`; you set a few properties and call the verbs in the order the protocol expects.

The class at a glance

Credentials and connection come from `TSynaClient` (`TargetHost`, `TargetPort`, `UserName`, `Password`). `TSMTPSend` adds the SMTP verbs and ESMTP state:

```
TSMTPSend = class(TSynaClient)
  function Login: Boolean;            // connect, EHLO/HELO, STARTTLS, AUTH
  function Logout: Boolean;           // QUIT + close
  function MailFrom(const Value: string; Size: Integer): Boolean;   // MAIL FROM
  function MailTo(const Value: string): Boolean;                    // RCPT TO
  function MailData(const Value: TStrings): Boolean;                // DATA + body
  function Reset: Boolean;            // RSET
  function NoOp: Boolean;
  function Verify(const Value: string): Boolean;   // VRFY
  function Etrn(const Value: string): Boolean;
  function StartTLS: Boolean;         // STARTTLS upgrade
  function FindCap(const Value: string): string;   // query an ESMTP capability

  property ResultCode: Integer;       // the numeric SMTP reply code (250, 550…)
  property ResultString: string;      // the last reply line
  property FullResult: TStringList;   // the full (maybe multiline) reply
  property ESMTPcap: TStringList;     // capabilities from EHLO
  property ESMTP: Boolean;            // True if the server spoke ESMTP
  property AuthDone: Boolean;         // True if AUTH succeeded
  property ESMTPSize: Boolean;
  property MaxSize: Integer;          // server's max message size, if advertised
  property SystemName: string;        // name we send in HELO/EHLO
  property AutoTLS: Boolean;
  property FullSSL: Boolean;
  property Sock: TTCPBlockSocket;
end;
```

Unlike POP3's `1/0`, SMTP's `ResultCode` is the real three-digit reply code, so you can reason about it directly (`250` OK, `354` "send your data", `550` rejected, …). Synapse also parses RFC-3463 enhanced status codes into `EnhCode1/2/3`, with `EnhCodeString` giving a human sentence for the last one.

What `Login` does for you

`Login` is a lot of protocol folded into one call. Reading the source, it:

1. Connects (doing an immediate TLS handshake first if `FullSSL` is set).
2. Sends **`EHLO`**; if the server rejects ESMTP it falls back to plain `HELO`.
3. Parses the advertised capabilities into `ESMTPcap`.
4. If `AutoTLS` is set and the server advertises `STARTTLS`, upgrades the live connection to TLS and re-runs `EHLO`.
5. If you supplied a `UserName`/`Password`, authenticates — **preferring CRAM-MD5**, then `PLAIN`, then `LOGIN`, based on what the server's `AUTH` capability lists. All three digests/encodings come from `synacode` (HMAC-MD5, Base64) — no external crypto library. `AuthDone` tells you whether auth actually happened.

The name announced in `HELO`/`EHLO` defaults to the socket's local name; override it via `SystemName` if you need a specific identity.

STARTTLS via the plugin

TLS is the usual SSL plugin seam — one unit in `uses`, then choose implicit or explicit:

```
uses smtpsend, ssl_openssl;
```

- **Submission with STARTTLS (explicit).** Set `AutoTLS := True`, use the submission port `TargetPort := '587'` (or plain 25). `Login` upgrades the connection before it authenticates, so your credentials go out encrypted.
- **SMTPS (implicit).** Set `FullSSL := True`, `TargetPort := '465'`. The TLS handshake happens the instant the socket connects.

  Honest caveat. `AUTH LOGIN` and `AUTH PLAIN` hand the server your password (Base64 is encoding, not encryption). Only send credentials over a TLS-upgraded session — which is exactly why `Login` performs STARTTLS *before* AUTH when `AutoTLS` is on.

The send rhythm

An SMTP transaction is a fixed sequence of verbs, and `TSMTPSend` maps one method to each:

```
Login  ->  MailFrom(sender)  ->  MailTo(rcpt)  [->  MailTo(rcpt2) …]  ->  MailData(lines)  ->
```

`MailFrom` takes an optional size (pass the byte length of your message, or `0`); if the server advertised the `SIZE` extension it will be sent along so an over-limit message is rejected up front. `MailTo` is called once per recipient. `MailData` takes a `TStrings` holding the **complete RFC-822 message** — headers, a blank line, then the body — and handles SMTP's dot-stuffing for you.

Worked example: send a plain message

```
program SmtpSend;

uses
  SysUtils, Classes,
  smtpsend, ssl_openssl;   // ssl_openssl enables STARTTLS / SMTPS

var
```

```
  Smtp: TSMTPSend;
  Msg: TStringList;
begin
  Smtp := TSMTPSend.Create;
  Msg  := TStringList.Create;
  try
    // Build the full RFC-822 message: headers, blank line, body.
    Msg.Add('From: Alice <alice@example.com>');
    Msg.Add('To: Bob <bob@example.org>');
    Msg.Add('Subject: Hello from Synapse');
    Msg.Add('Date: ' + Rfc822DateTime(Now));   // Rfc822DateTime is in synautil
    Msg.Add('');                                // <-- header/body separator
    Msg.Add('This message was sent with TSMTPSend.');

    Smtp.TargetHost := 'smtp.example.com';
    Smtp.TargetPort := '587';        // submission port
    Smtp.UserName   := 'alice@example.com';
    Smtp.Password   := 's3cret';
    Smtp.AutoTLS    := True;         // STARTTLS before AUTH

    if not Smtp.Login then
    begin
      Writeln('Login failed: ', Smtp.ResultString);
      Halt(1);
    end;

    if Smtp.MailFrom('alice@example.com', Length(Msg.Text)) and
       Smtp.MailTo('bob@example.org') and
       Smtp.MailData(Msg) then
      Writeln('Sent. Server said: ', Smtp.ResultString)
    else
      Writeln('Send failed: ', Smtp.ResultString);

    Smtp.Logout;
  finally
    Msg.Free;
    Smtp.Free;
  end;
end.
```

The envelope addresses in `MailFrom`/`MailTo` are the SMTP-level sender and recipients; the `From:`/`To:` *header* lines inside the message are separate (what the reader sees). They are usually the same, but need not be — that split is how mailing lists and `Bcc:` work.

Composing the body with mimemess

Hand-assembling headers is fine for a one-line notification, but the moment you want a subject with non-ASCII characters, an HTML alternative, or a file attachment, reach for **`TMimeMess`** (in `mimemess`). It builds a correct MIME message and serialises it to the `TStrings` that `MailData` wants:

```
uses smtpsend, mimemess, mimepart, ssl_openssl;

var
```

```
  Mime: TMimeMess;
  Body: TStringList;
begin
  Mime := TMimeMess.Create;
  Body := TStringList.Create;
  try
    Mime.Header.From := 'alice@example.com';
    Mime.Header.ToList.Add('bob@example.org');
    Mime.Header.Subject := 'Report attached';

    Body.Add('Hi Bob, see the attached report.');
    Mime.AddPartText(Body, nil);                  // text/plain part
    Mime.AddPartBinaryFromFile('report.pdf', nil);  // attachment

    Mime.EncodeMessage;        // builds the full message into Mime.Lines

    // Mime.Lines is now the complete RFC-822 message -> hand it to MailData:
    // Smtp.MailData(Mime.Lines);
  finally
    Body.Free;
    Mime.Free;
  end;
end;
```

After `EncodeMessage`, `Mime.Lines` is exactly the `TStrings` `MailData` expects, so the two compose cleanly: `TMimeMess` owns *what the message is*, `TSMTPSend` owns *getting it to the server*. See the mail-message chapter for the full `TMimeMess` treatment (parts, alternatives, inline images, charset handling).

The shortcut functions

If you just want to fire one message off and do not need the object, `smtpsend` exposes three module-level helpers that create a `TSMTPSend`, run the whole `Login/MailFrom/MailTo/MailData/Logout` dance, and free it:

```
function SendTo(const MailFrom, MailTo, Subject, SMTPHost: string;
  const MailData: TStrings): Boolean;
function SendToEx(const MailFrom, MailTo, Subject, SMTPHost: string;
  const MailData: TStrings; const Username, Password: string): Boolean;
function SendToRaw(const MailFrom, MailTo, SMTPHost: string;
  const MailData: TStrings; const Username, Password: string): Boolean;
```

`SendTo` builds the `From/To/Subject/Date` headers for you and needs only the body; `SendToEx` adds authentication; `SendToRaw` takes an already-complete message (headers and all — the natural partner for `TMimeMess.Lines`). Append a `:port` to `SMTPHost` to override the port. They are the “send and forget” path; drop to the object whenever you need TLS toggles, capability inspection, or per- recipient control.

THTTPSend — talking HTTP

`THTTPSend` (in `httpsend`) is probably the most-used class in Synapse, and it comes at two levels. For the common cases there are one-line module functions — `HttpGetText`, `HttpPostURL`, and friends — that create the object, do the transfer, and free it. When you need control over headers, methods, cookies, or the raw response, you drop to the object itself. Both sit on the same `TSynaClient`-over-`TTCPBlockSocket` foundation as the rest of the protocol clients, and both get HTTPS for free from the SSL plugin seam.

The class at a glance

`THTTPSend` diverges a little from the mail clients: there is no `Login`/`Logout` pair, because HTTP is request-response. Instead the engine is a single method, `HTTPMethod`, and everything around it is properties you set before and read after:

```
THTTPSend = class(TSynaClient)
  function HTTPMethod(const Method, URL: string): Boolean;

  property Headers: TStringList;      // request headers in / response headers out
  property Cookies: TStringList;      // parsed Set-Cookie, reused on next request
  property Document: TMemoryStream;   // request body in / response body out
  property MimeType: string;          // Content-Type for the request body
  property Protocol: string;          // '1.0' (default) or '1.1'
  property KeepAlive: Boolean;
  property UserAgent: string;
  property ResultCode: Integer;       // HTTP status code: 200, 404, 500…
  property ResultString: string;      // the status reason phrase
  property DownloadSize: int64;
  property UploadSize: int64;
  property Sock: TTCPBlockSocket;
  property ProxyHost, ProxyPort, ProxyUser, ProxyPass: string;
end;
```

The one object you touch most is **`Document`**, a `TMemoryStream`. It is bidirectional: you fill it with the request body before a POST/PUT, and after any call it holds the response body. `ResultCode` is the real HTTP status code, so you branch on it directly.

HTTPS is automatic

Here Synapse is at its quietest. `HTTPMethod` calls `ParseURL` on the URL, and if the scheme is `https`, it does the TLS handshake for you — no property to set:

```
if UpperCase(Prot) = 'HTTPS' then
  ...            // connect with SSL
```

So all HTTPS needs is an SSL plugin in `uses` and an `https://` URL:

```
uses httpsend, ssl_openssl;
...
HttpGetText('https://example.com/', Response);   // just works
```

No plugin, and an `https://` URL will fail at the handshake — the seam is present but empty. That is the whole "HTTPS wiring."

The convenience functions

For everyday work you rarely need the object. `httpsend` exposes these module-level helpers (each spins up a `THTTPSend`, runs one transfer, frees it):

```
function HttpGetText(const URL: string; const Response: TStrings): Boolean;
function HttpGetBinary(const URL: string; const Response: TStream): Boolean;
function HttpPostBinary(const URL: string; const Data: TStream): Boolean;
function HttpPostURL(const URL, URLData: string; const Data: TStream): Boolean;
function HttpPostFile(const URL, FieldName, FileName: string;
  const Data: TStream; const ResultData: TStrings): Boolean;
```

- **`HttpGetText`** — GET a URL, load the body into a `TStrings`. Perfect for an API that returns text/JSON.
- **`HttpGetBinary`** — GET into any `TStream` (a file, a memory stream) for images, downloads, binaries.
- **`HttpPostURL`** — POST form data. It sets `Content-Type: application/x-www-form-urlencoded`, sends `URLData` as the body, and returns the response in `Data`.
- **`HttpPostBinary`** — POST a raw octet-stream body.
- **`HttpPostFile`** — POST a `multipart/form-data` file upload, boundary and all.

Worked example: a quick GET

```
program HttpGet;

uses
  SysUtils, Classes,
  httpsend, ssl_openssl;   // ssl_openssl -> https:// works

var
  Response: TStringList;
begin
  Response := TStringList.Create;
  try
    if HttpGetText('https://example.com/', Response) then
      Writeln(Response.Text)
    else
      Writeln('Request failed');
  finally
    Response.Free;
  end;
end.
```

Worked example: a form POST

```
program HttpPost;

uses
  SysUtils, Classes,
  httpsend, synacode, synautil;   // synacode: EncodeURLElement; synautil: ReadStrFromStream

var
  Reply: TMemoryStream;
  FormData: string;
begin
  Reply := TMemoryStream.Create;
  try
    // application/x-www-form-urlencoded: key=value&key=value, values escaped.
    FormData := 'name=' + EncodeURLElement('Alice') +
                '&city=' + EncodeURLElement('São Paulo');

    if HttpPostURL('http://httpbin.org/post', FormData, Reply) then
    begin
      Reply.Position := 0;
      Writeln('Response:');
      Writeln(ReadStrFromStream(Reply, Reply.Size));   // synautil helper
    end
    else
      Writeln('POST failed');
  finally
    Reply.Free;
  end;
end.
```

Object-level usage: full control

When you need custom headers, a specific method, a status code, or the response headers, use `THTTPSend` directly. The pattern is: set `Headers`/`Document`/`MimeType`, call `HTTPMethod(verb, url)`, then read `ResultCode`, `Headers`, and `Document`.

```
program HttpJsonApi;

uses
  SysUtils, Classes,
  httpsend, ssl_openssl;

var
  Http: THTTPSend;
  Body: string;
begin
  Http := THTTPSend.Create;
  try
    Http.Protocol := '1.1';
    Http.MimeType := 'application/json';          // Content-Type of our body
    Http.Headers.Add('Authorization: Bearer ' + 'my-token');
    Http.Headers.Add('Accept: application/json');
```

```
    // Put the request body into Document.
    WriteStrToStream(Http.Document, '{"name":"Alice"}');  // synautil helper

    if Http.HTTPMethod('POST', 'https://api.example.com/users') then
    begin
      Writeln('Status: ', Http.ResultCode, ' ', Http.ResultString);

      // Response body is in Document; rewind and read it out.
      Http.Document.Position := 0;
      SetLength(Body, Http.Document.Size);
      if Http.Document.Size > 0 then
        Http.Document.ReadBuffer(Body[1], Http.Document.Size);
      Writeln(Body);

      // Response headers are in Headers after the call.
      Writeln('--- response headers ---');
      Writeln(Http.Headers.Text);
    end
    else
      Writeln('Transport failed (no reply)');
  finally
    Http.Free;
  end;
end.
```

A few things the source makes clear, worth knowing:

- **`Headers` is reused in both directions.** You add request headers to it before the call; after the call it holds the *response* headers (the request ones having been sent and cleared). Read them straight back out of the same list.
- **`HTTPMethod` returns the transport result, not the HTTP result.** It returns `True` when a reply was received at all — even a `404` or `500`. Always inspect `ResultCode` to learn what the server actually said; a `True` return with `ResultCode = 404` is a perfectly normal "not found."
- **`Document` is a `TMemoryStream`, not text.** For a text/JSON response, read its bytes out (as above) or `LoadFromStream` it into a `TStringList`. Remember to set `Position := 0` before reading.
- **`MimeType`** becomes the request's `Content-Type`; **`Protocol`** selects HTTP/1.0 (default) or 1.1; **`KeepAlive`** (on by default) lets you reuse one `THTTPSend` across several requests to the same host, saving reconnects.
- **Cookies** set by the server are parsed into the `Cookies` list and sent back automatically on the next request from the same object — a small session for free.
- **Proxies** go through `ProxyHost`/`ProxyPort`/`ProxyUser`/`ProxyPass`; Basic auth to the target is handled if you embed `user:pass@` in the URL.

When to use which

Reach for the **convenience function** whenever a single GET or POST with default headers will do — it is a one-liner and manages the object's lifetime for you. Reach for the **object** the moment you need custom headers, a non-GET/POST verb (PUT, DELETE, PATCH — `HTTPMethod` takes any method string), the status code, the response headers, cookie continuity, or keep-alive across calls. Both are the same engine; the functions are just `THTTPSend` with the common case pre-wired.

TIMAPSend — the mailbox client

`TIMAPSend` (in `imapsend`) is the one client in this chapter that does more than push a message through a pipe: it drives a *stateful* mailbox on the server. IMAP keeps your mail on the server and lets you browse folders, fetch individual messages or just their headers, search, and set flags — all without downloading the whole mailbox. `TIMAPSend` is Synapse's IMAP4rev1 implementation (RFC-2060, RFC-2595), and like every client here it is a thin `TSynaClient` descendant over a `TTCPBlockSocket`: set a few properties, call the verbs in order, read the result.

The class at a glance

Connection and credentials come from `TSynaClient` (`TargetHost`, `TargetPort`, `UserName`, `Password`, `OAuth2Token`). `TIMAPSend` adds the IMAP verbs and the selected-folder state:

```
TIMAPSend = class(TSynaClient)
  function Login: Boolean;            // connect, greeting, STARTTLS, LOGIN
  function Logout: Boolean;           // LOGOUT + close
  function NoOp: Boolean;             // NOOP (keep-alive)
  function Capability: Boolean;       // CAPABILITY -> fills IMAPcap

  // folders
  function List(FromFolder: string; const FolderList: TStrings): Boolean;
  function ListSubscribed(FromFolder: string; const FolderList: TStrings): Boolean;
  function ListSearch(FromFolder, Search: string; const FolderList: TStrings): Boolean;
  function CreateFolder(FolderName: string): Boolean;
  function DeleteFolder(FolderName: string): Boolean;
  function RenameFolder(FolderName, NewFolderName: string): Boolean;
  function SubscribeFolder(FolderName: string): Boolean;
  function UnsubscribeFolder(FolderName: string): Boolean;
  function SelectFolder(FolderName: string): Boolean;    // SELECT (read/write)
  function SelectROFolder(FolderName: string): Boolean;  // EXAMINE (read-only)
  function CloseFolder: Boolean;
  function StatusFolder(FolderName, Value: string): integer;
  function ExpungeFolder: Boolean;
  function CheckFolder: Boolean;

  // messages
  function FetchMess(MessID: integer; const Mess: TStrings): Boolean;     // whole message
  function FetchHeader(MessID: integer; const Headers: TStrings): Boolean;// headers only
  function MessageSize(MessID: integer): integer;
  function AppendMess(ToFolder: string; const Mess: TStrings): Boolean;
  function DeleteMess(MessID: integer): Boolean;         // mark \Deleted
```

```
  function CopyMess(MessID: integer; ToFolder: string): Boolean;
  function SearchMess(Criteria: string; const FoundMess: TStrings): Boolean;
  function GetUID(MessID: integer; var UID: Integer): Boolean;

  // flags
  function GetFlagsMess(MessID: integer; var Flags: string): Boolean;
  function SetFlagsMess(MessID: integer; Flags: string): Boolean;   // replace
  function AddFlagsMess(MessID: integer; Flags: string): Boolean;   // +FLAGS
  function DelFlagsMess(MessID: integer; Flags: string): Boolean;   // -FLAGS

  function StartTLS: Boolean;
  function FindCap(const Value: string): string;

  property ResultString: string;        // status line of the last operation
  property FullResult: TStringList;      // every reply line of the last operation
  property IMAPcap: TStringList;         // server capabilities
  property AuthDone: Boolean;
  property UID: Boolean;                 // interpret MessID as UID, not sequence no.
  property SelectedFolder: string;
  property SelectedCount: integer;       // messages in the selected folder
  property SelectedRecent: integer;
  property SelectedUIDvalidity: integer;
  property AutoTLS: Boolean;
  property FullSSL: Boolean;
  property Sock: TTCPBlockSocket;
end;
```

Note what is *not* here. Unlike SMTP or HTTP there is no numeric `ResultCode` — IMAP replies are tagged text (`OK`, `NO`, `BAD`), and the source models that directly: the internal `IMAPcommand` returns the uppercase status word and every public verb boils it down to `= 'OK'`. So a method returning `True` means the server tagged the command `OK`; when it returns `False`, `ResultString` holds the raw tagged line the server sent, which is your diagnostic.

What `Login` does for you

Reading the source, `Login` folds the whole session opening into one call:

1. Connects (doing an immediate TLS handshake first if `FullSSL` is set).
2. Reads the server greeting. `* PREAUTH` means the connection is already authenticated (and sets `AuthDone`); `* OK` means proceed to login; anything else fails.
3. Runs `CAPABILITY` and requires the server to advertise `IMAP4rev1` — Synapse will not talk to a server that does not claim the base protocol.
4. If `AutoTLS` is set and the server advertises `STARTTLS`, upgrades the live connection to TLS and re-reads the capabilities.
5. Authenticates. With an `OAuth2Token` set it sends `AUTHENTICATE XOAUTH2`; otherwise it sends a plain `LOGIN "user" "pass"` (user and password escaped for the quoting rules — see below). `AuthDone` reports whether auth succeeded.

FullSSL vs. STARTTLS

TLS is the usual SSL plugin seam — one unit in `uses`, then pick implicit or explicit, exactly as described in the chapter intro:

```
uses imapsend, ssl_openssl;
```

- **IMAPS (implicit).** Set `FullSSL := True`, `TargetPort := '993'`. The TLS handshake happens the instant the socket connects.
- **STARTTLS (explicit).** Set `AutoTLS := True`, keep the plain port `143`. `Login` upgrades the connection *before* it sends `LOGIN`, so your credentials travel encrypted. You can also drive the upgrade yourself with the public `StartTLS` method (it checks `FindCap('STARTTLS')` first and returns `False` if the server never advertised it).

  Honest caveat. `LOGIN` sends your username and password in the clear (IMAP does no hashing of its own here). Only ever log in over a TLS session — which is exactly why `Login` performs STARTTLS *before* `LOGIN` when `AutoTLS` is on.

Sequence numbers vs. UIDs

Every message verb takes a `MessID: integer`, and the `UID` property decides how the server reads it. With `UID := False` (the default), `MessID` is the message's *sequence number* within the selected folder — `1` is the first message, `SelectedCount` is the last. Sequence numbers are stable only for the current selection: expunging a message renumbers everything after it. With `UID := True`, Synapse prefixes each command with `UID` and `MessID` is the message's permanent unique identifier — stable across sessions, paired with `SelectedUIDvalidity` to detect a folder that was deleted and recreated. Reach for UIDs the moment you need an identifier that survives past the current `SELECT`.

Selecting a folder is a mode switch

IMAP is stateful: most message verbs only work once a folder is *selected*.

- **`SelectFolder`** issues `SELECT` and enters read/write state — flags and deletions take effect. After it returns, `SelectedCount`, `SelectedRecent`, and `SelectedUIDvalidity` describe the folder, and `SelectedFolder` names it.
- **`SelectROFolder`** issues `EXAMINE` — the same, but read-only, so you can browse without marking anything `\Seen`.
- **`CloseFolder`** leaves selected state (and silently expunges `\Deleted` messages, per the protocol).

`List` fills a `TStrings` with folder *names* (it filters out `\Noselect` containers and un-escapes quoted names for you); pass an empty `FromFolder` to list everything. `ListSubscribed` does the same over `LSUB`, and `ListSearch` takes a wildcard pattern.

Deleting is two steps

`DeleteMess` does **not** remove a message — it sets the `\Deleted` flag (`STORE … +FLAGS.SILENT (\Deleted)`). The message physically goes away only when you call `ExpungeFolder` (or `CloseFolder`, which expunges on its way out). That two-step is IMAP's design, not a Synapse quirk: it lets a client "undelete" by clearing the flag before the expunge. The flag verbs are the general form — `AddFlagsMess`/`DelFlagsMess` add or remove flags (`\Seen`, `\Flagged`, `\Answered`, …), `SetFlagsMess` replaces the whole set, and `GetFlagsMess` reads it back.

Worked example: fetch a message from a folder

The core rhythm is `Login → SelectFolder → Fetch → Logout`:

```
program ImapFetch;

uses
  SysUtils, Classes,
  imapsend, ssl_openssl;   // ssl_openssl enables IMAPS / STARTTLS

var
  Imap: TIMAPSend;
  Msg: TStringList;
begin
  Imap := TIMAPSend.Create;
  Msg  := TStringList.Create;
  try
    Imap.TargetHost := 'imap.example.com';
    Imap.TargetPort := '993';        // IMAPS
    Imap.FullSSL    := True;          // implicit TLS from the first byte
    Imap.UserName   := 'alice@example.com';
    Imap.Password   := 's3cret';

    if not Imap.Login then
    begin
      Writeln('Login failed: ', Imap.ResultString);
      Halt(1);
    end;

    // Enter the mailbox. After this, SelectedCount is the message count.
    if not Imap.SelectFolder('INBOX') then
    begin
      Writeln('Cannot select INBOX: ', Imap.ResultString);
      Imap.Logout;
      Halt(1);
    end;
    Writeln('INBOX holds ', Imap.SelectedCount, ' message(s).');

    if Imap.SelectedCount > 0 then
    begin
      // Fetch the newest message (highest sequence number) in full.
      if Imap.FetchMess(Imap.SelectedCount, Msg) then
      begin
        Writeln('--- message ', Imap.SelectedCount, ' ---');
        Writeln(Msg.Text);        // full RFC-822 message: headers + body
      end
      else
        Writeln('Fetch failed: ', Imap.ResultString);
    end;

    Imap.Logout;
  finally
    Msg.Free;
    Imap.Free;
  end;
end.
```

`FetchMess` retrieves the whole message (`FETCH n (RFC822)`) into the `TStrings` as a complete

RFC-822 message — headers, blank line, body. When you only need the envelope (a message-list view, say), `FetchHeader` fetches just the headers (`RFC822.HEADER`), and `MessageSize` returns the byte count without transferring anything. To turn the fetched `Msg` into structured headers, alternatives, and attachments, hand it to `TMimeMess.DecodeMessage` — the mail-message chapter covers that; `TIMAPSend` owns *getting the bytes*, `TMimeMess` owns *what they mean*.

Searching

`SearchMess` sends IMAP's `SEARCH` and returns the matching message numbers (or UIDs, if `UID` is set) as a `TStrings`. The criteria string is IMAP's own search language — powerful, SQL-ish, but not SQL:

```
var
  Hits: TStringList;
begin
  Hits := TStringList.Create;
  try
    Imap.SelectFolder('INBOX');
    // messages from a given sender that are still unseen
    if Imap.SearchMess('FROM "bob@example.org" UNSEEN', Hits) then
      Writeln(Hits.Count, ' match(es): ', Hits.CommaText);
  finally
    Hits.Free;
  end;
end;
```

Each entry in `Hits` is a message number you can pass straight to `FetchMess`, `GetFlagsMess`, `CopyMess`, or `DeleteMess`.

Escaping and the raw command

IMAP folder and mailbox names are quoted strings, and characters like `"` and `\` must be escaped (RFC-2683). Synapse handles this for you inside every verb via `EscapeSpecialCharacters`, and un-escapes returned names via `UnescapeSpecialCharacters`; both are public if you build commands by hand. And you can always drop to the raw protocol: `IMAPcommand(cmd)` sends any tagged command and returns the status word, leaving the reply in `FullResult` — the escape hatch for server-specific extensions Synapse does not wrap.

When to use which verb

`SelectFolder`/`SelectROFolder` gate everything — call one first. Then `FetchHeader` + `MessageSize` to *browse* cheaply, `FetchMess` to *read* in full, `SearchMess` to *find*, the flag verbs to *mark*, and `DeleteMess` + `ExpungeFolder` (or `CloseFolder`) to *remove*. `NoOp` keeps an idle connection from timing out. It is the same blocking, one-verb-at-a-time rhythm as the rest of the chapter — just with a folder's worth of state hanging off the client between calls.

TFTPSend — file transfer over two connections

`TFTPSend` (in `ftpsend`) is the classic FTP client, and it is the one client in this chapter with a twist the others do not have: FTP uses *two* sockets. The **control** connection carries commands and replies; a separate **data** connection carries each file or listing. `TFTPSend` manages both — it owns two `TTCPBlockSockets` (`Sock` for control, `DSock` for data) — but the two-channel dance is mostly hidden behind ordinary-looking verbs. As always it descends from `TSynaClient`, so you set a few properties and call methods in order.

The class at a glance

Connection and credentials come from `TSynaClient`. Uniquely, the constructor pre-fills anonymous credentials (`UserName := 'anonymous'`, `Password := 'anonymous@<localname>'`), so a public server needs nothing but a host. `TFTPSend` adds the FTP verbs plus the data-channel machinery:

```
TFTPSend = class(TSynaClient)
  function Login: Boolean;            // connect, greeting, AUTH TLS, logon sequence
  function Logout: Boolean;           // QUIT + close
  function FTPCommand(const Value: string): integer;   // raw command -> reply code

  // transfers
  function RetrieveFile(const FileName: string; Restore: Boolean): Boolean;  // RETR
  function StoreFile(const FileName: string; Restore: Boolean): Boolean;     // STOR
  function StoreUniqueFile: Boolean;                                         // STOU
  function AppendFile(const FileName: string): Boolean;                      // APPE
  function List(Directory: string; NameList: Boolean): Boolean;             // LIST/NLST/MLSD

  // filesystem
  function RenameFile(const OldName, NewName: string): Boolean;
  function DeleteFile(const FileName: string): Boolean;
  function FileSize(const FileName: string): int64;                          // SIZE
  function ChangeWorkingDir(const Directory: string): Boolean;              // CWD
  function ChangeToParentDir: Boolean;                                       // CDUP
  function ChangeToRootDir: Boolean;
  function DeleteDir(const Directory: string): Boolean;                      // RMD
  function CreateDir(const Directory: string): Boolean;                      // MKD
  function GetCurrentDir: String;                                            // PWD
  function NoOp: Boolean;

  // low-level data channel (only for custom commands)
```

```
  function DataRead(const DestStream: TStream): Boolean;
  function DataWrite(const SourceStream: TStream): Boolean;
  procedure Abort;
  procedure TelnetAbort;

  property ResultCode: Integer;       // numeric reply code of the last command
  property ResultString: string;      // its main reply line
  property FullResult: TStringList;    // every line of the reply
  property Sock: TTCPBlockSocket;      // control connection
  property DSock: TTCPBlockSocket;     // data connection
  property DataStream: TMemoryStream;  // in-memory transfer buffer (default sink/source)
  property DirectFile: Boolean;        // transfer straight to/from a disk file instead
  property DirectFileName: string;
  property FtpList: TFTPList;          // parsed directory listing after List(...)
  property PassiveMode: Boolean;       // default True
  property BinaryMode: Boolean;        // default True (TYPE I); False = ASCII
  property CanResume: Boolean;         // set by Login if the server supports REST
  property AutoTLS: Boolean;           // explicit FTPS (AUTH TLS)
  property FullSSL: Boolean;           // implicit FTPS (TLS from connect)
  property IsTLS: Boolean;             // control channel is encrypted
  property IsDataTLS: Boolean;         // data channel is encrypted
  property TLSonData: Boolean;         // default True: encrypt data too
  property ForceDefaultPort: Boolean;
  property ForceOldPort: Boolean;      // disable EPSV/EPRT (breaks IPv6)
  property UseMLSDList: Boolean;
  // plus firewall/proxy: FWHost, FWPort, FWUsername, FWPassword, FWMode, Account
end;
```

FTP reply codes are the real three-digit numbers, so `ResultCode` reads directly (`200` OK, `226` transfer complete, `550` no such file). Internally almost every verb is `(FTPCommand(...) div 100) = 2` — "did the server answer in the 2xx family?" — which is why you can reason about `ResultCode` the same way the class does.

What `Login` does for you

Reading the source, `Login`:

1. Connects the control socket (immediate TLS handshake if `FullSSL` is set).
2. Reads the greeting, waiting past any `1xx` preliminary lines for the real reply; a non-2xx greeting fails.
3. If `AutoTLS` is set, sends `AUTH TLS` and upgrades the control connection.
4. Runs the **logon sequence** — `USER` / `PASS` / `ACCT` — driven by `FWMode`. With no firewall (`FWHost` empty) it is the plain three-step login; the other modes thread the login through a firewall/proxy, and `FWMode := -1` runs a `CustomLogon` sequence you supply.
5. On a TLS session, negotiates data-channel protection (`PBSZ 0`, then `PROT P` if `TLSonData`, else `PROT C`).
6. Sets binary type (`TYPE I`), stream mode, and *probes* for resume support with `REST` — setting `CanResume` accordingly.

The data-connection model

Every transfer — a file or a directory listing — needs a fresh data connection, and there are two ways to open one. This is the single most important thing to understand about FTP, and

`PassiveMode` (default **`True`**) picks between them:

- **Passive (`PassiveMode := True`).** The client asks the server "what port should I connect to?" (`PASV`, or `EPSV` for IPv6), the server answers with an address, and the *client* opens the data connection. Because the client makes both outbound connections, passive mode sails through client-side firewalls and NAT — which is why it is the default and almost always the right choice.
- **Active (`PassiveMode := False`).** The client opens a listening socket and tells the server where to connect back (`PORT`, or `EPRT` for IPv6); the *server* dials in. This needs the client to be reachable from the server, so it often trips over NAT. `ForceDefaultPort` pins the listen to port 20; `ForceOldPort` disables the `EPSV`/`EPRT` extensions (which also disables IPv6).

You rarely touch any of this. The high-level verbs (`RetrieveFile`, `StoreFile`, `List`) open, use, and close the data connection internally via `DataSocket` / `DataRead` / `DataWrite`. Those three lower-level methods are exposed only for the rare case of implementing an unsupported command by hand.

DataStream vs. DirectFile

Where do the bytes go? Two modes, chosen by `DirectFile`:

- **`DirectFile := False` (default).** All transfers run through `DataStream`, an in-memory `TMemoryStream`. After `RetrieveFile` it holds the downloaded bytes (rewound to position 0); before `StoreFile` you fill it with what to upload. Simple, and fine for anything that fits in RAM.
- **`DirectFile := True`.** Transfers stream straight to or from a disk file named by `DirectFileName` — a `TFileStream` under the hood — so a multi-gigabyte download never has to fit in memory. This is the mode the convenience functions use.

FTPS: explicit and implicit

TLS is again the SSL plugin seam — add a plugin, then choose:

```
uses ftpsend, ssl_openssl;
```

- **Explicit FTPS (`AUTH TLS`).** Set `AutoTLS := True` on the standard port 21. `Login` upgrades the control connection after connecting, then negotiates data-channel encryption. This is the modern default for secure FTP.
- **Implicit FTPS.** Set `FullSSL := True` (historically port 990); TLS starts at connect time.
- **`TLSonData`** (default `True`) controls whether the *data* connection is encrypted too. `IsTLS` and `IsDataTLS` report what actually happened after `Login`.

  Honest caveat. Plain FTP sends your password — and every byte of every file — in the clear. Prefer `AutoTLS := True`, and check `IsTLS` (and `IsDataTLS` if the payload is sensitive) before trusting the session.

Worked example: download a file

The rhythm is `Login → (navigate) → RetrieveFile → Logout`. Here streaming straight to disk with `DirectFile`:

```
program FtpDownload;
```

```
uses
  SysUtils, Classes,
  ftpsend, ssl_openssl;   // ssl_openssl enables AUTH TLS / FTPS

var
  Ftp: TFTPSend;
begin
  Ftp := TFTPSend.Create;
  try
    Ftp.TargetHost := 'ftp.example.com';
    // TargetPort defaults to 21; UserName/Password default to anonymous.
    Ftp.UserName := 'alice';
    Ftp.Password := 's3cret';
    Ftp.AutoTLS  := True;             // explicit FTPS

    if not Ftp.Login then
    begin
      Writeln('Login failed (', Ftp.ResultCode, '): ', Ftp.ResultString);
      Halt(1);
    end;

    Ftp.ChangeWorkingDir('/pub/reports');

    // Stream the download straight to a local file.
    Ftp.DirectFileName := 'report.pdf';
    Ftp.DirectFile     := True;

    if Ftp.RetrieveFile('report.pdf', False) then   // Restore=False: fresh download
      Writeln('Downloaded. Server said: ', Ftp.ResultString)
    else
      Writeln('Download failed (', Ftp.ResultCode, '): ', Ftp.ResultString);

    Ftp.Logout;
  finally
    Ftp.Free;
  end;
end.
```

Set `Restore := True` and, if the server advertised resume support (`CanResume`), `RetrieveFile` picks up where an interrupted download left off — it seeks to the end of the existing local file and sends `REST`. If you would rather keep the bytes in memory, leave `DirectFile := False` and read them from `DataStream` after the call (remember it is already rewound to position 0).

Worked example: upload a file

`StoreFile` is the mirror image. In memory this time:

```
program FtpUpload;

uses
  SysUtils, Classes,
  ftpsend, ssl_openssl;

var
```

```
  Ftp: TFTPSend;
begin
  Ftp := TFTPSend.Create;
  try
    Ftp.TargetHost := 'ftp.example.com';
    Ftp.UserName   := 'alice';
    Ftp.Password   := 's3cret';
    Ftp.AutoTLS    := True;

    if not Ftp.Login then
    begin
      Writeln('Login failed: ', Ftp.ResultString);
      Halt(1);
    end;

    // Fill DataStream with the payload (DirectFile stays False).
    Ftp.DataStream.LoadFromFile('local-notes.txt');

    if Ftp.StoreFile('notes.txt', False) then
      Writeln('Uploaded. Server said: ', Ftp.ResultString)
    else
      Writeln('Upload failed (', Ftp.ResultCode, '): ', Ftp.ResultString);

    Ftp.Logout;
  finally
    Ftp.Free;
  end;
end.
```

Related store verbs: `AppendFile` (`APPE`) appends to an existing remote file instead of overwriting, and `StoreUniqueFile` (`STOU`) lets the server assign a unique name. Set `BinaryMode := False` before a transfer to switch to ASCII mode (`TYPE A`, which translates line endings) — leave it at the default `True` for anything that is not plain text.

Listing a directory

`List` fetches a directory over the data connection and, for a full listing, also *parses* it:

```
var
  i: Integer;
  rec: TFTPListRec;
begin
  if Ftp.List('/pub', False) then          // NameList=False -> full parsed listing
    for i := 0 to Ftp.FtpList.Count - 1 do
    begin
      rec := Ftp.FtpList.Items[i];
      if rec.Directory then
        Writeln(Format('%-30s  <dir>', [rec.FileName]))
      else
        Writeln(Format('%-30s %12d', [rec.FileName, rec.FileSize]));
    end;
end;
```

With `NameList := False`, `List` issues `LIST` (or `MLSD` if you set `UseMLSDList := True`) and parses

each line into `FtpList`, a `TFTPList` of `TFTPListRec` — each record exposing `FileName`, `FileSize`, `Directory`, `FileTime`, `Permission`, and the `OriginalLine`. Directory listing formats are a notorious swamp (Unix, Windows, Novell, VMS…), and Synapse ships a table of masks to cope; lines it cannot parse land in `FtpList.UnparsedLines`. With `NameList := True` it issues `NLST` and you get bare names in `DataStream`, no parsing. `MLSD` (RFC-3659) is the machine-readable modern format — prefer it when the server supports it, since it sidesteps the parsing guesswork entirely.

The convenience functions

For a one-shot transfer, `ftpsend` exposes three module-level helpers that create a `TFTPSend`, run the whole `Login`/transfer/`Logout`, and free it:

```
function FtpGetFile(const IP, Port, FileName, LocalFile, User, Pass: string): Boolean;
function FtpPutFile(const IP, Port, FileName, LocalFile, User, Pass: string): Boolean;
function FtpInterServerTransfer(
  const FromIP, FromPort, FromFile, FromUser, FromPass: string;
  const ToIP, ToPort, ToFile, ToUser, ToPass: string): Boolean;
```

`FtpGetFile` and `FtpPutFile` are the download/upload examples above in one line — they use `DirectFile` internally, so they stream to/from `LocalFile` on disk. Pass an empty `User` to log in anonymously. `FtpInterServerTransfer` wires two servers together with a `PASV`/`PORT` pair so bytes flow server-to-server without passing through your machine (the classic "FXP" transfer). Reach for the functions for the common case; drop to the object the moment you need TLS toggles, resume, directory navigation, or the parsed listing.

A note on `ftptsend` — a different protocol

Do not confuse `ftpsend` with **`ftptsend`** (class `TTFTPSend`). Despite the near-identical name, TFTP — *Trivial* File Transfer Protocol (RFC-1350) — is a completely separate, minimal protocol that runs over **UDP**, with no login, no directory listing, and no security. `TTFTPSend` is its client *and* server, with `SendFile`/`RecvFile` and a `Data` buffer, and it is used mostly for bootstrapping network devices, not general file transfer. If you mean ordinary FTP, you want `ftpsend` / `TFTPSend` — this chapter.

TDNSSend

DNS is the one protocol client in this chapter that does *not* speak a line-based request/response conversation over TCP. It is a single binary packet out, a single binary packet back, over UDP. `TDNSSend` (in `dnssend`) hides all of that wire encoding behind one method — `DNSQuery` — and hands you back the answers as plain strings, one resource record per line. There is no `Login`, no `Logout`, no session: you create the object, point it at a resolver, and ask.

The class at a glance

Like every client in this chapter, `TDNSSend` descends from `TSynaClient`, so the connection knobs (`TargetHost`, `TargetPort`, `Timeout`, `IPInterface`) come from the base. The constructor sets `TargetPort` to 53. What `TDNSSend` adds is one query method and a set of read-only result holders:

```
TDNSSend = class(TSynaClient)
  constructor Create;
  destructor Destroy; override;

  {: Query TargetHost for QType records matching Name. Answers land in Reply,
     one record per line; multi-field records are comma-delimited. }
  function DNSQuery(Name: AnsiString; QType: Integer;
    const Reply: TStrings): Boolean;

  property Sock: TUDPBlockSocket;      // the UDP socket (for OnStatus, tuning)
  property TCPSock: TTCPBlockSocket;   // the TCP socket, used only when UseTCP
  property UseTCP: Boolean;            // send over TCP instead of UDP
  property RCode: Integer;             // DNS reply code: 0 ok, 3 name error, ...
  property Authoritative: Boolean;     // answer came from an authoritative server
  property Truncated: Boolean;         // reply was truncated (see "UDP vs TCP")
  property AnswerInfo: TStringList;    // detailed answer-section records
  property NameserverInfo: TStringList;// detailed authority-section records
  property AdditionalInfo: TStringList;// detailed additional-section records
end;
```

The `QType` argument is one of the `QTYPE_*` constants the unit exports. The ones you will reach for:

Constant	Value	Record	Asks for
`QTYPE_A`	1	A	IPv4 address of a host
`QTYPE_NS`	2	NS	authoritative name servers for a zone
`QTYPE_CNAME`	5	CNAME	canonical name (alias target)
`QTYPE_SOA`	6	SOA	zone start-of-authority

Constant	Value	Record	Asks for
`QTYPE_PTR`	12	PTR	host name for an address (reverse lookup)
`QTYPE_MX`	15	MX	mail exchangers for a domain
`QTYPE_TXT`	16	TXT	free-form text (SPF, verification, …)
`QTYPE_AAAA`	28	AAAA	IPv6 address of a host
`QTYPE_SRV`	33	SRV	service location
`QTYPE_AXFR`	252	—	full zone transfer (**TCP only**)
`QTYPE_ALL`	255	*	any/all records

The full list (`QTYPE_MD`, `QTYPE_HINFO`, `QTYPE_KX`, and the rest) is in the `const` block at the top of `dnssend.pas`.

How answers are shaped

`DNSQuery` returns `True` when it got a well-formed reply with `RCode = 0`, and fills `Reply` with the answer records. The encoding is deliberately flat: **one record per line**, and where a record has several fields they are joined with commas. An `MX` record with preference `10` for `mail.example.com`, for example, arrives as the single line:

```
10,mail.example.com
```

An `SRV` record comes back as `priority,weight,port,target`. A and AAAA records are just the address as text; PTR and CNAME are just the name. Numbers and IP addresses are always converted to their string form for you — you never decode raw bytes.

The three `*Info` string lists give you the *detailed* view of the reply's three sections (answer, authority, additional). Each line there is a comma-delimited `name,type,ttl,data` tuple, so if you need the TTL or the record type number alongside the value, read `AnswerInfo` instead of `Reply`.

Reverse lookups are automatic

You do not build `.in-addr.arpa` names by hand. If the `Name` you pass to `DNSQuery` is a literal IP address, `TDNSSend` detects that and rewrites it into the correct reverse-DNS form for you — `1.2.3.4` becomes `4.3.2.1.in-addr.arpa`, and an IPv6 literal becomes the matching `.ip6.arpa` name. So a PTR lookup is just "pass the address with `QTYPE_PTR`"; the class does the inversion.

UDP vs TCP — and the truncation caveat

By default `TDNSSend` sends the query as a **single UDP datagram** and reads a **single UDP packet** back. That is the right transport for ordinary lookups and what you almost always want.

Two things push you to TCP:

- **Zone transfers.** `QTYPE_AXFR` streams an entire zone and only works over TCP; `DNSQuery` has special multi-packet handling for it, but *only* when `UseTCP` is set.
- **Truncated replies.** A UDP answer that overflows is flagged by the server, and `TDNSSend` surfaces that in the read-only `Truncated` property.

  Honest caveat — no automatic TCP fallback. Unlike a full stub resolver, `TDNSSend` does **not** silently retry a truncated UDP answer over TCP. It exposes `Truncated` and

leaves the decision to you: if you see it set, set `UseTCP := True` and call `DNSQuery` again to get the complete reply over the TCP socket. `UseTCP` simply switches which socket the query uses; there is no hybrid mode.

Worked example: resolve a host (A record)

```
program DnsA;

uses
  SysUtils, Classes,
  dnssend;

var
  Dns: TDNSSend;
  Reply: TStringList;
  i: Integer;
begin
  Dns := TDNSSend.Create;
  Reply := TStringList.Create;
  try
    Dns.TargetHost := '8.8.8.8';          // any resolver you trust

    if Dns.DNSQuery('www.example.com', QTYPE_A, Reply) then
      for i := 0 to Reply.Count - 1 do
        Writeln('A: ', Reply[i])          // each line is one IPv4 address
    else
      Writeln('query failed, RCode=', Dns.RCode);
  finally
    Reply.Free;
    Dns.Free;
  end;
end.
```

`TargetHost` here is the *resolver* you send the question to (a recursive server such as your ISP's, `8.8.8.8`, `1.1.1.1`, ...), not the host you are asking about. That distinction is the whole mental model of the class: `TDNSSend` is a DNS *client* that talks to a *server*; it does not itself walk the delegation tree.

Worked example: look up mail exchangers (MX)

An MX query returns `preference,hostname` lines, and for real use you want them tried lowest-preference first. You can sort them yourself, but the unit already ships a convenience that does exactly that:

```
{: Query DNSHost for the MX records of Domain, and return the mail-server
   host names in Servers, already sorted by preference (best first) and with
   the preference numbers stripped. }
function GetMailServers(const DNSHost, Domain: AnsiString;
  const Servers: TStrings): Boolean;
```

Using it:

```
program MxLookup;
```

```
uses
  SysUtils, Classes,
  dnssend;

var
  Servers: TStringList;
  i: Integer;
begin
  Servers := TStringList.Create;
  try
    // DNSHost = resolver to ask; second arg = domain whose mail we want.
    if GetMailServers('8.8.8.8', 'example.com', Servers) then
      for i := 0 to Servers.Count - 1 do
        Writeln(i + 1, '. ', Servers[i])   // already in preference order
    else
      Writeln('no MX records found');
  finally
    Servers.Free;
  end;
end.
```

If you want the raw preference numbers, skip the helper and call `DNSQuery(Domain, QTYPE_MX, Reply)` directly — each line is then `pref,host`, and you sort them yourself. `GetMailServers` is just that call plus a normalise-and-sort pass, and reading its (short) body in `dnssend.pas` is the best possible tutorial on driving `DNSQuery` for a multi-field record.

No TLS here

Note what is *absent* from this chapter: there is no `ssl_*` plugin, no `FullSSL`, no `AutoTLS`. Classic DNS over UDP/53 is unencrypted, and `TDNSSend` implements exactly that. DoT / DoH are different protocols and are not part of this class. If you need the transport authenticated or private, that is a layer above what `TDNSSend` provides.

The rest of the protocol clients

The chapters so far worked through the clients most programs reach for. Synapse ships several more, and every one of them follows the same shape you have already learned: a `TSynaClient` descendant, a socket exposed as `Sock`, a default port set by the constructor, and a small set of verbs. This chapter is a guided tour of six of them — NNTP, SNMP, SNTP, ping, telnet, and syslog — with enough of each to place it and a minimal example. When you need the details, the unit source is short and readable in every case.

Everything below is verified against the units in the Synapse source (one unit per protocol).

NNTP — Usenet news (`nntpsend`, `TNNTPSend`, port 119)

What it's for. Reading and posting Usenet articles: selecting a newsgroup, downloading articles by number or message-id, listing groups, and posting.

Key calls. `Login` / `Logout` bracket the session (the same connect-and-greet rhythm as the mail clients; `Login` also runs `AUTHINFO USER`/`PASS` if you set `UserName`/`Password`, and will `STARTTLS` when `AutoTLS` is set and the server advertises it). Then `SelectGroup(name)` switches group, `GetArticle(id)` / `GetBody(id)` / `GetHead(id)` download into the `Data` string list, and `PostArticle` posts the article you have placed in `Data`. Results come back as `ResultCode` / `ResultString`, with multi-line payloads in `Data`.

```
uses nntpsend;

var
  Nntp: TNNTPSend;
  i: Integer;
begin
  Nntp := TNNTPSend.Create;
  try
    Nntp.TargetHost := 'news.example.com';
    if Nntp.Login then
    begin
      if Nntp.SelectGroup('comp.lang.pascal.misc') then
        Writeln('group selected: ', Nntp.ResultString);

      if Nntp.GetArticle('1') then          // article #1 -> Data
        for i := 0 to Nntp.Data.Count - 1 do
          Writeln(Nntp.Data[i]);

      Nntp.Logout;
    end;
```

```
  finally
    Nntp.Free;
  end;
end.
```

TLS is available exactly as elsewhere: add an `ssl_*` unit and set `FullSSL` (implicit) or `AutoTLS` (STARTTLS upgrade). `StartTLS` is public if you want to drive the upgrade yourself.

SNMP — device monitoring (`snmpsend`, `TSNMPSend`, port 161)

What it's for. Reading and writing values (MIB objects, addressed by OID) on network devices — switches, printers, UPSes — using a *community string* as the access credential.

Key calls. The class `TSNMPSend` owns a `Query` and a `Reply` record (`TSNMPRec`, each carrying a `Community`, a `PDUType`, and a list of MIB entries), and `SendRequest` performs one round trip. But for the common case you never touch the class directly — the unit exports three convenience functions that build the record, send, and read the answer for you:

```
function SNMPGet(const OID, Community, SNMPHost: AnsiString;
  var Value: AnsiString): Boolean;
function SNMPGetNext(var OID: AnsiString; const Community, SNMPHost: AnsiString;
  var Value: AnsiString): Boolean;
function SNMPSet(const OID, Community, SNMPHost, Value: AnsiString;
  ValueType: Integer): Boolean;
```

Reading a single value is one line:

```
uses snmpsend, asn1util;   // asn1util for the ASN1_* value-type constants

var
  V: AnsiString;
begin
  // sysDescr.0 with the read community 'public'
  if SNMPGet('1.3.6.1.2.1.1.1.0', 'public', '192.168.1.1', V) then
    Writeln('sysDescr = ', V);

  // Writing needs the value's ASN.1 type, e.g. ASN1_OCTSTR for a string,
  // ASN1_INT for an integer:
  // SNMPSet('1.3.6.1.2.1.1.5.0', 'private', '192.168.1.1', 'newname', ASN1_OCTSTR);
end.
```

> **Honest caveat.** These functions default to SNMP **v1/v2c**, whose community string is sent in clear text and is not real security. (`TSNMPRec` does carry v3 username/auth fields for the SNMPv3 path — the `Community` string is unused there — but that is well beyond a one-liner.) The `ValueType` argument to `SNMPSet` is one of the `ASN1_*` constants from `asn1util` (`ASN1_INT = $02`, `ASN1_OCTSTR = $04`, `ASN1_OBJID = $06`, …).

The unit also offers `SNMPGetTable` / `SNMPGetTableElement` for walking MIB tables, and `SendTrap` / `RecvTrap` for the trap side (port 162).

SNTP — network time (`sntpsend`, `TSNTPSend`, port 123)

What it's for. Asking an NTP server for the current time. It is a single UDP exchange, so like DNS there is no login — you create, call, read.

Key call. `GetSNTP` sends the request and, on success, fills `NTPTime` (a `TDateTime`, in UTC) and the raw `NTPReply` record. `GetNTP` does the fuller four-timestamp exchange that also yields `NTPOffset` and `NTPDelay`; `GetBroadcastNTP` listens for a broadcast time packet. If you set `SyncTime := True`, a successful call will also set the local clock (subject to `MaxSyncDiff`) — which of course needs the privilege to do so.

```
uses sntpsend;

var
  Sntp: TSNTPSend;
begin
  Sntp := TSNTPSend.Create;
  try
    Sntp.TargetHost := 'pool.ntp.org';
    if Sntp.GetSNTP then
      Writeln('server time (UTC): ', DateTimeToStr(Sntp.NTPTime))
    else
      Writeln('no reply');
  finally
    Sntp.Free;
  end;
end.
```

Ping — ICMP echo (`pingsend`, `TPINGSend`, no TCP port)

What it's for. Testing reachability and round-trip time with an ICMP echo ("ping"). Because ICMP is not TCP or UDP, this client uses a raw socket rather than a normal port.

Key call. `Ping(host)` sends one echo and returns `True` on a reply, filling `PingTime` (ms), `ReplyFrom`, and `ReplyError`. For a throwaway check the unit exports a convenience:

```
function PingHost(const Host: string): Integer;   // ms, or -1 on failure/timeout
```

```
uses pingsend;

var
  Ms: Integer;
begin
  Ms := PingHost('example.com');
  if Ms >= 0 then
    Writeln('round trip: ', Ms, ' ms')
  else
    Writeln('no reply');
end.
```

> **Honest caveat — needs privileges.** Raw ICMP sockets are privileged. On Linux this means running as root or granting the binary the `cap_net_raw` capability; on Windows it needs the analogous rights. Without them the raw socket cannot be created and the ping fails regardless of whether the host is up. (On Windows, `TPINGSend`

can fall back to the IP Helper `IcmpSendEcho` path internally, but the privilege point still stands.) The unit also exports `TraceRouteHost` for a traceroute-style probe.

Telnet — terminal sessions (`tlntsend`, `TTelnetSend`, port 23)

What it's for. Driving a line-oriented telnet server: connecting, waiting for prompts, sending commands, and reading the response. `TTelnetSend` handles telnet option negotiation (IAC sequences) transparently and accumulates everything it sees in `SessionLog`.

Key calls. `Login` connects (it is just a connect — telnet has no protocol login step; you authenticate by scripting the login prompt yourself). Then `WaitFor(text)` blocks until `text` appears in the stream, `Send(text)` sends input, and `RecvString` reads a CRLF-terminated line. `Logout` closes the socket.

```
uses tlntsend;

var
  Tel: TTelnetSend;
begin
  Tel := TTelnetSend.Create;
  try
    Tel.TargetHost := '192.168.1.1';
    if Tel.Login then
    begin
      Tel.WaitFor('login:');
      Tel.Send('admin' + #13#10);
      Tel.WaitFor('Password:');
      Tel.Send('secret' + #13#10);
      Tel.WaitFor('$');            // shell prompt
      Tel.Send('uptime' + #13#10);
      Writeln(Tel.RecvString);
      Tel.Logout;
    end;
  finally
    Tel.Free;
  end;
end.
```

Caveat. Telnet is plaintext — credentials and session both cross the wire unencrypted. `TTelnetSend` does expose an `SSHLogin` method that reuses the socket's SSL layer to negotiate an SSHv2 transport (via the libssh2 plugin), if you have that plugin in `uses`; plain `Login` gives you classic telnet.

Syslog — remote logging (`slogsend`, `TSyslogSend`, port 514, UDP)

What it's for. Shipping a log line to a syslog collector as a single UDP packet (RFC 3164, with RFC 5424 also supported by the message class).

Key call. `TSyslogSend` carries a `SysLogMessage` (a `TSyslogMessage` with `Facility`, `Severity`,

Tag/AppName, and LogMessage), and DoIt sends it. For a one-shot line the unit exports a convenience wrapper:

```
function ToSysLog(const SyslogServer: string; Facil: Byte;
  Sever: TSyslogSeverity; const Content: string): Boolean;
```

TSyslogSeverity is the RFC enum: Emergency, Alert, Critical, Error, Warning, Notice, Info, Debug. Facility is the numeric code (e.g. 1 = user, 16 = local0).

```
uses slogsend;

begin
  // facility 16 (local0), severity Info, one line to the collector
  ToSysLog('192.168.1.10', 16, Info, 'backup completed');
end.
```

Driving the class directly lets you set the Tag/AppName and other header fields before calling DoIt:

```
uses slogsend;

var
  Log: TSyslogSend;
begin
  Log := TSyslogSend.Create;
  try
    Log.TargetHost := '192.168.1.10';
    Log.SysLogMessage.Facility   := 16;
    Log.SysLogMessage.Severity   := Warning;
    Log.SysLogMessage.Tag        := 'myapp';
    Log.SysLogMessage.LogMessage := 'disk 85% full';
    Log.DoIt;
  finally
    Log.Free;
  end;
end.
```

> **Note.** Classic syslog over UDP/514 is fire-and-forget and unencrypted: DoIt returning True means the packet was sent, not that anything received it. That is inherent to the protocol, not a limitation of the client.

Where to go next

Every client here rewards a glance at its unit source — they are small, and each convenience function (GetMailServers, SNMPGet, PingHost, ToSysLog) is itself a worked example of driving the underlying class. Once the request / response rhythm from the overview is in your hands, none of these hold surprises; they differ only in the verbs of their protocol.

MIME & Mail Messages

`TSMTPSend` gets a message to the server, but it does not care what the message *is* — `MailData` takes a `TStrings` holding a complete RFC-822 message and sends it verbatim. Producing that `TStrings` correctly — headers with non-ASCII subjects, a text body, an HTML alternative, file attachments, inline images — is a separate job, and it belongs to a separate pair of units:

- **`mimepart`** — `TMimePart`, one node of a MIME tree. Knows how to encode and decode a single part (its headers, its transfer encoding, its body) and holds a list of child parts.
- **`mimemess`** — `TMimeMess`, the whole message: a `TMessHeader` (the mail headers) plus a root `TMimePart` (the body tree), with convenience methods that build the common part shapes for you.
- **`mimeinln`** — the inline (RFC-2047) header encoding, so a `Subject:` or a display name can carry accented characters. `TMessHeader` calls into it for you; you rarely call it directly.

Together they are the "what the message is" half of the story; `TSMTPSend` (TSMTPSend — sending mail) is the "getting it there" half. The seam between them is one property: `TMimeMess.Lines`, a `TStringList`, is exactly the `TStrings` that `TSMTPSend.MailData` expects.

A message is a tree of parts

The core idea in `mimepart` is that a MIME message is a **tree**. Each `TMimePart` is a node; a node is either a leaf (text, HTML, a binary attachment) or a `multipart` container whose children are more parts. A simple "body plus attachment" message is a three-node tree:

```
multipart/mixed              <- root container
├── text/plain               <- the body
└── application/pdf           <- the attachment
```

`TMimePart` (in `mimepart`) carries the properties of one node. The ones you touch most:

```
TMimePart = class(TObject)
published
  property Primary: string;         // 'text', 'multipart', 'message', 'application'…
  property Secondary: string;       // 'plain', 'mixed', 'html', 'pdf'…
  property PrimaryCode: TMimePrimary;   // MP_TEXT, MP_MULTIPART, MP_MESSAGE, MP_BINARY
  property EncodingCode: TMimeEncoding; // ME_7BIT, ME_8BIT, ME_QUOTED_PRINTABLE, ME_BASE64, ME
  property CharsetCode: TMimeChar;  // charset for text parts
  property Disposition: string;     // 'inline' or 'attachment'
  property ContentID: string;       // the cid: for inline images
  property FileName: string;        // attachment file name
  property Boundary: string;        // multipart delimiter (multipart parts only)

  property Lines: TStringList;      // the raw, encoded part (headers + body)
  property Headers: TStringList;    // this part's header lines
```

```
  property PartBody: TStringList;   // the still-encoded body
  property DecodedLines: TMemoryStream; // the decoded body bytes
end;
```

Two `TStrings`/stream properties are the crux of encode-vs-decode: **`Lines`** is the wire form (encoded), and **`DecodedLines`** is the raw bytes. Encoding goes `DecodedLines -> Lines`; decoding goes `Lines -> DecodedLines`. You put content into `DecodedLines` and call `EncodePart`; you read decoded content out of `DecodedLines` after calling `DecodePart`.

The four primary types are a closed set:

```
TMimePrimary  = (MP_TEXT, MP_MULTIPART, MP_MESSAGE, MP_BINARY);
TMimeEncoding = (ME_7BIT, ME_8BIT, ME_QUOTED_PRINTABLE, ME_BASE64, ME_UU, ME_XX);
```

An unrecognised primary type decodes to `MP_BINARY`; an unrecognised transfer encoding decodes to `ME_7BIT`.

The message: TMimeMess

You rarely build the tree by hand. `TMimeMess` (in `mimemess`) owns the root part and a header object, and hands you `AddPart…` helpers that create a child node, set its type, load its content, and encode it in one call:

```
TMimeMess = class(TObject)
  function AddPart(const PartParent: TMimePart): TMimePart;
  function AddPartMultipart(const MultipartType: String; const PartParent: TMimePart): TMimePar
  function AddPartText(const Value: TStrings; const PartParent: TMimePart): TMimePart;
  function AddPartTextEx(const Value: TStrings; const PartParent: TMimePart;
    PartCharset: TMimeChar; Raw: Boolean; PartEncoding: TMimeEncoding): TMimePart;
  function AddPartHTML(const Value: TStrings; const PartParent: TMimePart): TMimePart;
  function AddPartTextFromFile(const FileName: String; const PartParent: TMimePart): TMimePart;
  function AddPartHTMLFromFile(const FileName: String; const PartParent: TMimePart): TMimePart;
  function AddPartBinary(const Stream: TStream; const FileName: string; const PartParent: TMime
  function AddPartBinaryFromFile(const FileName: string; const PartParent: TMimePart): TMimePar
  function AddPartHTMLBinary(const Stream: TStream; const FileName, Cid: string; const PartPare
  function AddPartHTMLBinaryFromFile(const FileName, Cid: string; const PartParent: TMimePart):
  function AddPartMess(const Value: TStrings; const PartParent: TMimePart): TMimePart;
  function AddPartMessFromFile(const FileName: string; const PartParent: TMimePart): TMimePart;

  procedure EncodeMessage; virtual;   // build MessagePart + Header into Lines
  procedure DecodeMessage; virtual;   // parse Lines into Header + MessagePart
published
  property MessagePart: TMimePart;    // the root of the part tree
  property Lines: TStringList;        // the raw RFC-822 message
  property Header: TMessHeader;       // From/To/Subject/…
end;
```

The `PartParent` argument is how you place a node in the tree. Pass `nil` and the part becomes the **root** (`MessagePart` itself); pass an existing multipart part and the new part is added as its child. So the pattern for a multipart message is: create a `multipart/mixed` root with `AddPartMultipart('mixed', nil)`, then add each leaf with that root as parent.

> **What each helper produces (read from the source).** `AddPartText` makes a `text/plain`, `inline`, quoted-printable part and picks an ideal charset for the content. `AddPartHTML` makes a `text/html`, `inline`, quoted-printable, UTF-8 part. `AddPartBinary`/`…FromFile` makes an `attachment`, base64 part and derives the MIME

type from the file extension. `AddPartHTMLBinary` makes an `inline` base64 part with a `Content-ID` (`Cid`) — the referent for a `<img src="cid:…">` inside an HTML part. `AddPartMess` embeds a whole `message/rfc822`. Each helper calls `EncodePart` + `EncodePartHeader` before returning, so the part is fully encoded the moment you get it back.

The headers: TMessHeader

`TMimeMess.Header` is a `TMessHeader` holding the parsed mail headers as typed fields, not raw strings:

```
TMessHeader = class(TObject)
published
  property From: string;
  property ToList: TStringList;       // one recipient per line
  property CCList: TStringList;       // one CC per line
  property Subject: string;
  property Organization: string;
  property ReplyTo: string;
  property MessageID: string;
  property Date: TDateTime;
  property XMailer: string;
  property Priority: TMessPriority;   // MP_unknown, MP_low, MP_normal, MP_high
  property CharsetCode: TMimeChar;    // charset for encoding the headers
  property CustomHeaders: TStringList; // everything not parsed into a field above
end;
```

Note the shape: `From`, `Subject`, `ReplyTo` are single strings, but `ToList` and `CCList` are `TStringLists` — you `.Add` one address per line rather than assigning a comma-joined string. On encode, `TMessHeader.EncodeHeaders` joins them with `,` and runs each through the inline encoder. It also fills in sensible defaults you did not set: a `Date:` of `Now` if you left it `0`, a `MIME-Version: 1.0`, and an `X-mailer:` identifying Synapse. Anything the parser does not recognise on decode lands in `CustomHeaders`, readable with `FindHeader` / `FindHeaderList`.

Encodings: transfer encoding vs. inline header encoding

MIME has two distinct encoding problems, and Synapse keeps them in two places:

1. **Transfer encoding of part bodies** — making arbitrary bytes safe for a 7-bit mail path. This is `EncodingCode` on a part: `ME_QUOTED_PRINTABLE` for mostly-ASCII text (the default for `AddPartText`/`AddPartHTML`), `ME_BASE64` for binary attachments (the default for `AddPartBinary`). The actual base64/quoted-printable routines live in `synacode` (see Rolling its own crypto & encoding) and are driven by `TMimePart.EncodePart` / `DecodePart`.
2. **Inline encoding of header text** — a `Subject:` or a display name cannot carry raw 8-bit bytes, so RFC-2047 wraps them as `=?charset?B?…?=` / `=?charset?Q?…?=` words. That is `mimeinln`:

   ```
   function InlineDecode(const Value: string; CP: TMimeChar): string;
   function InlineEncode(const Value: string; CP, MimeP: TMimeChar): string;
   function InlineCode(const Value: string): string;
   function InlineCodeEx(const Value: string; FromCP: TMimeChar): string;
   function InlineEmail(const Value: string): string;
   function InlineEmailEx(const Value: string; FromCP: TMimeChar): string;
   function NeedInline(const Value: AnsiString): boolean;
   ```

`TMessHeader` calls `InlineCodeEx` (for `Subject`/`Organization`) and `InlineEmailEx` (for addresses, which must keep the `<addr>` un-encoded) on your behalf, so a `Subject := 'Æblegrød'` just works. You reach for these directly only when hand-building headers outside `TMessHeader`.

Where to go next

- Building a message — assemble a multipart message with a text body and a file attachment, then feed `Lines` to `TSMTPSend.MailData`.
- Parsing a message — take a raw received message, decode it, walk the part tree, and pull out the text and the attachments.

Building a Message

The goal: a mail with a plain-text body **and** a file attachment, ready to hand to `TSMTPSend`. That is a two-leaf tree under a `multipart/mixed` root, and `TMimeMess` builds it in a handful of calls.

The shape you are building

```
multipart/mixed              <- AddPartMultipart('mixed', nil)  -> root
├── text/plain               <- AddPartText(Body, root)
└── application/pdf           <- AddPartBinaryFromFile('report.pdf', root)
```

The rule from the source: to add **more than one** subpart you must have a multipart parent, and the first argument to every `AddPart…` helper is that parent. Pass `nil` for the root; pass the multipart part for each leaf.

Worked example

```
program BuildMail;

uses
  SysUtils, Classes,
  mimemess, mimepart,
  smtpsend, ssl_openssl;   // ssl_openssl enables STARTTLS / SMTPS

var
  Mime: TMimeMess;
  Body: TStringList;
  Root: TMimePart;
  Smtp: TSMTPSend;
begin
  Mime := TMimeMess.Create;
  Body := TStringList.Create;
  try
    // --- Headers -----------------------------------------------------------
    // From / ReplyTo / Subject are single strings; ToList / CCList take one
    // address per line. Subject is inline-encoded automatically, so non-ASCII
    // is fine.
    Mime.Header.From := 'alice@example.com';
    Mime.Header.ToList.Add('bob@example.org');
    Mime.Header.CCList.Add('carol@example.org');
    Mime.Header.Subject := 'Quarterly report (Æblegrød included)';
    // Date, Message-ID, MIME-Version and X-mailer are filled in for you on
```

```
    // encode if you leave them unset.

    // --- Body tree ---------------------------------------------------------
    // A message with an attachment needs a multipart/mixed container as root.
    Root := Mime.AddPartMultipart('mixed', nil);

    // The text body: text/plain, inline, quoted-printable, ideal charset.
    Body.Add('Hi Bob,');
    Body.Add('');
    Body.Add('The quarterly report is attached as a PDF.');
    Body.Add('');
    Body.Add('-- Alice');
    Mime.AddPartText(Body, Root);

    // The attachment: MIME type derived from the extension, base64, marked as
    // an attachment with this file name.
    Mime.AddPartBinaryFromFile('report.pdf', Root);

    // --- Serialise ---------------------------------------------------------
    // EncodeMessage merges Header into the root part and composes the whole
    // tree into Mime.Lines.
    Mime.EncodeMessage;

    // Mime.Lines is now a complete RFC-822 message. Send it.
    Smtp := TSMTPSend.Create;
    try
      Smtp.TargetHost := 'smtp.example.com';
      Smtp.TargetPort := '587';
      Smtp.UserName   := 'alice@example.com';
      Smtp.Password   := 's3cret';
      Smtp.AutoTLS    := True;             // STARTTLS before AUTH

      if not Smtp.Login then
        raise Exception.Create('SMTP login failed: ' + Smtp.ResultString);

      if Smtp.MailFrom('alice@example.com', Length(Mime.Lines.Text)) and
         Smtp.MailTo('bob@example.org') and
         Smtp.MailTo('carol@example.org') and
         Smtp.MailData(Mime.Lines) then      // <-- the seam: Lines -> MailData
        Writeln('Sent: ', Smtp.ResultString)
      else
        Writeln('Send failed: ', Smtp.ResultString);

      Smtp.Logout;
    finally
      Smtp.Free;
    end;
  finally
    Body.Free;
    Mime.Free;
  end;
end.
```

What the calls actually did

- **`AddPartMultipart('mixed', nil)`** created the root part, set it to `multipart/mixed`, gave it a fresh `Boundary` (via `GenerateBoundary`), and encoded its header. Its return value is the parent you pass to the leaves.
- **`AddPartText(Body, Root)`** saved `Body` into the new part's `DecodedLines`, set `text/plain`, `inline`, quoted-printable, chose an ideal charset for the text, then called `EncodePart` + `EncodePartHeader`. The part comes back fully encoded.
- **`AddPartBinaryFromFile('report.pdf', Root)`** loaded the file into a memory stream, derived the MIME type from the extension (`application/PDF` from the built-in table; unknown extensions fall back to `application/octet-string`), set `Disposition := 'attachment'` and `FileName`, base64-encoded it, and built the header.
- **`EncodeMessage`** lifted the root part's `Content-*` headers up next to the `From/To/Subject/…` headers, composed every part into the wire form, and copied the result into `Mime.Lines`.

Variations

HTML with an inline image. Swap the text leaf for an HTML leaf and add the image as an inline (not attachment) part carrying a `Content-ID`:

```
Root := Mime.AddPartMultipart('related', nil);        // 'related' ties HTML + cid
Html := TStringList.Create;
Html.Add('<p>See the chart: <img src="cid:chart1"></p>');
Mime.AddPartHTML(Html, Root);
Mime.AddPartHTMLBinaryFromFile('chart.png', 'chart1', Root);  // Cid = 'chart1'
```

`AddPartHTMLBinary…` sets `Disposition := 'inline'` and `ContentID := Cid`; the HTML references it as `cid:chart1`. Use `multipart/related` (not `mixed`) so mail clients know the image belongs to the HTML body.

Content from files. `AddPartTextFromFile` and `AddPartHTMLFromFile` are the same as their in-memory versions but `LoadFromFile` the content for you.

Precise charset / encoding control. `AddPartText` picks the charset and forces quoted-printable. When you need to override either — a fixed charset, a `Raw` (no charset conversion) part, or base64 text — use `AddPartTextEx`:

```
Mime.AddPartTextEx(Body, Root, UTF_8, {Raw=}False, ME_BASE64);
```

Caveats

- **Build the tree before `EncodeMessage`.** `EncodeMessage` composes whatever is in `MessagePart` at call time; add all parts first, then encode once. Call it again only after changing the tree.
- **Envelope vs. header addresses are separate.** `MailFrom`/`MailTo` are the SMTP envelope; `Header.From`/`Header.ToList` are what the reader sees. They are usually the same, but `Bcc` works precisely by adding a `MailTo` with no matching header line — so do not expect a `Header.CCList` entry to reach a recipient unless you also `MailTo` them.
- **A single-part message needs no multipart.** If you only have a text body, skip `AddPartMultipart` and call `AddPartText(Body, nil)` to make the text the root directly. The multipart wrapper is only needed once you have two or more leaves.

Parsing a Message

Receiving is the mirror of building. You have the raw RFC-822 bytes (from `TPOP3Send`, `TIMAPSend`, an `.eml` file — anywhere), and you want the parsed headers, the readable text, and the attachments written to disk. `TMimeMess` decodes the message; walking the `TMimePart` tree pulls out the pieces.

The two-step decode

`TMimeMess.DecodeMessage` does two things, from the source:

1. `Header.DecodeHeaders(Lines)` — parses the mail headers into `From`, `ToList`, `Subject`, `Date`, ... (and everything else into `CustomHeaders`).
2. `MessagePart.DecomposeParts` — parses the body into the part tree.

Crucially, `DecomposeParts` **does not decode the part bodies** — it only splits the tree and reads each part's headers. Decoding a body (base64 → bytes, quoted-printable → text, charset conversion) is a separate `DecodePart` call per part, deliberately, so you don't pay to decode a 10 MB attachment you are going to skip. So the rhythm is: `DecodeMessage` once, then `DecodePart` on each leaf you actually want.

Walking the tree

Two ways to visit every node:

- **Manual recursion** with `GetSubPartCount` / `GetSubPart(i)` — full control, easy to read.
- **The `WalkPart` hook** — assign an `OnWalkPart` handler and Synapse calls it for the part and every descendant (it propagates the hook down the tree for you).

This example uses manual recursion.

Worked example

```
program ParseMail;

uses
  SysUtils, Classes,
  mimemess, mimepart;

// Recursively visit every part; decode text into Text, save attachments.
procedure HandlePart(Part: TMimePart; const Text: TStrings);
var
  i: Integer;
```

```
begin
  case Part.PrimaryCode of
    MP_MULTIPART:
      // A container — recurse into its children, nothing to decode here.
      for i := 0 to Part.GetSubPartCount - 1 do
        HandlePart(Part.GetSubPart(i), Text);

    MP_MESSAGE:
      // An embedded message/rfc822 — it too has subparts; recurse.
      for i := 0 to Part.GetSubPartCount - 1 do
        HandlePart(Part.GetSubPart(i), Text);

    MP_TEXT:
      begin
        Part.DecodePart;   // Lines -> DecodedLines (+ charset conversion)
        // Only fold plain text into the body; skip 'html' if you prefer.
        if LowerCase(Part.Secondary) = 'plain' then
        begin
          Part.DecodedLines.Position := 0;
          Text.LoadFromStream(Part.DecodedLines);
        end;
      end;

    MP_BINARY:
      begin
        // An attachment (or inline image). Decode and write it out.
        Part.DecodePart;
        if Part.FileName <> '' then
        begin
          Part.DecodedLines.Position := 0;
          Part.DecodedLines.SaveToFile('inbox_' + ExtractFileName(Part.FileName));
          Writeln('Saved attachment: ', Part.FileName,
                  ' (', Part.DecodedLines.Size, ' bytes)');
        end;
      end;
  end;
end;

var
  Mime: TMimeMess;
  Text: TStringList;
begin
  Mime := TMimeMess.Create;
  Text := TStringList.Create;
  try
    // Load the raw message into Lines. (Here from a file; in practice this is
    // whatever POP3/IMAP handed you.)
    Mime.Lines.LoadFromFile('received.eml');

    // Step 1: parse headers + split the part tree (bodies NOT yet decoded).
    Mime.DecodeMessage;

    Writeln('From:    ', Mime.Header.From);
    Writeln('Subject: ', Mime.Header.Subject);
```

```
    Writeln('Date:    ', DateTimeToStr(Mime.Header.Date));
    Writeln('To:      ', Mime.Header.ToList.CommaText);

    // Step 2: walk the tree, decoding parts on demand.
    HandlePart(Mime.MessagePart, Text);

    Writeln('--- body ---');
    Writeln(Text.Text);
  finally
    Text.Free;
    Mime.Free;
  end;
end.
```

Reading the results

- **Headers** are on `Mime.Header` as typed fields the moment `DecodeMessage` returns — `From`, `Subject`, `ToList`, `Date`, and so on. Anything the parser did not special-case (e.g. `Received:`, `X-Spam-Score:`) is in `CustomHeaders`; fetch it with `Header.FindHeader('X-Spam-Score')` or, for a header that repeats, `Header.FindHeaderList('Received', List)`.
- **The body** is the `MessagePart` tree. For a simple message the root *is* the text part (`PrimaryCode = MP_TEXT`, `GetSubPartCount = 0`); for a multipart message the root is `MP_MULTIPART` and the leaves are its children. The recursion above handles both without a special case.
- **Distinguishing body from attachment** is by `Disposition` and `FileName`: an attachment has `Disposition = 'attachment'` and a non-empty `FileName`; an inline image has `Disposition = 'inline'` and a `ContentID`. The example keys off `FileName` being set, which catches both cases where there is a file to write.

Using the WalkPart hook instead

If you prefer a flat callback to explicit recursion, assign `OnWalkPart` on the root and call `WalkPart` — Synapse invokes your handler for the root and every descendant:

```
type
  TCollector = class
    procedure OnPart(const Sender: TMimePart);
  end;

procedure TCollector.OnPart(const Sender: TMimePart);
begin
  if (Sender.PrimaryCode = MP_BINARY) and (Sender.FileName <> '') then
  begin
    Sender.DecodePart;
    Sender.DecodedLines.Position := 0;
    Sender.DecodedLines.SaveToFile(ExtractFileName(Sender.FileName));
  end;
end;

// ...
Coll := TCollector.Create;
Mime.MessagePart.OnWalkPart := Coll.OnPart;
Mime.MessagePart.WalkPart;    // fires OnPart for the root and all subparts
```

`WalkPart` only calls your hook if `OnWalkPart` is assigned, and it copies the hook onto each child before descending — so one assignment on the root covers the whole tree.

Caveats

- **`DecomposeParts` splits, `DecodePart` decodes.** Reading `DecodedLines` without first calling `DecodePart` on that part gives you nothing useful — the decoded stream is only populated by `DecodePart`. Always decode the leaf before you read its `DecodedLines`.
- **Reset the stream position.** `DecodedLines` is a `TMemoryStream`; set `Position := 0` before `LoadFromStream`/reading, or `SaveToFile` (which seeks itself) as shown.
- **Malformed real-world mail.** Broken senders (the source singles out some Outlook versions) omit charset labels; `DefaultCharset` on the part governs how such text is decoded — adjust it before `DecodePart` if you see mojibake.
- **Depth limiting.** `MaxSubLevel` on a part caps how deep `DecomposeParts` will recurse (default `-1` = unlimited). Set it to guard against pathological deeply-nested messages.
- **Binary receive path.** If the raw message came from `THTTPSend` (headers and an 8-bit body stream held separately, no transfer encoding applied), use `DecodeMessageBinary(AHeader, AData)` instead of `DecodeMessage` — it is the 8-bit-native equivalent.

TBlockSerial — the sockets model, applied to RS-232

Here is the quietly lovely thing about Synapse: when its authors needed a serial port library, they did not invent a new vocabulary. They took the *exact* mental model of `TBlockSocket` — blocking calls, millisecond timeouts, a `LastError` you check after each step, `RecvString`/`RecvTerminated` for line protocols — and pointed it at a UART instead of a TCP stream. The unit's own header says so:

> This class provides numerous methods with same name and functionality as methods of the Ararat Synapse TCP/IP library.

If you have read the `TBlockSocket` chapter, you already know most of `TBlockSerial`. This chapter is about the parts that are genuinely different — opening a port instead of connecting to a host, configuring baud/bits/parity, and driving the modem control lines — and about how little else changes.

The unit is `synaser`. The one class you use is `TBlockSerial`.

The lifecycle

A serial port has the same explicit life as a socket, with `Connect` opening a device and `CloseSocket` closing it:

`Create` ⟶ `Connect(device)` ⟶ `Config(...)` ⟶ `Send/Recv …` ⟶ `CloseSocket` ⟶ `Free`

- **`constructor Create`** — makes the object. It does *not* open a port yet.
- **`procedure Connect(comport: string)`** — opens the named device. The parameters are whatever the OS had configured for that port; call `Config` afterwards if you need specific settings.
- **`procedure CloseSocket`** — closes the handle. The destructor calls it, so `Free` is enough, but closing explicitly is good manners (and releases any lock file — see privileges below).

Note the name: it really is `CloseSocket`, not `ClosePort`. That is the design consistency showing through — the method names were kept identical to the socket library on purpose.

```
uses synaser;

var ser: TBlockSerial;
begin
  ser := TBlockSerial.Create;
  try
    ser.Connect('/dev/ttyUSB0');
    if ser.LastError <> 0 then
    begin
```

```
      Writeln('open: ', ser.LastErrorDesc);
      Exit;
    end;
    ser.Config(9600, 8, 'N', SB1, False, False);   // 9600 8N1, no flow control
    { ...talk to the device... }
  finally
    ser.CloseSocket;
    ser.Free;
  end;
end;
```

Naming the device — ‘COM1’ vs ‘/dev/ttyS0’

`Connect` takes a device *name*, and Synapse is bi-dialect about it. From the source documentation on `Connect`:

> Comport can be used in Windows style (COM2), or in Linux style (/dev/ttyS1). When you use windows style in Linux, then it will be converted to Linux name. And vice versa!

So `Connect('COM2')` on Linux is translated to `/dev/ttyS1` (the conversion is zero-based on the Unix side: `COM1` → `/dev/ttyS0`, `COM2` → `/dev/ttyS1`), and a `/dev/ttyS1` on Windows maps back to a `COM` number. The translation lives in the `GetComNr` helper and only recognises the `COM`*n* and `/dev/ttyS`*n* forms.

That last point matters in practice: **anything that is not a `COMn` or `/dev/ttySn` name is passed through unchanged.** USB-serial adapters (`/dev/ttyUSB0`), ACM/CDC modems (`/dev/ttyACM0`), and ARM on-chip UARTs (`/dev/ttyAMA0`) are *not* rewritten — you give the literal path and it is opened as-is. In modern life that is most of what you connect to, so the usual advice is simply: **on Unix, name the real device path; on Windows, use `COMn`.** The `Device` property tells you the actual OS name that ended up being opened.

To discover what is present, the unit exports a free function:

```
function GetSerialPortNames: string;   // comma-separated list of port names
```

On Unix it globs `/dev/ttyS*`, `/dev/ttyUSB*`, `/dev/ttyAM*` and `/dev/ttyACM*`; on Windows it reads them from the registry’s `SERIALCOMM` map. Handy for populating a “choose a port” dropdown.

A privilege caveat (Unix)

Opening a serial device on Linux/Unix normally requires membership in the `dialout` group (or equivalent), because `/dev/ttyS*` and `/dev/ttyUSB*` are owned `root:dialout`. If `Connect` comes back with `LastError` set to a permission error, that is almost always the cause — add the user to `dialout` rather than running as root. Synapse can also create a lock file under `/var/lock` to cooperate with other programs using the port; this is governed by the `LinuxLock` property (default `True`), and the `/var/lock` directory must be writable for it. On Windows there is no such group; access is governed by whether another process already holds the port open.

Config — baud, bits, parity, stop bits, flow

One call sets every line parameter. You must be connected first:

```
procedure Config(baud, bits: integer; parity: char; stop: integer;
  softflow, hardflow: boolean); virtual;
```

- **baud** — bits per second, an *actual number* (not an enum): `9600`, `19200`, `115200`, `460800`, up to `4000000` on capable hardware. Internally Synapse maps it to the nearest supported `Bxxx` termios rate on Unix.
- **bits** — data bits per character, typically 8 (or 7, 6, 5).
- **parity** — a single character: **`'N'`** none, **`'O'`** odd, **`'E'`** even, **`'M'`** mark, **`'S'`** space. (Case-insensitive.)
- **stop** — stop bits, using the named constants **`SB1`** (1 stop bit, value 0), **`SB1andHalf`** (1.5, value 1), or **`SB2`** (2, value 2).
- **softflow** — enable XON/XOFF software handshake.
- **hardflow** — enable RTS/CTS hardware handshake.

So the ubiquitous "9600 8N1, no flow control" is:

```
ser.Config(9600, 8, 'N', SB1, False, False);
```

and "115200 8N1 with hardware handshake" is:

```
ser.Config(115200, 8, 'N', SB1, False, True);
```

`Config` may be called again at any time to reconfigure a live port on the fly. Under the hood it fills a `TDCB` (Windows' Device Control Block, *simulated* on Unix so the same code path serves both) and pushes it with `SetCommState`; the inverse `GetCommState` reads the current settings back. You rarely touch those two directly.

Sending — identical to the socket API

Every send method has the same name and shape as on `TBlockSocket`:

```
function  SendBuffer(buffer: pointer; length: integer): integer; virtual;
procedure SendByte(data: byte); virtual;
procedure SendString(data: AnsiString); virtual;   // bytes as-is — NO terminator added
procedure SendInteger(Data: integer); virtual;      // four bytes
procedure SendBlock(const Data: AnsiString); virtual;   // 4-byte length prefix, then data
procedure SendStreamRaw(const Stream: TStream); virtual;
procedure SendStream(const Stream: TStream); virtual;
```

The same honest caveat applies as with the socket: **`SendString` appends no terminator.** The source is explicit — "No terminator is appended by this method. If you need to send a string with CR/LF terminator, you must append the CR/LF characters." Because it adds nothing, it doubles as your binary-send path.

```
ser.SendString('AT' + CRLF);          // you supply the CRLF
ser.SendByte($02);                    // a single control byte
```

`CR`, `LF`, and `CRLF` are declared right in `synaser`, so you do not need to pull them from elsewhere.

Receiving — the same three you already reach for

```
function RecvBuffer(buffer: pointer; length: integer): integer; virtual;
function RecvBufferEx(buffer: pointer; length, timeout: integer): integer; virtual;
function RecvBufferStr(Length, Timeout: Integer): AnsiString; virtual;
function RecvPacket(Timeout: Integer): AnsiString; virtual;
function RecvByte(timeout: integer): byte; virtual;
function RecvTerminated(Timeout: Integer; const Terminator: AnsiString): AnsiString; virtual;
```

```
function RecvString(Timeout: Integer): AnsiString; virtual;
function RecvInteger(Timeout: Integer): Integer; virtual;
function RecvBlock(Timeout: Integer): AnsiString; virtual;
```

Every one that can wait takes a **millisecond `Timeout`** and sets `LastError` to **`ErrTimeout`** if it lapses — exactly the blocking-with-timeouts contract from the sockets chapter, and exactly what makes serial code safe to write in a straight line. The three you will use most:

- **`RecvString(Timeout)`** — reads up to a CR/LF terminator and returns the line *without* it. This is the workhorse for line-oriented instruments and modems. It is `RecvTerminated(Timeout, CRLF)` under the hood, and it keeps its own internal buffer, so do not interleave it with raw `RecvBuffer` calls on the same port (the source warns this can lose data).
- **`RecvTerminated(Timeout, Terminator)`** — the same idea for devices that end each reply with something other than CR/LF: an ETX (`#3`), a `'>'` prompt, a `'#'`, whatever your protocol uses. Returns the payload without the terminator.
- **`RecvPacket(Timeout)`** — "give me whatever bytes are waiting right now." Ideal when the reply has no fixed framing and you just want to drain the port.

If a device uses lone-CR or lone-LF line ends rather than strict CR/LF, set `ConvertLineEnd := True` and `RecvString` will accept any of them. `MaxLineLength` caps how much `RecvString`/`RecvTerminated` will buffer before erroring (default `0` = unlimited) — worth setting when a misbehaving device might never send a terminator.

Readiness and buffers

Also mirrored from the socket class — you seldom need these because the `Recv*` timeouts already poll internally, but they are here:

```
function  CanRead(Timeout: integer): boolean; virtual;    // data available to read
function  CanWrite(Timeout: integer): boolean; virtual;   // room to write
function  CanReadEx(Timeout: integer): boolean; virtual;  // also true if LineBuffer has data
function  WaitingData: integer; virtual;                  // bytes ready to read now
function  WaitingDataEx: integer; virtual;                // ... including LineBuffer
function  SendingData: integer; virtual;                  // bytes still queued to send
procedure Flush; virtual;                                 // wait until the output buffer drain
procedure Purge; virtual;                                 // discard both buffers immediately
```

`Timeout = 0` on the `Can*` calls means "test and return now"; `-1` means "wait, possibly forever." **`Purge`** is the one to remember for serial work: after a timeout or a protocol error, `Purge` throws away any half-received garbage in both directions so your next exchange starts clean. **`Flush`** blocks until the transmit buffer has been handed to the hardware.

The `LastError` model — reports, not exceptions

Identical philosophy to the sockets. After any operation:

```
property LastError: integer read FLastError;        // 0 = success
property LastErrorDesc: string read FLastErrorDesc; // human-readable
```

On success `LastError` is `0`. On failure it is either an OS error code or one of synaser's own constants, which are worth recognising:

Constant	Meaning
`ErrTimeout` (9997)	A read/write timed out
`ErrPortNotOpen` (9994)	Operation attempted with no open port
`ErrAccessDenied` (9990)	Could not open the device (permissions / in use)
`ErrAlreadyInUse` (9992)	Port is locked by another process
`ErrAlreadyOwned` (9991)	This process already owns the lock
`ErrWrongParameter` (9993)	Bad `Config` parameter
`ErrFrame` (9999)	Framing error on the line
`ErrOverrun` / `ErrRxOver` (10000/10001)	Receiver overrun
`ErrRxParity` (10002)	Parity error

As with the sockets, if you prefer exceptions set `RaiseExcept := True` and a failing call raises `ESynaSerError` (carrying `ErrorCode`); the default `False` keeps the linear check-after-each-step style.

Modem control lines — RTS, DTR, and the status inputs

This is the part that has no socket equivalent, and Synapse exposes the RS-232 control lines as plain properties. The two *outputs* you drive are **write-only**:

```
property RTS: Boolean write SetRTSF;   // Request To Send  (output)
property DTR: Boolean write SetDTRF;   // Data Terminal Ready (output)
```

and the *inputs* you read are read-only:

```
property CTS: Boolean read GetCTS;         // Clear To Send
property DSR: Boolean read GetDSR;         // Data Set Ready
property Carrier: Boolean read GetCarrier; // DCD — carrier detect
property Ring: Boolean read GetRing;       // RI — ring indicator
```

By default, a successful `Connect` **raises DTR** (and RTS too, unless you asked for hardware handshake). If you need the lines left exactly as found — because you are bit-banging DTR/RTS yourself, e.g. to control a device's reset or power line — set **`ManualSignals := True`** *before* connecting. Then `Connect` and `Config` never touch RTS/DTR, and their values are entirely yours:

```
ser.ManualSignals := True;
ser.Connect('/dev/ttyUSB0');
ser.DTR := False;   // hold the target in reset
Sleep(50);
ser.DTR := True;    // release
```

Related odds and ends: `EnableRTSToggle(True)` puts the driver in half-duplex RTS-toggle mode for RS-485 transceivers; `SetBreak(Duration)` sends a line break for `Duration` ms; `ModemStatus` returns the raw status word the input properties decode. There is also a small AT-command helper layer (`ATCommand`, `ATConnect`, `ATResult`) for talking to Hayes modems, which is where this library was born.

A worked example — 9600 8N1, command and terminated reply

Putting it together: open a USB-serial device, configure it, send a command, and read one CR/LF-terminated line back — the whole exchange in linear, timeout-safe code that reads like the socket examples because it *is* the same shape.

```
uses synaser;

function QueryInstrument(const Device, Cmd: string): string;
var
  ser: TBlockSerial;
begin
  Result := '';
  ser := TBlockSerial.Create;
  try
    // Open the port. Unix: a literal /dev path; Windows: 'COM3'.
    ser.Connect(Device);
    if ser.LastError <> 0 then
    begin
      Writeln('open ', Device, ': ', ser.LastErrorDesc);
      Exit;
    end;

    // 9600 baud, 8 data bits, no parity, 1 stop bit, no flow control.
    ser.Config(9600, 8, 'N', SB1, False, False);
    if ser.LastError <> 0 then
    begin
      Writeln('config: ', ser.LastErrorDesc);
      Exit;
    end;

    // Some instruments need a moment after the line settles.
    ser.Purge;                          // start from a clean slate

    // Send a command. We supply the terminator ourselves, as always.
    ser.SendString(Cmd + CRLF);

    // Read one CR/LF-terminated reply, waiting up to 2 seconds.
    Result := ser.RecvString(2000);
    if ser.LastError = ErrTimeout then
      Writeln('no reply within 2 s')
    else if ser.LastError <> 0 then
      Writeln('read: ', ser.LastErrorDesc);
  finally
    ser.CloseSocket;
    ser.Free;
  end;
end;

// usage:
//   Writeln(QueryInstrument('/dev/ttyUSB0', '*IDN?'));   // Unix
//   Writeln(QueryInstrument('COM3', '*IDN?'));           // Windows
```

If your device does not frame replies with CR/LF, swap the one line:

```
Result := ser.RecvTerminated(2000, '>');   // read up to a '>' prompt
```

That is the entire story. The port-opening and line-configuration parts are new; everything after `Config` — `SendString`, `RecvString`/`RecvTerminated`, the `Timeout` argument, the `LastError` check, `Purge` — is the socket vocabulary you already speak, now driving a UART. Synapse chose one I/O model and applied it everywhere, and *that* consistency is the feature.

Encoding & Crypto

Synapse carries its own digests, MACs, ciphers, transfer encodings, and charset tables — no external dependency for any of them. The architecture chapter Rolling Its Own Crypto & Encoding explains *why* that choice was made and where it stops (TLS is delegated to a plugin, not reimplemented). This section is the practical companion: **which function to call, what it returns, and the honest caveats.**

The three units

- **`synacode`** — digests (`MD5`, `MD4`, `SHA1`), keyed MACs (`HMAC_MD5`, `HMAC_SHA1`), and the MIME transfer encodings: Base64, quoted-printable, URL triplet encoding, UU/XX/yEnc, and CRC-16/CRC-32. Everything here operates on `AnsiString` and returns `AnsiString` (raw bytes, not text — see below).
- **`synacrypt`** — symmetric block ciphers (DES, 3DES, AES) as a small class hierarchy rooted at `TSynaBlockCipher`, with ECB/CBC/CFB/OFB/CTR modes.
- **`synachar`** — charset detection and conversion between `TMimeChar` values (ISO-8859-*, the CP125x Windows codepages, UTF-8/16/32, and dozens more), optionally backed by `iconv` on non-Windows platforms via `synaicnv`.

You usually don't call these directly

The protocol clients call them for you. A few real examples from the upstream source:

- `TPOP3Send.AuthAPOP` computes `StrToHex(MD5(FTimeStamp + FPassWord))` for you (`pop3send.pas`).
- `TSMTPSend` does CRAM-MD5 as `EncodeBase64(HMAC_MD5(challenge, FPassword))` and AUTH LOGIN as `EncodeBase64(FUsername)` (`smtpsend.pas`).
- `TMimePart` encodes and decodes bodies with `EncodeBase64` / `EncodeQuotedPrintable` based on the part's declared transfer encoding (`mimepart.pas`).
- `THTTPSend` runs `DecodeURL` over the credentials in a `user:pass@host` URL (`httpsend.pas`).
- `mimemess` / `mimeinln` reach into `synachar` to encode non-ASCII mail headers correctly.

So reach into these units directly when you are **speaking a protocol Synapse doesn't wrap**, decoding a blob by hand, or converting a charset the higher layers didn't handle for you. The rest of the time, let the client do it.

The recipes

- Hashing & HMAC — `MD5`/`SHA1`/`HMAC_MD5`/`HMAC_SHA1`, why the output is raw bytes, and how to hex it.

- Transfer encodings — Base64, quoted-printable, URL encoding, and the exact character sets each one escapes.
- Charsets — `TMimeChar`, `GetCPFromID`, `CharsetConversion`, and how non-ASCII mail/HTTP text survives a round trip.

One caveat up front

`MD5`, `MD4`, `SHA-1`, and single-DES live here because the *protocols* still specify them — APOP, CRAM-MD5, legacy SASL, and older cipher suites. They are **not** security primitives for new designs. Use them to speak the protocol that demands them; reach for the SSL plugin seam when you need real transport strength. This caveat is repeated on each recipe because it matters at each call site.

Hashing & HMAC

All the digest and MAC functions live in `synacode` and share one signature shape: `AnsiString` in, `AnsiString` out.

```
function MD5(const Value: AnsiString): AnsiString;
function MD4(const Value: AnsiString): AnsiString;
function SHA1(const Value: AnsiString): AnsiString;
function HMAC_MD5(Text, Key: AnsiString): AnsiString;
function HMAC_SHA1(Text, Key: AnsiString): AnsiString;
function MD5LongHash(const Value: AnsiString; Len: integer): AnsiString;
function SHA1LongHash(const Value: AnsiString; Len: integer): AnsiString;
```

(All verified in `synacode.pas` interface, lines 216–236.)

The output is raw bytes, not text

This is the one thing to internalize. `MD5('abc')` does **not** return the 32-character hex string you see in most tools — it returns the **16 raw digest bytes** packed into an `AnsiString`. `SHA1` returns 20 raw bytes, `HMAC_MD5` 16, `HMAC_SHA1` 20. Printing that `AnsiString` gives you binary garbage.

That is deliberate: for most protocol work you want the raw bytes so you can Base64-them or feed them onward. When you want the familiar hex, convert explicitly with `StrToHex` from `synautil`:

```
function StrToHex(const Value: Ansistring): string;   // synautil.pas
```

It walks the bytes as `IntToHex(Byte(...), 2)` and lowercases the result, so you get the conventional lowercase hex digest.

```
uses
  synacode, synautil;

var
  raw, hex: AnsiString;
begin
  raw := MD5('The quick brown fox jumps over the lazy dog');
  hex := StrToHex(raw);
  // hex = '9e107d9d372bb6826bd81d3542a419d6'
  WriteLn(hex);
end;
```

The same pattern gives you a hex SHA-1:

```
WriteLn(StrToHex(SHA1('abc')));
// a9993e364706816aba3e25717850c26c9cd0d89d
```

HMAC — keyed message authentication

`HMAC_MD5` and `HMAC_SHA1` take the message as `Text` and the secret as `Key` (note: both are plain value parameters, in that order). They too return raw MAC bytes.

```
uses
  synacode, synautil;

var
  mac: AnsiString;
begin
  mac := HMAC_MD5('message-to-authenticate', 'shared-secret-key');
  WriteLn(StrToHex(mac));   // 32 hex chars
end;
```

Real protocol usage — you often don't call these yourself

The mail clients already do the right thing. Two examples straight from upstream:

APOP (`pop3send.pas`, `TPOP3Send.AuthAPOP`) — the server's timestamp is concatenated with the password, MD5'd, and hex-encoded:

```
s := StrToHex(MD5(FTimeStamp + FPassWord));
Result := CustomCommand('APOP ' + FUserName + ' ' + s, False);
```

CRAM-MD5 (`smtpsend.pas`) — the Base64 challenge is decoded, HMAC-MD5'd with the password as key, and the raw MAC is re-encoded to Base64:

```
s := DecodeBase64(s);              // server challenge
s := HMAC_MD5(s, FPassword);       // raw 16-byte MAC
FSock.SendString(EncodeBase64(FUsername + ' ' + StrToHex(s)) + CRLF);
```

Notice the two different tail conversions: APOP wants **hex**, CRAM-MD5's MAC is carried as **hex inside a Base64 blob**. The raw-bytes return value is what lets each protocol format it its own way — which is exactly why the functions don't hex for you.

Long-hash variants

`MD5LongHash` and `SHA1LongHash` take a `Len` and produce a digest stream of that length by iterating the hash (a PRF-style key stretch used by a few protocols). They are niche; reach for them only when a spec names them.

The honest caveat

MD5, MD4, and SHA-1 are here for protocol compatibility, not security. APOP, CRAM-MD5, and legacy SASL still specify them, so Synapse implements them to speak those protocols. All three are broken for collision resistance (MD5 and SHA-1 demonstrably so), and HMAC-MD5 / HMAC-SHA1, while not collision-dependent, are not what you would pick for a new authentication scheme.

Do **not** use these as your application's password hashing, integrity, or signing primitives. For those, use a modern KDF/AEAD outside Synapse, and for transport strength use the SSL plugin seam. Everything on this page exists to let old servers authenticate you, nothing more. See the architecture chapter Rolling Its Own Crypto & Encoding for the reasoning behind that scope.

Transfer Encodings

Base64, quoted-printable, and URL encoding all live in `synacode`, and all share the `AnsiString → AnsiString` shape. These are the encodings the mail and HTTP clients lean on constantly, and they are the ones you are most likely to call by hand when decoding a blob yourself.

Base64

```
function EncodeBase64(const Value: AnsiString): AnsiString;
function DecodeBase64(const Value: AnsiString): AnsiString;
```

(`synacode.pas`, interface lines 179–182.) Straightforward and byte-safe — encode any bytes, decode back to the exact same bytes:

```
uses
  synacode;

var
  enc, dec: AnsiString;
begin
  enc := EncodeBase64('Aladdin:open sesame');
  // enc = 'QWxhZGRpbjpvcGVuIHNlc2FtZQ=='
  dec := DecodeBase64(enc);
  // dec = 'Aladdin:open sesame'
end;
```

Internally `EncodeBase64` is `Encode3to4(Value, TableBase64)` and `DecodeBase64` is `Decode4to3Ex(Value, ReTableBase64)` — the standard RFC alphabet `A–Z a–z 0–9 + /` with `=` padding. `DecodeBase64` is tolerant of characters outside the alphabet (line breaks in wrapped Base64 are skipped), so you can feed it MIME-wrapped input directly.

There is also a URL-safe-ish variant pair, `EncodeBase64mod` / `DecodeBase64mod`, which uses `+` and `,` instead of `+` and `/` (the "modified" IMAP-style alphabet). Use it only when a spec calls for it.

This is the workhorse of SASL and MIME: `TSMTPSend` sends `EncodeBase64(FUsername)` for AUTH LOGIN, and `TMimePart` Base64-encodes binary attachment bodies.

Quoted-printable

```
function EncodeQuotedPrintable(const Value: AnsiString): AnsiString;
function DecodeQuotedPrintable(const Value: AnsiString): AnsiString;
function EncodeSafeQuotedPrintable(const Value: AnsiString): AnsiString;
```

(synacode.pas, interface lines 137, 149, 153.) Quoted-printable leaves ASCII text mostly readable and escapes only the bytes that need it, as =XX triplets:

```
uses
  synacode;

var
  s: AnsiString;
begin
  s := EncodeQuotedPrintable('Caf'#$E9' — 5'#$80);
  // non-ASCII bytes become =E9, =80, etc.; '=' itself becomes =3D
  WriteLn(s);
  WriteLn(DecodeQuotedPrintable(s));   // round-trips back to the bytes
end;
```

The two encoders differ in **which characters they escape**:

- `EncodeQuotedPrintable` escapes = plus all non-ASCII bytes (`#0..#31, #127..#255`). This is the normal body encoding.
- `EncodeSafeQuotedPrintable` escapes a wider `SpecialChar` set as well (`( ) [ ] < > : ; , @ / ? \ " _` and =), producing text safe to drop into places where those punctuation characters are structural — e.g. RFC-2047 encoded-words in mail headers.

`DecodeQuotedPrintable` is just `DecodeTriplet(Value, '=')` — it reverses either encoder. `TMimePart` calls these for text parts declared `quoted-printable`.

URL encoding

```
function EncodeURL(const Value: AnsiString): AnsiString;
function EncodeURLElement(const Value: AnsiString): AnsiString;
function DecodeURL(const Value: AnsiString): AnsiString;
```

(synacode.pas, interface lines 162, 158, 140. Exact names confirmed — note `EncodeURLElement`, not `EncodeURIComponent` or similar.) Same triplet mechanism, %XX delimiter. The two encoders differ, and the difference matters:

- **`EncodeURL`** escapes only `URLSpecialChar` — control bytes, high bytes, and the characters that are unsafe *anywhere* in a URL (`< > " % { } | \ ^ [ ]` backtick, space, `#$7F..#$FF`). It **leaves the URL structure alone**: `/`, `?`, `:`, `@`, &, =, `+`, `;`, # all pass through. Use it to clean up a whole URL without breaking it.
- **`EncodeURLElement`** escapes `URLSpecialChar` **plus** `URLFullSpecialChar` (`; / ? : @ = & # +`) — i.e. the reserved delimiters too. Use it to encode a single component (a query-string value, a path segment) that must not be interpreted as structure.

```
uses
  synacode;

begin
  WriteLn(EncodeURL('http://h/a b?x=1'));
  // EncodeURL keeps ':/?=' → http://h/a%20b?x=1

  WriteLn(EncodeURLElement('a b?x=1'));
  // EncodeURLElement escapes the reserved chars too → a%20b%3Fx%3D1
end;
```

`httpsend.pas` documents that the value put into a request field "must be encoded by the `EncodeURLElement` function," and it runs `DecodeURL` over the userinfo it parses out of a

`user:pass@host` URL. `DecodeURL` reverses both encoders (it just undoes every `%XX`).

No security caveat here — but a correctness one

These are encodings, not ciphers: they provide **zero** confidentiality or integrity. Base64 is not encryption. The only pitfall is **round-tripping**: pick the right encoder for the context (`EncodeURL` for a whole URL vs `EncodeURLElement` for one component; plain vs safe quoted-printable), or you will either over-escape and corrupt structure, or under-escape and break the message. For where these sit in the larger design, see Rolling Its Own Crypto & Encoding.

Charsets

`synachar` is Synapse's charset layer: it knows a large set of encodings, can guess which one a byte string is, and can convert between any two of them. This is what keeps an accented `Subject:` line or a Latin-1 HTTP body from turning into mojibake. The mail units (`mimemess`, `mimeinln`, `mimepart`) call into it for you; you touch it directly when you are decoding non-ASCII text the higher layers handed you raw.

Charsets are a `TMimeChar` enum, not numeric IDs

The set of supported charsets is the enumerated type `TMimeChar` (`synachar.pas`, line 101). Each member *is* the charset identifier — there is no separate `idNN` constant table; you name charsets by these enum values:

```
type
  TMimeChar = (ISO_8859_1, ISO_8859_2, ISO_8859_3, ISO_8859_4, ISO_8859_5,
    ISO_8859_6, ISO_8859_7, ISO_8859_8, ISO_8859_9, ISO_8859_10, ISO_8859_13,
    ISO_8859_14, ISO_8859_15, CP1250, CP1251, CP1252, CP1253, CP1254, CP1255,
    CP1256, CP1257, CP1258, KOI8_R, CP895, CP852, UCS_2, UCS_4, UTF_8, UTF_7,
    UTF_7mod, UCS_2LE, UCS_4LE,
    // the following are supported through iconv only:
    UTF_16, UTF_16LE, UTF_32, UTF_32LE, C99, JAVA, ISO_8859_16, KOI8_U, ...);
```

(The list continues with the Mac charsets, the CJK encodings — `EUC_JP`, `SHIFT_JIS`, `GB2312`, `BIG5`, `EUC_KR` — and more. See the full declaration in `synachar.pas`.)

Two important companion sets in the same unit:

- `IconvOnlyChars` — charsets that only work when `iconv` is present (via `synaicnv`; non-Windows). On a build without iconv these are unavailable.
- `NoIconvChars` — `[CP895, UTF_7mod]`, the two Synapse handles with its own internal routines regardless of iconv.

Name ↔ enum: `GetCPFromID` / `GetIDFromCP`

Wire formats carry charset *names* (`"utf-8"`, `"iso-8859-1"`, `"windows-1252"`), so you convert between the string label and the enum:

```
function GetCPFromID(Value: AnsiString): TMimeChar;   // "iso-8859-2" -> ISO_8859_2
function GetIDFromCP(Value: TMimeChar): AnsiString;    // ISO_8859_2  -> "ISO-8859-2"
```

(`synachar.pas`, interface lines 213, 216.) `GetCPFromID` is generous: it uppercases the input and matches against a large alias table (so `"LATIN1"`, `"CP819"`, `"ISO8859-1"` all resolve to `ISO_8859_1`), with special-cased names for `CP895`/Kamenicky and modified UTF-7. Feed it the charset token straight out of a MIME `Content-Type` or HTTP `charset=` parameter.

```
uses
  synachar;

var
  cs: TMimeChar;
begin
  cs := GetCPFromID('windows-1252');   // -> CP1252
  WriteLn(GetIDFromCP(cs));            // canonical name back
end;
```

Converting: `CharsetConversion`

The core conversion is:

```
function CharsetConversion(const Value: AnsiString;
  CharFrom: TMimeChar; CharTo: TMimeChar): AnsiString;
```

(`synachar.pas`, interface line 191.) Give it the bytes, the charset they are *in*, and the charset you want them *in*:

```
uses
  synachar;

var
  latin1, utf8: AnsiString;
begin
  latin1 := 'Caf'#$E9;                              // 'Café' in ISO-8859-1
  utf8 := CharsetConversion(latin1, ISO_8859_1, UTF_8);
  // utf8 now holds the UTF-8 bytes 'Caf' + C3 A9
end;
```

If you have a charset *name* rather than an enum, resolve it first:

```
from := GetCPFromID(headerCharsetName);
utf8 := CharsetConversion(rawBody, from, UTF_8);
```

The richer variants

```
function CharsetConversionEx(const Value: AnsiString; CharFrom: TMimeChar;
  CharTo: TMimeChar; const TransformTable: array of Word): AnsiString;

function CharsetConversionTrans(Value: AnsiString; CharFrom: TMimeChar;
  CharTo: TMimeChar; const TransformTable: array of Word;
  Translit: Boolean): AnsiString;
```

(interface lines 196, 202.) `CharsetConversionEx` applies an extra character replacement table during conversion — pass `Replace_None` for no substitution, or `Replace_Czech` (both declared in `synachar`) to strip Czech diacritics. `CharsetConversionTrans` adds a `Translit` flag: when `False`, unconvertible characters are *not* transliterated to approximations. Plain `CharsetConversion` is `CharsetConversionEx` with `Replace_None` and transliteration on.

Detection helpers

```
function NeedCharsetConversion(const Value: AnsiString): Boolean;
function GetCurCP: TMimeChar;
```

```
function GetCurOEMCP: TMimeChar;
function IdealCharsetCoding(const Value: AnsiString; CharFrom: TMimeChar;
  CharTo: TMimeSetChar): TMimeChar;
```

(interface lines 219, 206, 210, 223.)

- `NeedCharsetConversion` returns `True` only when the string has non-7-bit-ASCII bytes — a cheap gate so you skip conversion for pure-ASCII text.
- `GetCurCP` / `GetCurOEMCP` report the operating system's current charset (and the OEM/DOS-box charset on Windows), useful as a `CharFrom` when reading platform-local text.
- `IdealCharsetCoding` picks, from a candidate `TMimeSetChar`, the target charset that encodes `Value` with the fewest unconvertible characters — this is how the mail encoder chooses the smallest adequate charset for an outgoing header.

```
uses
  synachar;

begin
  if NeedCharsetConversion(body) then
    body := CharsetConversion(body, GetCPFromID(declaredCharset), UTF_8);
  // pure-ASCII bodies pass through untouched
end;
```

iconv: the reach beyond the built-in tables

Many of the `TMimeChar` members (everything in `IconvOnlyChars` — the CJK encodings, UTF-16/32, the Mac charsets) are handled by delegating to the system `iconv` through `synaicnv`. On a platform without iconv, or with the global `DisableIconv` flag set, those charsets are unavailable and conversions involving them will fail; the built-in ISO-8859-*, CP125x, KOI8, UTF-7/8, and UCS conversions always work. If you support arbitrary inbound charsets, test on your target platform with iconv actually present.

No security dimension — just correctness

Charset conversion is neither encryption nor sanitization: it changes byte *representation*, not meaning, and provides no safety guarantees. The failure mode is silent corruption — pass the wrong `CharFrom`, or run the bytes through conversion twice, and you get mojibake or lost characters. Always convert from the charset the data actually declares (from the MIME/HTTP header via `GetCPFromID`), convert once, and prefer `UTF_8` as your internal target. For where this fits the overall design, see Rolling Its Own Crypto & Encoding.

The Utility Units

Under every Synapse protocol class sits a thin layer of plain functions — no objects, no state, just helpers that the protocol code (and you) call directly. They are the parts of Synapse you end up reaching for even when you never open a socket: split a header line, format an RFC-822 date, validate an IP address, encode an ASN.1 packet, ask the OS which DNS server it uses.

This chapter covers the four units that make up that layer:

Unit	What it is for
`synautil.pas`	The general toolbox: string splitting, date/time, hex/binary, headers
`asn1util.pas`	BER/DER encode & decode — the bytes under LDAP and SNMP
`synaip.pas`	IP-address parsing and formatting (IPv4 and IPv6)
`synamisc.pas`	Host/system info: DNS servers, local IPs, proxy settings, Wake-on-LAN

How they fit together

None of these units depend on a socket. `synautil` depends on nothing but the RTL; `synaip` uses `synautil`; `asn1util` is self-contained; `synamisc` reaches into the OS. That independence is the point — the protocol classes are built *on top* of these functions, so anything a protocol can do to a string or an address, you can do too, in isolation, without instantiating anything.

- **`synautil` is the workhorse.** Almost every other unit in Synapse uses it. When you parse a MIME header, decode a `Date:` field, or `Fetch` tokens off a server response, you are calling `synautil`. It is the one to learn first.
- **`asn1util` is a translation layer.** ASN.1 BER is how LDAP and SNMP put structured data on the wire. The unit is small — a handful of `ASNEnc*` / `ASNItem` functions — because BER itself is a small, recursive grammar. You rarely call it directly (the LDAP and SNMP classes do), but understanding it demystifies both protocols.
- **`synaip` is pure conversion.** No I/O — it turns `'192.168.0.1'` into an integer and back, validates addresses, expands short-form IPv6. It never resolves a name (that is DNS, and lives elsewhere); `IsIP` explicitly rejects symbolic names.
- **`synamisc` asks the operating system questions.** Its answers (the system's DNS servers, every local IP, the configured proxy) come from Windows registry / IP Helper calls or from `/etc/resolv.conf`-style probing on POSIX, so the same function name returns platform-specific truth.

A note on strings

Everything here works with `AnsiString` — byte strings, not UTF-16. Synapse treats the wire as bytes and these helpers do too: `StrToHex`, `ASNObject`, `IPToID`, and the `Fetch` family all move

raw octets. That is deliberate and it is why the same code compiles on FPC and every Delphi from 7 up.

The rest of the chapter: the `synautil` toolbox, ASN.1 with `asn1util`, and IP and host helpers.

The `synautil` Toolbox

`synautil.pas` is the single most-used non-socket unit in Synapse. It is a flat collection of functions — no classes — that the protocol code leans on for string surgery, date formatting, and byte packing. This page walks the ones you will actually reach for. Every signature below is copied from the unit's interface section.

Add it to your `uses` and call directly:

```
uses synautil;
```

Splitting strings

The two most-called functions in the whole unit split a string on a delimiter:

```
{:Returns a portion of the "Value" string located to the left of the "Delimiter"
 string. If a delimiter is not found, results is original string.}
function SeparateLeft(const Value, Delimiter: string): string;

{:Returns the portion of the "Value" string located to the right of the
 "Delimiter" string. If a delimiter is not found, results is original string.}
function SeparateRight(const Value, Delimiter: string): string;
```

Both search for the *first* occurrence of `Delimiter`. If it is absent, `SeparateLeft` returns the whole string and `SeparateRight` also returns the whole string — so pair them carefully.

```
host := SeparateLeft('example.org:443', ':');   // 'example.org'
port := SeparateRight('example.org:443', ':');  // '443'

// No delimiter present:
SeparateLeft('plainhost', ':');                 // 'plainhost'
SeparateRight('plainhost', ':');                // 'plainhost'
```

`GetBetween` — respects nesting

```
{:Get string between PairBegin and PairEnd. This function respect nesting.}
function GetBetween(const PairBegin, PairEnd, Value: string): string;
```

Unlike a naive left/right split, `GetBetween` matches balanced pairs:

```
GetBetween('(', ')', 'Hi! (hello(yes!))');   // 'hello(yes!)'
```

The `Fetch` family — consume tokens left to right

`Fetch` is destructive: it removes the returned token *and* the delimiter from `Value` (a `var` parameter), so you can loop until the string is empty.

```
{:Fetch string from left of Value string.}
function Fetch(var Value: string; const Delimiter: string): string;

{:Like fetch, but working with binary strings, not with text.}
function FetchBin(var Value: string; const Delimiter: string): string;

{:Fetch string from left of Value string. This function ignore delimiters inside
 quotations.}
function FetchEx(var Value: string; const Delimiter, Quotation: string): string;
```

`Fetch` trims surrounding spaces from the token; `FetchBin` does not (it is for binary data where every byte matters); `FetchEx` skips delimiters that fall inside a quoted run.

```
var
  s, tok: string;
begin
  s := 'alice, bob, carol';
  while s <> '' do
  begin
    tok := Fetch(s, ',');     // 'alice', then 'bob', then 'carol'
    Writeln(tok);
  end;
end;
```

Trimming — spaces only

Standard `Trim` also strips control characters. Synapse's variants remove *only* spaces, which matters when a protocol field may legitimately end in a tab or a control byte:

```
{:Like TrimLeft, but remove only spaces, not control characters!}
function TrimSPLeft(const S: string): string;
{:Like TrimRight, but remove only spaces, not control characters!}
function TrimSPRight(const S: string): string;
{:Like Trim, but remove only spaces, not control characters!}
function TrimSP(const S: string): string;
```

Dates and times

Synapse speaks the date formats the protocols require. The two you will use most are the RFC-822 formatter (for mail and HTTP headers) and the general decoder.

```
{:Returns current time in format defined in RFC-822. ... (Example
 'Fri, 15 Oct 1999 21:14:56 +0200')}
function Rfc822DateTime(t: TDateTime): string;

{:Same as Rfc822DateTime, but GMT timezone is used.}
function Rfc822DateTimeGMT(t: TDateTime): string;
```

```
{:Returns date and time in format defined in RFC-3339 "yyyy-mm-ddThh:nn:ss.zzz"}
function Rfc3339DateTime(t: TDateTime): string;
```

Formatting a header value:

```
Writeln('Date: ', Rfc822DateTime(Now));
// Date: Sat, 11 Jul 2026 14:03:22 +0200
```

Going the other way, `DecodeRfcDateTime` is the forgiving parser — it accepts RFC-822/1123, RFC-850, and C `asctime()` shapes, and applies the timezone correction so the result is always a comparable `TDateTime`:

```
{:Decode various string representations of date and time to TDateTime type.
 This function do all timezone corrections too!}
function DecodeRfcDateTime(Value: string): TDateTime;
```

```
dt := DecodeRfcDateTime('Sun, 06 Nov 1994 08:49:37 GMT');
dt := DecodeRfcDateTime('Sunday, 06-Nov-94 08:49:37 GMT');
dt := DecodeRfcDateTime('Sun Nov  6 08:49:37 1994');   // all three parse
```

Supporting pieces you can call on their own:

```
{:Return your timezone bias from UTC time in minutes.}
function TimeZoneBias: integer;
{:Return your timezone bias ... in string representation like "+0200".}
function TimeZone: string;
{:Decode three-letter month name to month number (0 if no match).}
function GetMonthNumber(Value: String): integer;
{:Decode time from string with ':' separator ("hh:mm" or "hh:mm:ss").}
function GetTimeFromStr(Value: string): TDateTime;
{:Decode TimeZone string (CEST, GMT, +0200, ...) to offset.}
function DecodeTimeZone(Value: string; var Zone: integer): Boolean;
```

Header lists

When you have a `TStringList` of raw `Name: value` header lines, these convert in place to and from the `Name=value` form that `TStringList.Values` understands:

```
{:Convert lines in stringlist from 'name: value' form to 'name=value' form.}
procedure HeadersToList(const Value: TStrings);
{:Convert lines in stringlist from 'name=value' form to 'name: value' form.}
procedure ListToHeaders(const Value: TStrings);
```

Related, for pulling a single parameter out of a header value:

```
{:Returns parameter value from string in format:
 parameter1="value1"; parameter2=value2}
function GetParameter(const Value, Parameter: string): string;
```

```
charset := GetParameter('text/html; charset="utf-8"', 'charset');  // 'utf-8'
```

Email address extraction

```
{:Returns only the e-mail portion of an address from the full address format.}
function GetEmailAddr(const Value: string): string;
{:Returns only the description part from a full address format.}
function GetEmailDesc(Value: string): string;
```

```
GetEmailAddr('"someone" <nobody@somewhere.com>');  // 'nobody@somewhere.com'
GetEmailDesc('"someone" <nobody@somewhere.com>');  // 'someone'
```

Hex, binary, and byte packing

```
{:Returns hexadecimal digits for the bytes in "Value".}
function StrToHex(const Value: Ansistring): string;
{:Returns a string of binary "Digits" representing "Value".}
function IntToBin(Value: Integer; Digits: Byte): string;
{:Returns an integer equivalent of the binary string in "Value".}
function BinToInt(const Value: string): Integer;
```

The `Code*`/`Decode*` pair packs integers into big-endian byte strings — handy for binary protocols:

```
{:Return two characters representing the value in byte format. (High-endian)}
function CodeInt(Value: Word): Ansistring;
{:Decodes two characters at "Index" of "Value" to a Word.}
function DecodeInt(const Value: Ansistring; Index: Integer): Word;
{:Return four characters representing the value in byte format. (High-endian)}
function CodeLongInt(Value: LongInt): Ansistring;
{:Decodes four characters at "Index" of "Value" to a LongInt.}
function DecodeLongInt(const Value: Ansistring; Index: Integer): LongInt;
```

```
StrToHex(CodeInt(258));      // '0102'  (two big-endian bytes)
DecodeInt(CodeInt(258), 1);  // 258     (Index is 1-based)
```

Line-terminator and quoting helpers

```
{:Return position of string terminator in string. Possible terminators are:
 CRLF, LFCR, CR, LF. The one found is returned in Terminator.}
function PosCRLF(const Value: AnsiString; var Terminator: AnsiString): integer;

{:Remove quotation from Value string. If not quoted, returns it unchanged.}
function UnquoteStr(const Value: string; Quote: Char): string;
{:Quote Value string. If Value contains Quote chars, they are doubled.}
function QuoteStr(const Value: string; Quote: Char): string;

{:Replaces all "Search" found within "Value" with "Replace".}
function ReplaceString(Value, Search, Replace: AnsiString): AnsiString;
{:Like POS, but from the right side of Value.}
function RPos(const Sub, Value: String): Integer;
{:Return count of Chr in Value string.}
function CountOfChar(const Value: string; Chr: char): integer;
```

> **Note on `CRLF`.** The `CR`, `LF`, and `CRLF` constants themselves are not declared in `synautil` — they live in `blcksock.pas` (`CRLF = CR + LF`). `synautil` gives you `PosCRLF` to *find* any of the four line terminators in a buffer, but for the literal constant you uses `blcksock`.

URL parsing

```
{:Parses a URL to its various components.}
function ParseURL(URL: string; var Prot, User, Pass, Host, Port, Path,
  Para: string): string;

var Prot, User, Pass, Host, Port, Path, Para: string;
begin
  ParseURL('http://user:pw@example.org:8080/path?q=1',
    Prot, User, Pass, Host, Port, Path, Para);
  // Prot='http' User='user' Pass='pw' Host='example.org'
  // Port='8080' Path='/path' Para='q=1'
end;
```

Streams and temp files

```
{:Read a string of "len" bytes from a stream.}
function ReadStrFromStream(const Stream: TStream; len: integer): AnsiString;
{:Write a string to a stream.}
procedure WriteStrToStream(const Stream: TStream; Value: AnsiString);
{:Return a unique temp-file name in "Dir" with the given "prefix".}
function GetTempFile(const Dir, prefix: String): String;
```

IP validation lives next door

`synautil` does *not* define `IsIP` / `IsIP6` — those are in synaip, which `synautil` does not even use. If you want to validate an address, reach for that unit.

When to use what

You want to…	Call
Split `host:port`	`SeparateLeft` / `SeparateRight`
Loop tokens off a list, consuming as you go	`Fetch`
Pull text between balanced brackets	`GetBetween`
Format a header date	`Rfc822DateTime`
Parse any RFC date string	`DecodeRfcDateTime`
Read `charset=` out of a Content-Type	`GetParameter`
Turn `Name: value` lines into `Values[]`	`HeadersToList`
Pack an integer big-endian for a binary wire	`CodeInt` / `CodeLongInt`
Hex-dump a byte string	`StrToHex`
Break a URL into pieces	`ParseURL`

ASN.1 BER with `asn1util`

LDAP and SNMP do not send text. They send **ASN.1 BER** — a compact, recursive, type-length-value binary encoding. `asn1util.pas` is Synapse's implementation of that encoding: a small set of functions that turn Pascal values into BER bytes and back. The LDAP (`ldapsend.pas`) and SNMP (`snmpsend.pas`) classes are built directly on it; you rarely call it yourself, but knowing it is what turns those protocols from magic into plumbing.

```
uses asn1util;
```

The TLV idea

Every ASN.1 element is three parts laid end to end:

```
+--------+-----------+-------------------+
|  Type  |  Length   |      Contents     |
| 1 byte | 1+ bytes  |   Length bytes    |
+--------+-----------+-------------------+
```

- **Type** — one byte naming the kind of value (integer, string, sequence…).
- **Length** — how many content bytes follow. Short lengths (`< 128`) fit in one byte; longer ones use a multi-byte form flagged by the high bit.
- **Contents** — the value, whose interpretation depends on Type. For the *constructed* types (like `SEQUENCE`) the contents are themselves more TLV elements — that recursion is the whole grammar.

The type constants

`asn1util` defines the BER type tags it understands:

```
ASN1_BOOL      = $01;
ASN1_INT       = $02;
ASN1_OCTSTR    = $04;   // octet string
ASN1_NULL      = $05;
ASN1_OBJID     = $06;   // object identifier (an OID)
ASN1_ENUM      = $0a;
ASN1_SEQ       = $30;   // sequence (constructed)
ASN1_SETOF     = $31;   // set-of  (constructed)
ASN1_IPADDR    = $40;   // SNMP application types below
ASN1_COUNTER   = $41;
ASN1_GAUGE     = $42;
ASN1_TIMETICKS = $43;
ASN1_OPAQUE    = $44;
ASN1_COUNTER64 = $46;
```

The `$40`-range tags are SNMP's application-specific types; the rest are universal ASN.1.

Encoding

The workhorse is `ASNObject` — it wraps any content in its Type and Length, producing a finished element:

```
{:Encodes ASN.1 object to binary form.}
function ASNObject(const Data: AnsiString; ASNType: Integer): AnsiString;
```

Its whole body is the TLV definition made literal:

```
Result := AnsiChar(ASNType) + ASNEncLen(Length(Data)) + Data;
```

You build the *content* with the typed encoders, then wrap it:

```
{:Encodes a signed integer to ASN.1 binary}
function ASNEncInt(Value: Int64): AnsiString;
{:Encodes unsigned integer into ASN.1 binary}
function ASNEncUInt(Value: Integer): AnsiString;
{:Encodes the length of an ASN.1 element to binary.}
function ASNEncLen(Len: Integer): AnsiString;
{:Encodes an OID item to binary form.}
function ASNEncOIDItem(Value: Int64): AnsiString;
```

Encoding an integer element:

```
var
  elem: AnsiString;
begin
  elem := ASNObject(ASNEncInt(42), ASN1_INT);
  // bytes: $02 $01 $2A  (type=INT, length=1, value=42)
  Writeln(StrToHex(elem));   // '02012a'  (StrToHex from synautil)
end;
```

Because `SEQUENCE` content is just concatenated elements, you nest by wrapping a concatenation:

```
var
  body, seq: AnsiString;
begin
  body := ASNObject(ASNEncInt(1),        ASN1_INT)
        + ASNObject('hello',             ASN1_OCTSTR);
  seq  := ASNObject(body, ASN1_SEQ);
  // seq is a SEQUENCE containing an INTEGER and an OCTET STRING
end;
```

Decoding

Decoding walks the buffer with a **`var Start` cursor** — each call advances it past the element it read, so you decode a sequence by calling repeatedly:

```
{:Beginning with the "Start" position, decode the ASN.1 item of the next element
 in "Buffer". Type of item is stored in "ValueType."}
function ASNItem(var Start: Integer; const Buffer: AnsiString;
  var ValueType: Integer): AnsiString;

{:Decodes length of next element in "Buffer" from the "Start" position.}
```

```
function ASNDecLen(var Start: Integer; const Buffer: AnsiString): Integer;
{:Decodes an OID item of the next element from the "Start" position.}
function ASNDecOIDItem(var Start: Integer; const Buffer: AnsiString): Int64;
```

`ASNItem` returns the decoded value as a string and reports its BER type through `ValueType`. `Start` is **1-based** (Pascal string indexing):

```
var
  pos, vt: Integer;
  value: AnsiString;
begin
  pos := 1;
  value := ASNItem(pos, elem, vt);
  // for the '02012a' buffer above:
  //   vt    = ASN1_INT ($02)
  //   value = '42'   (integer types come back as their decimal string)
  //   pos   = 4      (advanced past the element)
end;
```

For a `SEQUENCE`, first decode the outer element (which for constructed types returns the raw inner bytes), then loop `ASNItem` over those with a fresh cursor.

OIDs and MIBs

SNMP object identifiers are dotted-number MIB strings like `1.3.6.1.2.1.1.1.0`. `asn1util` converts between the string and its BER binary form:

```
{:Encodes an MIB OID string to binary form.}
function MibToId(Mib: String): AnsiString;
{:Decodes MIB OID from binary form to string form.}
function IdToMib(const Id: AnsiString): String;
```

```
var
  oid: AnsiString;
begin
  oid := MibToId('1.3.6.1.2.1.1.1.0');   // BER-encoded OID bytes
  Writeln(IdToMib(oid));                  // '1.3.6.1.2.1.1.1.0'  (round-trip)
end;
```

To place an OID on the wire, wrap it as an `ASN1_OBJID` element:

```
oidElem := ASNObject(MibToId('1.3.6.1.2.1.1.1.0'), ASN1_OBJID);
```

Debugging

When a decode goes wrong, dump the buffer in human-readable form:

```
{:Convert ASN.1 BER encoded buffer to human readable form for debugging.}
function ASNdump(const Value: AnsiString): AnsiString;
```

`ASNdump` walks the TLV structure and prints each element's type and value — the fastest way to see what an LDAP or SNMP packet actually contains.

Summary

You want to…	Call
Wrap content as a typed element	`ASNObject(data, ASN1_*)`
Encode a signed / unsigned integer	`ASNEncInt` / `ASNEncUInt`
Encode an OID from a MIB string	`MibToId`
Read the next element from a buffer	`ASNItem(Start, buf, vt)`
Turn an OID back into dotted form	`IdToMib`
Inspect a packet while debugging	`ASNdump`

You will most often *see* these functions inside `ldapsend.pas` and `snmpsend.pas` rather than call them — but every LDAP filter and every SNMP varbind on the wire is exactly this: `ASNObject` calls nested inside an `ASN1_SEQ`, unwound on the far end by `ASNItem`.

IP Addresses and Host Info

Two small units cover the address end of Synapse. `synaip.pas` is pure conversion — strings to addresses and back, no I/O. `synamisc.pas` asks the operating system what it knows: your DNS servers, your local IPs, your proxy. Between them they answer "is this a valid address?", "what does it look like in binary?", and "what is my machine configured to use?".

`synaip` — parse and format addresses

```
uses synaip;
```

This unit does no networking. It never resolves a name — `IsIP` explicitly rejects symbolic hostnames — it only transforms address *text*.

Validation

```
{:Returns TRUE, if "Value" is a valid IPv4 address. Cannot be a symbolic Name!}
function IsIP(const Value: string): Boolean;
{:Returns TRUE, if "Value" is a valid IPv6 address. Cannot be a symbolic Name!}
function IsIP6(const Value: string): Boolean;
```

```
IsIP('192.168.0.1');    // True
IsIP('example.org');    // False  – a name, not an address
IsIP6('::1');           // True
IsIP6('192.168.0.1');   // False  – that is IPv4
```

A common pattern is to decide whether you already have an address or still need to resolve a name:

```
if IsIP(target) or IsIP6(target) then
  // use it directly
else
  // hand it to DNS
```

IPv4 conversion

```
{:Convert IPv4 address from their string form to binary.}
function StrToIp(value: string): integer;
{:Convert IPv4 address from binary to string form.}
function IpToStr(value: integer): string;
```

```
n := StrToIp('192.168.0.1');   // packed into an integer
s := IpToStr(n);               // '192.168.0.1'  (round-trip)
```

IPv6 conversion

IPv6 addresses are 16 bytes, so they go through a byte-array type, `TIp6Bytes = array [0..15] of Byte`:

```
{:Convert IPv6 address from their string form to binary byte array.}
function StrToIp6(value: string): TIp6Bytes;
{:Convert IPv6 address from binary byte array to string form.}
function Ip6ToStr(value: TIp6Bytes): string;
{:Expand short form of IPv6 address to long form.}
function ExpandIP6(Value: AnsiString): AnsiString;
```

`ExpandIP6` turns the compressed `::` form into the full eight-group form — useful when you need to compare two addresses textually:

```
ExpandIP6('2001:db8::1');
// '2001:0DB8:0000:0000:0000:0000:0000:0001'
```

Reverse form (for PTR / rDNS)

```
{:Returns a string with the "Host" ip address converted to binary form.}
function IPToID(Host: string): Ansistring;
{:Convert IPv4 address to reverse form.}
function ReverseIP(Value: AnsiString): AnsiString;
{:Convert IPv6 address to reverse form.}
function ReverseIP6(Value: AnsiString): AnsiString;
```

`ReverseIP` produces the reversed-octet order that reverse-DNS lookups use:

```
ReverseIP('192.168.0.1');   // '1.0.168.192'
```

`IPToID` returns the raw binary (byte-string) form of an address — the shape DNS records and some protocol fields carry it in.

`synamisc` — what the OS knows

```
uses synamisc;
```

Where `synaip` is pure math, `synamisc` reaches into the operating system. The same function name returns platform-specific truth: on Windows it reads the registry and IP Helper API; on POSIX it probes the system's resolver configuration.

DNS servers

```
{:Autodetect current DNS servers used by the system. If more than one DNS server
 is defined, then the result is comma-delimited.}
function GetDNS: string;
```

```
Writeln('System DNS: ', GetDNS);
// e.g. '192.168.0.1,8.8.8.8'
```

This is the function to call when you want to do your *own* DNS query but need to know which server to ask — Synapse's own resolver uses it for exactly that.

Local IP addresses

```
{:Return all known IP addresses on the local system. Addresses are
 comma-delimited.}
function GetLocalIPs: string;
{:Return all known IP addresses of the required type on the local system.}
function GetLocalIPsFamily(value: TSocketFamily): string;
```

`GetLocalIPs` returns every address the host has; `GetLocalIPsFamily` filters by family (`TSocketFamily`, e.g. IPv4-only or IPv6-only):

```
Writeln('This host: ', GetLocalIPs);
// e.g. '127.0.0.1,192.168.0.42,::1'
```

Proxy settings

Proxy discovery is Windows-oriented (it reads Internet Explorer / WinHTTP config) and returns a record:

```
TProxySetting = record
  Host: string;
  Port: string;
  Bypass: string;
  ResultCode: integer;
  Autodetected: boolean;
end;

{:Read Internet Explorer 5.0+ proxy setting for given protocol. Windows only!}
function GetIEProxy(protocol: string): TProxySetting;

{$IFDEF MSWINDOWS}
{:Autodetect system proxy setting for a specified URL. Windows only!}
function GetProxyForURL(const AURL: WideString): TProxySetting;
{$ENDIF}
```

`GetProxyForURL` is compiled only on Windows (`{$IFDEF MSWINDOWS}`); on other platforms it does not exist, so guard your calls with the same define.

Wake-on-LAN

A genuinely miscellaneous member — send the magic packet that powers on a sleeping machine:

```
{:Turn on a network computer that supports Wake-on-LAN. You need the MAC
 address; you can also give a target IP (broadcast is used if omitted).}
procedure WakeOnLan(MAC, IP: string);

WakeOnLan('00:11:22:33:44:55', '');   // broadcast on the local network
```

What is *not* here

Neither unit resolves a hostname to an address — there is no `GetIPByName`. Name resolution is DNS, and in Synapse that lives in the socket layer (`synsock` / `blcksock`) and the DNS client (`dnssend.pas`), not in these helpers. `synaip` validates and converts addresses you already have; `synamisc` tells you which DNS server *would* answer such a query. The lookup itself is a socket's job.

Summary

You want to…	Call	Unit
Check a string is a literal IPv4 / IPv6	`IsIP` / `IsIP6`	synaip
Pack / unpack an IPv4 address	`StrToIp` / `IpToStr`	synaip
Pack / unpack an IPv6 address	`StrToIp6` / `Ip6ToStr`	synaip
Expand `::` shorthand	`ExpandIP6`	synaip
Build a reverse-DNS name	`ReverseIP` / `ReverseIP6`	synaip
Find the system’s DNS servers	`GetDNS`	synamisc
List this host’s own IPs	`GetLocalIPs`	synamisc
Read the configured proxy (Windows)	`GetIEProxy` / `GetProxyForURL`	synamisc
Power on a machine by MAC	`WakeOnLan`	synamisc

Recipes

This is the section you copy from. Each recipe is a **single common task** — download a file, send a mail, resolve a hostname — answered with a minimal but complete `pascal` snippet you can drop into a program, compile, and adjust. No narrative build-up; just problem, code, and the one gotcha that bites people.

How to use this section

- **Each recipe is self-contained.** The snippet names every unit it needs in `uses` and checks status the Synapse way (`LastError` / a `Boolean` result / `ResultCode`). Paste it, change the host and data, compile.
- **HTTPS/TLS is a `uses` line.** Wherever a recipe talks to an `https://` URL or a TLS mail server, add `ssl_openssl` (or `ssl_openssl3`) to `uses` — that one line registers the SSL plugin globally and nothing else in the code changes. See the SSL plugin seam.
- **Snippets are trimmed for clarity.** Error handling is shown once per pattern, not belt-and-braces on every line. For the *why* behind a class, and its full property surface, follow the cross-link to its chapter.
- **The Synapse rhythm.** Create the object, call one method, check status, read the results off properties, free the object. Every recipe is a variation on that.

The recipes

Web — HTTP with `THTTPSend` and the `httpsend` one-liners. - Download a URL to a file - POST form data - Fetch JSON over HTTPS

Email — sending with `TSMTPSend`/`TMimeMess`, reading with `TPOP3Send`. - Send a plain-text mail - Send a mail with a file attachment - Fetch and list an inbox

Network tools — DNS, ICMP, and raw sockets. - Resolve a hostname (`TDNSSend`) - Ping a host (`TPINGSend` / `PingHost`) - A minimal TCP client and a threaded echo server

Serial — talking to a serial port with `TBlockSerial`. - Open a port and do a request/response

Where the depth lives

These recipes are the fast path. When you need the full API — headers, cookies, keep-alive, streaming receives, the blocking model — the chapters have it:

- HTTP — THTTPSend
- SMTP — TSMTPSend
- POP3 — TPOP3Send
- MIME messages

- Socket classes
- Serial ports
- Synapse in Two Hours — the guided on-ramp

Recipes — Web

HTTP tasks with `httpsend`. The module-level helpers (`HttpGetText`, `HttpGetBinary`, `HttpPostURL`) cover the common cases in one line; drop to `THTTPSend` when you need headers or the status code. Full API in THTTPSend — talking HTTP.

Download a URL to a file

Problem: fetch a remote resource (image, archive, anything binary) and save it to disk.

```
program Download;

{$mode objfpc}{$H+}

uses
  Classes,
  httpsend, ssl_openssl;   // ssl_openssl -> https:// works

var
  FileStream: TFileStream;
begin
  FileStream := TFileStream.Create('synapse.zip', fmCreate);
  try
    if HttpGetBinary('https://example.com/synapse.zip', FileStream) then
      WriteLn('Saved ', FileStream.Size, ' bytes')
    else
      WriteLn('Download failed');
  finally
    FileStream.Free;
  end;
end.
```

`HttpGetBinary(const URL: string; const Response: TStream): Boolean` streams the body straight into any `TStream`, so it never buffers the whole file in memory. **Gotcha:** a `True` return only means a reply arrived — for a broken link the server may hand you a `404` page as the body. If that matters, use the object and check `ResultCode` (next recipes).

POST form data

Problem: submit an HTML form (`application/x-www-form-urlencoded`).

```
program PostForm;
```

```
{$mode objfpc}{$H+}

uses
  Classes,
  httpsend, synacode, synautil;   // synacode: EncodeURLElement; synautil: ReadStrFromStream

var
  Reply: TMemoryStream;
  FormData: string;
begin
  Reply := TMemoryStream.Create;
  try
    // key=value&key=value, with every value percent-escaped.
    FormData := 'user=' + EncodeURLElement('Alice') +
                '&city=' + EncodeURLElement('São Paulo');

    if HttpPostURL('http://httpbin.org/post', FormData, Reply) then
    begin
      Reply.Position := 0;
      WriteLn(ReadStrFromStream(Reply, Reply.Size));
    end
    else
      WriteLn('POST failed');
  finally
    Reply.Free;
  end;
end.
```

`HttpPostURL(const URL, URLData: string; const Data: TStream): Boolean` sets the `Content-Type` to `application/x-www-form-urlencoded`, sends `URLData` as the body, and returns the response in `Data`. **Gotcha:** always run each value through `EncodeURLElement` (from `synacode`) — an unescaped `&` or space in a field corrupts the form.

Fetch JSON over HTTPS

Problem: call a JSON API with custom headers and read both the status code and the body.

```
program FetchJson;

{$mode objfpc}{$H+}

uses
  Classes,
  httpsend, ssl_openssl;

var
  Http: THTTPSend;
  Body: TStringList;
begin
  Http := THTTPSend.Create;
  Body := TStringList.Create;
  try
    Http.Headers.Add('Accept: application/json');
    Http.Headers.Add('Authorization: Bearer my-token');
```

```
    if Http.HTTPMethod('GET', 'https://api.example.com/status') then
    begin
      WriteLn('Status: ', Http.ResultCode, ' ', Http.ResultString);
      Http.Document.Position := 0;             // rewind before reading
      Body.LoadFromStream(Http.Document);      // Document is a TMemoryStream
      WriteLn(Body.Text);
    end
    else
      WriteLn('Transport failed: ', Http.Sock.LastErrorDesc);
  finally
    Body.Free;
    Http.Free;
  end;
end.
```

`HTTPMethod` returns `True` for *any* reply, including `404`/`500` — branch on `Http.ResultCode` for the real outcome. **Gotcha:** `Document` is a `TMemoryStream`, not text; set `Position := 0` before `LoadFromStream`, or you'll read zero bytes. To POST JSON, write the body into `Document`, set `Http.MimeType := 'application/json'`, and call `HTTPMethod('POST', URL)`.

Recipes — Email

Sending with `smtpsend`, composing MIME with `mimemess`, reading with `pop3send`. Depth in SMTP — TSMTPSend, POP3 — TPOP3Send, and MIME messages.

Send a plain-text mail

Problem: fire off one text email without touching the SMTP object.

```
program SendMail;

{$mode objfpc}{$H+}

uses
  Classes, smtpsend;

var
  Msg: TStringList;
begin
  Msg := TStringList.Create;
  try
    Msg.Add('Hello from Synapse.');
    Msg.Add('This is the body.');

    // SendTo(MailFrom, MailTo, Subject, SMTPHost, MailData): Boolean
    if SendTo('me@example.com', 'you@example.com',
              'Test message', 'mail.example.com', Msg) then
      WriteLn('Sent')
    else
      WriteLn('Send failed');
  finally
    Msg.Free;
  end;
end.
```

`SendTo` builds the headers, connects, and delivers in one call. **Gotcha:** it takes **no** username/password — it's for open/relay-by-IP servers. For authenticated SMTP use `SendToEx(..., Username, Password)`, or the `TSMTPSend` object (`FullSSL := True` for SMTPS on port 465, or `ssl_openssl` in `uses` + STARTTLS for 587).

Send a mail with an attachment

Problem: build a multipart message with a text body and a file attachment, then hand it to `TSMTPSend`.

```
program SendAttachment;

{$mode objfpc}{$H+}

uses
  Classes, mimemess, mimepart, smtpsend;

var
  Mime: TMimeMess;
  Body: TStringList;
  SMTP: TSMTPSend;
begin
  Mime := TMimeMess.Create;
  Body := TStringList.Create;
  try
    Body.Add('Report attached.');

    Mime.Header.From := 'me@example.com';
    Mime.Header.ToList.Add('you@example.com');
    Mime.Header.Subject := 'Monthly report';

    Mime.AddPartText(Body, nil);                    // the text body
    Mime.AddPartBinaryFromFile('report.pdf', nil);  // the attachment
    Mime.EncodeMessage;                             // -> Mime.Lines

    SMTP := TSMTPSend.Create;
    try
      SMTP.TargetHost := 'mail.example.com';
      SMTP.Username := 'me@example.com';
      SMTP.Password := 'secret';
      if SMTP.Login and
         SMTP.MailFrom('me@example.com', Length(Mime.Lines.Text)) and
         SMTP.MailTo('you@example.com') and
         SMTP.MailData(Mime.Lines) then
        WriteLn('Sent')
      else
        WriteLn('SMTP error: ', SMTP.ResultString);
      SMTP.Logout;
    finally
      SMTP.Free;
    end;
  finally
    Body.Free;
    Mime.Free;
  end;
end.
```

Gotcha: you must call `Mime.EncodeMessage` **before** reading `Mime.Lines` — that's the step that renders the parts into the encoded message. The recipient list on the envelope (`MailTo`) is separate from the `To:` header (`Header.ToList`); set both. For an authenticated/TLS server,

add `ssl_openssl` to `uses` and set `SMTP.FullSSL := True` (or STARTTLS) as the SMTP chapter shows.

Fetch and list an inbox

Problem: log into POP3, report how many messages are waiting, and show the first line of each.

```
program ListInbox;

{$mode objfpc}{$H+}

uses
  Classes, pop3send;

var
  POP3: TPOP3Send;
  i: Integer;
begin
  POP3 := TPOP3Send.Create;
  try
    POP3.TargetHost := 'mail.example.com';
    POP3.Username := 'me@example.com';
    POP3.Password := 'secret';

    if not POP3.Login then
    begin
      WriteLn('Login failed: ', POP3.ResultString);
      Exit;
    end;

    POP3.Stat;                                  // fills StatCount / StatSize
    WriteLn(POP3.StatCount, ' messages, ', POP3.StatSize, ' bytes');

    for i := 1 to POP3.StatCount do
      if POP3.Retr(i) then                      // full message -> FullResult
        WriteLn(i, ': ', POP3.FullResult[0]);

    POP3.Logout;
  finally
    POP3.Free;
  end;
end.
```

`Stat` populates `StatCount` and `StatSize`; messages are numbered `1..StatCount` and their text lands in the `FullResult` string list after `Retr`. **Gotcha:** `Retr` downloads the *whole* message — for just the headers use `Top(Index, 0)`, which is far cheaper on a big mailbox. For POP3S add `ssl_openssl` to `uses` and set `POP3.FullSSL := True` (port 995).

Recipes — Network tools

DNS lookups, ICMP pings, and raw TCP. The socket layer under all of it is the socket classes; the client/server walkthrough is Synapse in Two Hours.

Resolve a hostname

Problem: ask a DNS server for the A records of a name.

```
program Resolve;

{$mode objfpc}{$H+}

uses
  Classes, dnssend;

var
  DNS: TDNSSend;
  Answers: TStringList;
  i: Integer;
begin
  DNS := TDNSSend.Create;
  Answers := TStringList.Create;
  try
    DNS.TargetHost := '8.8.8.8';                 // the DNS server to query

    // QTYPE_A = IPv4 address records (QTYPE_AAAA for IPv6, QTYPE_MX for mail).
    if DNS.DNSQuery('example.com', QTYPE_A, Answers) then
      for i := 0 to Answers.Count - 1 do
        WriteLn(Answers[i])
    else
      WriteLn('Query failed, RCode=', DNS.RCode);
  finally
    Answers.Free;
    DNS.Free;
  end;
end.
```

`DNSQuery(Name, QType, Reply)` sends over UDP and fills `Reply` with one value per line. **Gotcha:** you must set `TargetHost` to an actual resolver — Synapse does not read your system's `/etc/resolv.conf`. For mail servers specifically, `GetMailServers(const DNSHost, Domain: AnsiString; const Servers: TStrings)` queries MX and returns the hosts already sorted by preference.

Ping a host

Problem: measure round-trip time to a host with ICMP.

```
program Ping;

{$mode objfpc}{$H+}

uses
  pingsend;

var
  Ms: Integer;
begin
  Ms := PingHost('example.com');   // round-trip in ms, or -1 on failure
  if Ms >= 0 then
    WriteLn('Reply in ', Ms, ' ms')
  else
    WriteLn('No reply');
end.
```

`PingHost` wraps `TPINGSend`, returning the round-trip time in milliseconds or `-1` if there's no reply. **Gotcha:** ICMP needs a raw socket, so on Unix this must run as root (or with `CAP_NET_RAW`). Use the `TPINGSend` object directly when you want to set `Timeout`/`TTL` or read `ReplyFrom` and `PingTime`.

A minimal TCP client

Problem: connect, send a line, read the reply — the bare socket pattern.

```
program TcpClient;

{$mode objfpc}{$H+}

uses
  blcksock;

var
  Sock: TTCPBlockSocket;
begin
  Sock := TTCPBlockSocket.Create;
  try
    Sock.Connect('example.com', '80');
    if Sock.LastError <> 0 then
    begin
      WriteLn('Connect failed: ', Sock.LastErrorDesc);
      Exit;
    end;
    Sock.SendString('GET / HTTP/1.0' + CRLF + CRLF);  // CRLF is exported by blcksock
    WriteLn(Sock.RecvString(5000));                    // first response line
  finally
    Sock.Free;   // destructor closes the socket
  end;
end.
```

Every call sets `LastError` (`0` = success) rather than raising — check it after `Connect`. `RecvString(Timeout)` returns one CRLF-terminated line without the line ending, and sets `LastError` to a timeout code when the peer goes quiet.

A threaded echo server

Problem: accept many clients at once, one thread per connection.

```
program EchoServer;

{$mode objfpc}{$H+}

uses
  {$IFDEF UNIX} cthreads, {$ENDIF}
  Classes, blcksock, synsock;   // synsock exports TSocket

type
  TEchoThread = class(TThread)
  private
    FSock: TTCPBlockSocket;
  public
    constructor Create(ClientSocket: TSocket);
    procedure Execute; override;
  end;

constructor TEchoThread.Create(ClientSocket: TSocket);
begin
  FSock := TTCPBlockSocket.Create;
  FSock.Socket := ClientSocket;   // adopt the handle Accept returned
  FreeOnTerminate := True;
  inherited Create(False);        // start immediately
end;

procedure TEchoThread.Execute;
var
  Line: string;
begin
  try
    repeat
      Line := FSock.RecvString(60000);
      if FSock.LastError <> 0 then Break;
      FSock.SendString('you said: ' + Line + CRLF);
    until Terminated;
  finally
    FSock.Free;
  end;
end;

var
  Listener: TTCPBlockSocket;
  Client: TSocket;
begin
  Listener := TTCPBlockSocket.Create;
```

```
  try
    Listener.CreateSocket;
    Listener.SetLinger(True, 10000);
    Listener.Bind('0.0.0.0', '9000');
    if Listener.LastError <> 0 then
    begin
      WriteLn('Bind failed: ', Listener.LastErrorDesc);
      Exit;
    end;
    Listener.Listen;
    WriteLn('Listening on 9000...');
    repeat
      if Listener.CanRead(1000) then           // stay interruptible
      begin
        Client := Listener.Accept;
        if Listener.LastError = 0 then
          TEchoThread.Create(Client);          // fire and forget
      end;
    until False;
  finally
    Listener.Free;
  end;
end.
```

The listener only accepts and delegates; each connection runs plain sequential code in its own thread. **Gotcha:** never share one `TBlockSocket` across threads — give every connection its own, as above. The full walkthrough (including the TLS one-liner) is in Synapse in Two Hours §3.

Recipes — Serial

`TBlockSerial` (in `synaser`) is the same blocking-with-timeouts model as the block sockets, aimed at a serial port instead of a network peer. Full API and platform notes in the serial-ports chapter.

Open a port and do a request/response

Problem: open a serial port, send a command, and read the device's reply.

```
program SerialQuery;

{$mode objfpc}{$H+}

uses
  synaser;

var
  Ser: TBlockSerial;
  Reply: string;
begin
  Ser := TBlockSerial.Create;
  try
    // COM3 on Windows, /dev/ttyUSB0 on Linux — names are auto-translated.
    Ser.Connect('/dev/ttyUSB0');
    if Ser.LastError <> 0 then
    begin
      WriteLn('Open failed: ', Ser.LastErrorDesc);
      Exit;
    end;

    // baud, data bits, parity, stop bits, software flow, hardware flow.
    Ser.Config(9600, 8, 'N', SB1, False, False);

    Ser.SendString('STATUS?' + CRLF);
    Reply := Ser.RecvString(2000);       // one CRLF-terminated line, 2 s timeout
    if Ser.LastError = 0 then
      WriteLn('Device said: ', Reply)
    else
      WriteLn('No reply: ', Ser.LastErrorDesc);
  finally
    Ser.Free;   // destructor closes the port
  end;
```

```
end.
```

`Connect` opens the port (check `LastError`), `Config(baud, bits, parity, stop, softflow, hardflow)` sets the line parameters, and `SendString`/`RecvString` mirror the socket API exactly — `RecvString(Timeout)` returns one CRLF-terminated line without the terminator. **Gotcha:** many devices are slow to wake, so allow a generous receive timeout and, for a fixed terminator other than CRLF, use `RecvTerminated(Timeout, Terminator)`; for raw byte counts use `RecvPacket`/`RecvBufferEx`. For a modem, `ATCommand('AT')` sends a command and returns the response in one call.

Visual Synapse

Where Synapse gives you socket classes and protocol *clients*, **Visual Synapse** is one project *implemented on* Synapse that provides two things Synapse itself does not: **RAD components** and **protocol servers**. It is not an extension of Synapse — just an application of it. It is a separate project; this chapter explains how it relates to the library the rest of this cookbook covers.

> **Live project:** https://github.com/yoctobyte/visualsynapse

What it adds over raw Synapse

1. **RAD components.** Thin, drop-on-form wrappers around Synapse's client protocols (HTTP, UDP, DNS, ICMP, TCP, SMTP), with Object-Inspector-editable properties and event callbacks. Commands run on a background thread and a callback fires per result — the blocking-plus-threads model packaged as visual components.
2. **Servers.** Synapse ships clients; Visual Synapse adds hand-written protocol *server* implementations — HTTP, FTP, SMTP, telnet, and a SQL server component. Each is a listener that accepts connections and hands every one to its own handler thread, speaking the protocol in straight-line code over a `TTCPBlockSocket` — exactly the pattern the architecture section describes.

It also historically carried **`TPastella`**, an experimental content-addressed peer-to-peer gossip layer built on the same socket foundation.

How it uses Synapse

Visual Synapse depends on Synapse; it does not replace it. The components and servers are built *on* `TBlockSocket` and friends, and they inherit Synapse's seams for free — including the SSL plugin: a Visual Synapse server gains TLS the same way any Synapse socket does, by which `ssl_*` unit is in `uses`.

Status & caveats

Visual Synapse is a 2004–2008 project, revived for preservation. Treat its servers as historical: the 0.60 HTTP server has a known, unpatched directory-traversal, and the code predates modern transport hardening. See the project's own `SECURITY.md` before running anything. For learning the underlying library, the rest of this cookbook — grounded directly in Synapse — is the better guide; this chapter exists to place Visual Synapse in the map.

Appendix — Synapse in Two Hours

Welcome. This is the on-ramp. In one sitting you will write a TCP client, fetch a web page in a single line, run a threaded server, and switch the whole thing to TLS by adding one word to a `uses` clause. Everything here is real code against the real Ararat Synapse API — no pseudocode, no hand-waving. If you know a little Object Pascal and can compile with FPC or Delphi, you have everything you need.

Read the architecture chapter first if you want the *why*; this chapter is the *how*. The one idea to carry in with you: Synapse is **blocking with timeouts**. Every call either finishes its job or returns when your timeout elapses — it never hangs forever, and it reports status through `LastError` rather than throwing. That single decision is what keeps every example below short and linear. See The blocking model & threads for the full story.

---

1. Hello, socket — a minimal TCP client

The workhorse class is `TTCPBlockSocket`, declared in `blcksock.pas`. It gives you TCP behind five methods you will use constantly:

- `Connect(IP, Port: string)` — open the connection (both arguments are strings; `Port` can be a number like `'13'` or a service name).
- `SendString(Data: AnsiString)` — write bytes.
- `RecvString(Timeout: Integer): AnsiString` — read one CRLF-terminated line, returning it *without* the line ending.
- `LastError: Integer` — `0` means success; anything else is the socket error code, with `LastErrorDesc` giving a human-readable string.
- `CloseSocket` — release the connection (the destructor calls it for you too).

Let's talk to a **daytime** service (TCP port 13), one of the oldest and simplest text protocols on the internet: connect, and the server sends back the current date and time as a line of text, then closes. No request needed.

```
program HelloSocket;

{$mode objfpc}{$H+}

uses
  blcksock;

var
  Sock: TTCPBlockSocket;
  Line: string;
begin
```

```
  Sock := TTCPBlockSocket.Create;
  try
    // Connect blocks until the connection is up or ConnectionTimeout elapses.
    Sock.Connect('time.nist.gov', '13');
    if Sock.LastError <> 0 then
    begin
      WriteLn('Connect failed: ', Sock.LastErrorDesc);
      Exit;
    end;

    // Read lines until the server closes and the read times out.
    repeat
      Line := Sock.RecvString(5000);   // wait up to 5 seconds for a line
      if Sock.LastError = 0 then
        WriteLn(Line);
    until Sock.LastError <> 0;

  finally
    Sock.Free;                         // destructor closes the socket
  end;
end.
```

Notice the **discipline**: after *every* operation we check `LastError` before trusting the result. That is the Synapse rhythm. `Connect` doesn't raise on failure — it sets `LastError` and returns, so you decide what to do. `RecvString` returns an empty string on timeout *and* sets `LastError` to `WSAETIMEDOUT`, which is exactly how we know the server has finished and it's time to stop reading.

That is a complete, correct network client. Twenty lines. This is what "blocking with timeouts" buys you: code that reads top-to-bottom like a description of the conversation.

Sending a request

The daytime server talks first. Most protocols want you to speak first. The pattern is identical — `SendString`, then `RecvString`:

```
Sock.Connect('example.com', '80');
if Sock.LastError = 0 then
begin
  Sock.SendString('GET / HTTP/1.0' + CRLF + CRLF);   // CRLF is declared in blcksock
  Line := Sock.RecvString(5000);
  WriteLn(Line);                                      // -> HTTP/1.1 200 OK
end;
```

`CRLF` is a constant Synapse exports from `blcksock` (`CR + LF`), so you never have to remember `#13#10`. You *could* speak the whole of HTTP this way, line by line. You shouldn't — because Synapse already did it for you. That's section 2.

---

2. One line of HTTP — the convenience layer

`httpsend.pas` sits on top of `TTCPBlockSocket` and speaks HTTP so you don't have to. For the common case there is a free function, `HttpGetText`, that does connect, request, read the response, and disconnect — all of it:

```
program OneLineHTTP;

{$mode objfpc}{$H+}

uses
  Classes, httpsend;

var
  Page: TStringList;
begin
  Page := TStringList.Create;
  try
    if HttpGetText('http://example.com/', Page) then
      WriteLn(Page.Text)
    else
      WriteLn('Request failed');
  finally
    Page.Free;
  end;
end.
```

`HttpGetText(const URL: string; const Response: TStrings): Boolean` returns `True` on a successful fetch and fills your `TStrings` with the response body. One call. Compare that to hand-rolling the request, parsing status lines, and handling headers — this is the payoff for the protocol classes.

When you need the details — THTTPSend

`HttpGetText` is a convenience wrapper around the real workhorse, `THTTPSend`. Reach for the class when you need headers, status codes, or anything other than a plain GET. The pattern is: create it, call `HTTPMethod(Method, URL)`, then read the results off its properties:

- `Document: TMemoryStream` — the response body.
- `Headers: TStringList` — request headers going out, response headers coming back.
- `ResultCode: Integer` / `ResultString: string` — the HTTP status.
- `MimeType: string` — content type for the body you send.

```
program HttpGetClass;

{$mode objfpc}{$H+}

uses
  Classes, httpsend;

var
  HTTP: THTTPSend;
  Body: TStringList;
begin
  HTTP := THTTPSend.Create;
  Body := TStringList.Create;
  try
    if HTTP.HTTPMethod('GET', 'http://example.com/') then
    begin
      WriteLn('Status: ', HTTP.ResultCode, ' ', HTTP.ResultString);
      Body.LoadFromStream(HTTP.Document);   // Document is a TMemoryStream
```

```
      WriteLn(Body.Text);
    end
    else
      WriteLn('Failed: ', HTTP.Sock.LastErrorDesc);
  finally
    Body.Free;
    HTTP.Free;
  end;
end.
```

Note `HTTP.Sock` — `THTTPSend` exposes the underlying `TTCPBlockSocket` it drives, so the same `LastError`/`LastErrorDesc` discipline is right there when a request fails at the socket level.

Posting data

For a POST you load the request body into `Document`, set `MimeType`, and call `HTTPMethod('POST', URL)`:

```
program HttpPost;

{$mode objfpc}{$H+}

uses
  Classes, httpsend;

var
  HTTP: THTTPSend;
  Payload: string;
begin
  HTTP := THTTPSend.Create;
  try
    Payload := '{"name":"synapse","stars":9001}';
    HTTP.Document.Write(Pointer(Payload)^, Length(Payload));   // fill the request body
    HTTP.MimeType := 'application/json';

    if HTTP.HTTPMethod('POST', 'http://example.com/api') then
      WriteLn('Server said: ', HTTP.ResultCode, ' ', HTTP.ResultString)
    else
      WriteLn('POST failed: ', HTTP.Sock.LastErrorDesc);
  finally
    HTTP.Free;
  end;
end.
```

`Document` is a `TMemoryStream`, so anything you can write to a stream — a string, a file, binary — becomes your request body. If you just want to submit a URL-encoded form, `httpsend` also exports the one-liner `HttpPostURL(const URL, URLData: string; const Data: TStream): Boolean`.

That's HTTP. The protocol classes (SMTP in `smtpsend.pas`, POP3, IMAP, FTP, and the rest) all follow the same shape: a thin class over a block socket, a handful of methods, status reported the Synapse way.

---

3. Your first server — a threaded echo/line server

A client is one conversation. A **server** is many at once, and here is where the blocking model earns its keep. The recipe:

1. `Bind(IP, Port)` — claim the address.
2. `Listen` — start accepting.
3. `Accept` — block until a client arrives; it returns a raw `TSocket` handle.
4. **Hand that handle to its own thread** and go straight back to `Accept`.

That fourth step is the whole trick. The listener's only job is to accept and delegate. Each accepted connection gets a `TThread` running plain, sequential protocol code — no shared state machine, no re-entrancy puzzles. One connection, one thread, one readable `Execute`.

The per-connection handler thread

```
type
  TEchoThread = class(TThread)
  private
    FSock: TTCPBlockSocket;
  public
    constructor Create(ClientSocket: TSocket);
    procedure Execute; override;
  end;

constructor TEchoThread.Create(ClientSocket: TSocket);
begin
  FSock := TTCPBlockSocket.Create;
  FSock.Socket := ClientSocket;   // adopt the handle Accept handed us
  FreeOnTerminate := True;        // clean itself up when done
  inherited Create(False);        // start running immediately
end;

procedure TEchoThread.Execute;
var
  Line: string;
begin
  try
    FSock.SendString('Welcome. Type something; QUIT to leave.' + CRLF);
    repeat
      Line := FSock.RecvString(60000);     // up to 60 s of idle before we drop them
      if FSock.LastError <> 0 then
        Break;                             // timeout or disconnect
      if Line = 'QUIT' then
        Break;
      FSock.SendString('you said: ' + Line + CRLF);
    until Terminated;
  finally
    FSock.Free;   // closes the connection
  end;
end;
```

Straight-line code. It reads exactly like the protocol it speaks, because a blocking socket lets it. Assigning to the `Socket` property is how you wrap the raw handle that `Accept` returns in a full `TTCPBlockSocket`.

The listener loop

```
program EchoServer;

{$mode objfpc}{$H+}

uses
  {$IFDEF UNIX} cthreads, {$ENDIF}   // FPC needs this unit for threads on Unix
  Classes, blcksock, synsock;        // synsock exports TSocket

  { ... TEchoThread from above ... }

var
  Listener: TTCPBlockSocket;
  ClientHandle: TSocket;
begin
  Listener := TTCPBlockSocket.Create;
  try
    Listener.CreateSocket;
    Listener.SetLinger(True, 10000);
    Listener.Bind('0.0.0.0', '9000');       // all interfaces, port 9000
    if Listener.LastError <> 0 then
    begin
      WriteLn('Bind failed: ', Listener.LastErrorDesc);
      Exit;
    end;
    Listener.Listen;
    WriteLn('Listening on port 9000...');

    repeat
      // CanRead lets the accept loop stay responsive and interruptible.
      if Listener.CanRead(1000) then
      begin
        ClientHandle := Listener.Accept;
        if Listener.LastError = 0 then
          TEchoThread.Create(ClientHandle);   // fire and forget; thread owns it now
      end;
    until False;   // real code: break on a shutdown flag
  finally
    Listener.Free;
  end;
end.
```

`CanRead(1000)` polls the listening socket for a pending connection with a one-second timeout, so the loop wakes up regularly instead of blocking forever in `Accept` — that's your hook for a clean shutdown flag. When a client is waiting, `Accept` returns instantly with its handle, we spin up a `TEchoThread`, and we're back at the top of the loop ready for the next one. The listener never does any real work; the threads do. That is how one small program serves hundreds of simultaneous connections.

> **Two rules for servers.** Give every connection its *own* thread, and never share one `TBlockSocket` across threads. Stay inside those two lines and Synapse's concurrency story stays as simple as the code above.

Connect to it with `telnet localhost 9000`, or point the client from section 1 at it. This is a real, working server.

---

4. Going encrypted — TLS by adding one word

Here is the most elegant trick in the library. You do **not** rewrite your client to speak HTTPS. TLS in Synapse is a *plugin*, selected purely by which unit you put in your `uses` clause. Add one:

```
uses
  Classes, httpsend,
  ssl_openssl;      // <-- this line turns on TLS
```

Just placing `ssl_openssl` (or `ssl_openssl3` for OpenSSL 3.x) in `uses` registers an SSL implementation globally. Now the very same `THTTPSend` code from section 2 handles `https://` URLs — the block socket underneath negotiates TLS on your behalf when the scheme calls for it:

```
if HTTP.HTTPMethod('GET', 'https://example.com/') then   // now works, unchanged
  WriteLn('Secure status: ', HTTP.ResultCode);
```

Nothing else in your program changes. That's the payoff of TLS-as-a-swappable-plugin: the socket classes were written once with a seam for encryption, and the `ssl_*` unit you link decides what fills it.

STARTTLS — upgrading a live connection

Some protocols (SMTP, IMAP, FTP) start in plaintext and upgrade the *existing* connection to TLS mid-stream. Synapse supports this directly: once you're connected, call `SSLDoConnect` on the block socket to perform the TLS handshake in place.

```
Sock.Connect('mail.example.com', '587');
Sock.SendString('STARTTLS' + CRLF);   // tell the server we want to go secure
Sock.RecvString(5000);                // read its go-ahead
Sock.SSLDoConnect;                     // upgrade this live socket to TLS
if Sock.SSL.LastError <> 0 then
  WriteLn('TLS handshake failed: ', Sock.SSL.LastErrorDesc);
```

`SSLDoConnect` (declared on `TTCPBlockSocket` in `blcksock.pas`) runs the handshake over the connection you already have. From that point on your `SendString` and `RecvString` calls are encrypted, with no other change to your code. TLS-level status lives on the socket's `SSL` object (`Sock.SSL.LastError` / `Sock.SSL.LastErrorDesc`).

For the full picture of how the plugin seam works — the global `SSLImplementation` metaclass and why a `uses` line is all it takes — read The SSL plugin seam.

---

5. Where to next

Two hours in, you can open connections, fetch and post over HTTP, run a threaded server, and encrypt any of it. That is the whole shape of Synapse — everything else is more protocols in the same mold.

- **The socket classes** — `TBlockSocket` and its family (TCP, UDP, raw, even serial), timeouts, `CanRead`/`CanWrite`, the receive functions beyond `RecvString` (`RecvTerminated`, `RecvPacket`). Start with The blocking model & threads, then the socket-classes section for the full API surface.

- **The protocol classes** — SMTP (`smtpsend.pas`; try the `SendTo` free function for a fire-and-forget email), POP3, IMAP, FTP, NNTP, LDAP, DNS and more. Each is a thin readable class over a block socket, and each follows the exact rhythm you learned here: create, call a method, check status the Synapse way.
- **The architecture chapters** — start here for the four seams that make all of this so small: the blocking model, the `synsock` OS seam, the SSL plugin, and Synapse's own hand-rolled crypto.

The library rewards curiosity. Because every layer is thin and orthogonal, reading the source of any one class is a short trip — and now you know the vocabulary to follow it.
